San José State University

CMPE 281 Cloud Technologies · Fall 2026

HomeAI Cloud

Intelligent Cloud Platform for Smart Homes

System Analysis and Design Document

Deliverable #1 · Project Option #1

Member	Name	Component
Member 1	Name	C1 IoT Device & House Data Manager
Member 2	Name	C2 AI Media Intelligence & Model Manager
Member 3	Name	C3 Alert Tracking, Monitoring & Notification
Member 4	Name	C4 Cloud Dashboard, User Services & Data Management

Instructor: Jerry Gao, Ph.D.

October 2026

Contents

Document Information **1**
Team and component ownership . . . 1
Rubric traceability . . . 1
Revision history . . . 2

1 Introduction **2**
1.1 Background . . . 2
1.2 Project objectives . . . 2
1.3 Scope . . . 2
1.4 Expected outcomes . . . 3

2 System Requirements and Analysis **3**
2.1 User Groups and Roles . . . 3
2.2 Use-Case Model . . . 4
2.3 Functional Requirements . . . 7
2.4 Non-Functional Requirements . . . 9
2.5 Process Analysis . . . 10
2.6 Constraints and Assumptions . . . 12

3 System Infrastructure and Architectures **12**
3.1 Cloud-Based System Infrastructure . . . 12
3.2 System Component-Oriented Function Architecture . . . 16
3.3 Cloud-Based System Design and Component Interaction Design . . . 19

4 System Database and Data Design **34**
4.1 Relational Database Design (PostgreSQL ERD) . . . 35
4.2 NoSQL Database Design (MongoDB) . . . 45
4.3 Edge-Based Data Buffer (SQLite) . . . 52
4.4 Cloud-Based System Communication Design . . . 53

5 System Dashboard UI Design **60**
5.1 System User Dashboard Architecture . . . 60
5.2 Page and Frame Designs . . . 62
5.3 Navigation Flow . . . 70
5.4 System UI Operation Behavior . . . 72

6 Project Development Plan and Schedule **75**
6.1 Project Team and Roles . . . 75
6.2 Project Schedule (PERT Chart) . . . 76

7 References **80**

List of Figures

1 Use-case model: 7 actors and 26 use cases across components C1–C4 5
2 End-to-end alert process in swimlanes: 15 activities and 2 decisions 11
3 HomeAI Cloud infrastructure on AWS: one VPC, two Availability Zones, three subnet tiers 13
4 Function-oriented tree: 4 components (high-level partition) and 31 sub-functions (low-level partition) 17
5 System functional block diagram: five cloud layers plus edge and external services . 18
6 Load balancing: ALB path routing to five target groups in two AZs, SQS competing consumers, Redis fan-out 22
7 Multi-tenancy: six tenant checks from identity to storage, with pool and silo models 23
8 Component black-box view: inputs and outputs of C1–C4 by data format 26
9 Sequence diagram, steps 1–4: clip ingest to AI detection 28
10 Sequence diagram, steps 5–6: alert, notification, acknowledgment and feedback . . . 30
11 ERD 1: tenancy, users and owner services (8 tables) 36
12 ERD 2: houses, rooms and devices (7 tables) 37
13 ERD 3: AI models and alerts (7 tables) 38
14 MongoDB reference map: 8 collections linked to PostgreSQL ids and S3 objects . . . 48
15 Communication channels between the front-end parts and the back-end cloud 54
16 Authentication sequence: login, token refresh after a 401, WebSocket connect 56
17 Web dashboard architecture: 5 layers, 4 feature modules, 2 transport clients 61
18 Wireframe 1: Login (all roles) 65
19 Wireframe 2: Home Overview (house owner landing page) 66
20 Wireframe 3: Alert Detail (house owner; cloud staff from Alert Monitor) 67
21 Wireframe 4: Device Manager and Add Device dialog (IoT staff landing page) 68
22 Wireframe 5: System Dashboard (cloud staff landing page) 69
23 Wireframe 6: AI Models with the Activate confirmation dialog (cloud staff) 70
24 Navigation flow: three role lanes, 16 sidebar pages, detail frames and dialogs 71
25 Alert card UI states: 5 states, 7 transitions and 1 escalation timer 74
26 PERT network: 22 tasks and 6 milestones, critical path highlighted 78
27 Project timeline drawn from the PERT table, Sep 2 – Dec 16, 2026 79

Document Information

HomeAI Cloud is our intelligent cloud platform for senior smart homes. Virtual cameras and microphones send media to AWS, pre-trained AI models label it, and the resulting alerts reach families, IoT staff and cloud staff through one dashboard.

Item	Detail
Course	CMPE 281 Cloud Technologies, Fall 2026
Instructor	Jerry Gao, Ph.D.
Project	Option #1: Intelligent Cloud Platform for Smart Homes
Deliverable	#1: System analysis and design document
Platform name	HomeAI Cloud
Deployment target	Amazon EC2, AWS us-west-2 (Oregon)

Team and component ownership

Member	Component	Scope
Member 1	C1 IoT Device & House Data Manager	Houses, rooms, virtual cameras and microphones, device status, configuration, test-media associations
Member 2	C2 AI Media Intelligence & Model Manager	Audio and video inference with pre-trained models, labels and confidence scores, model versions and status
Member 3	C3 Alert Tracking, Monitoring & Notification	Alert rules and thresholds, alert generation, status lifecycle, history, notifications
Member 4	C4 Cloud Dashboard, User Services & Data Management	Role-based access, owner service configuration, statistics dashboards, PostgreSQL and MongoDB design, EC2 deployment

Each member owns the UI, backend APIs, data structures, UML, implementation, tests and demo of their component. Open item: add each member's name.

Rubric traceability

Rubric criterion (points)	Where it is addressed
System requirements (1)	§2.1 user groups and roles, §2.2 use cases, §2.3–2.4 functional and non-functional requirements, §2.5 process analysis
System architecture design (1)	§3.1 infrastructure diagram, infrastructure elements, connectivity and user access; §3.2 function tree and functional block diagram
DB system design (1)	§4.1 PostgreSQL ERDs and data dictionary, §4.2 MongoDB collections and sample documents, §4.3 edge SQLite buffer
GUI system design (1)	§5.1 dashboard architecture and design system, §5.2 page table with function buttons and six wireframes, §5.3 navigation flow diagram, §5.4 operation flows
Cloud design (1.5)	§3.3(a): scalability and capacity estimate, load balancing, multi-tenancy, availability (Canvas outline item 4.3)

Rubric criterion (points)	Where it is addressed
High-level function partition & component design (1.5)	§3.2 component dependencies, §3.3(b) black-box views and interaction sequences, §3.3(c) API table, §4.4 communication design (Canvas outline item 4.2)

Revision history

Version	Date	Change
0.1	2026-10-03	First complete draft for team review

1 Introduction

HomeAI Cloud lets families and care staff watch over seniors who live alone. It turns home audio and video into classified events and delivers an alert to the right person within 10 seconds (p95).

1.1 Background

Seniors who live alone often cannot call for help after a fall, a kitchen fire or a break-in. Home cameras stream raw footage, but nobody can watch it all day, and most homes have no system that understands what it hears. The instructor's HomeAI framework fills that gap with sound-based intelligence: a home's sounds carry most of what matters, from a smoke alarm or breaking glass to a scream, a dog barking or the TV left on.

Our platform applies that framework to individual senior houses inside a senior-care community. Each house has virtual microphones and cameras grouped by room. Their media flows to AWS, where pre-trained models label it, rules turn labels into alerts, and a role-based dashboard shows the result: device data → AI analysis → alerts → dashboard.

1.2 Project objectives

1. Collect each house's audio and video data and learn the residents' daily life patterns by room and hour.
2. Detect and classify emergency, safety and security sounds such as smoke alarms, glass breaking, screams, thuds, gunshots and sirens.
3. Detect and recognize resident and animal sounds such as speech, coughing, snoring, dogs and cats; detect people and pets on video.
4. Generate alerts for family members, security staff and emergency handlers, with acknowledgment, escalation and false-positive feedback.
5. Serve three user groups (house owners, IoT staff and cloud staff) through role-based dashboards and statistics reports.
6. Run on Amazon EC2 with load balancing, auto scaling and multi-tenant data isolation.

1.3 Scope

In scope	Out of scope
Individual senior houses within a care community (one tenant per community)	Assisted-living facilities with shared spaces
Virtual cameras and microphones that play assigned test-media clips	Physical device hardware and firmware
Pre-trained models: YAMNet for audio, YOLOv8n for video	Training new models from scratch
Responsive web dashboard for all three roles	Native mobile apps
SMS and email notifications through AWS	Direct integration with 911 dispatch
Service plans and subscriptions as records	Payment processing

1.4 Expected outcomes

Outcome	Owner	How we will show it
Device and house manager for virtual cameras and microphones, with status, configuration and test media	C1	Demo houses with rooms and devices that send clips and heartbeats
AI service that labels audio and video clips with confidence scores and versioned models	C2	A clip uploaded from a device returns labels such as `smoke_alarm 0.94`; switching the active model version changes the result
Alert service with rules, ten alert types, a status lifecycle, history and notifications	C3	A detected `distress_sound` raises a critical alert, sends SMS and email, and escalates after 3 minutes without acknowledgment
Cloud dashboard with role-based access, owner service settings and system statistics	C4	Separate workspaces for `house_owner`, `iot_staff` and `cloud_staff` with live alerts
Integrated deployment on AWS	All	Public HTTPS endpoint on EC2 behind a load balancer, with PostgreSQL, MongoDB, S3 and SQS
Course documents	All	This design (D1), component designs with UML (D2), demo videos (D3), presentation (D4), IEEE-format report (D5)

2 System Requirements and Analysis

HomeAI Cloud serves 3 login roles, 1 notified group without accounts and 3 system actors, captured below as 26 use cases, 41 functional and 14 non-functional requirements.

Requirements follow the IEEE 830 SRS form: an ID, one "The system shall …" statement and a MoSCoW priority. **Must** = needed for the integrated demo (D4, 2026-12-02); **Should** = in the design, built if time allows; **Could** = stretch goal.

2.1 User Groups and Roles

Three login roles are checked by RBAC on every API call; external recipients have no account, and three system actors act through device keys or IAM credentials.

User group	Role code	Description	Goals	Key functions	Data access scope
House owner	`house_owner`	Senior resident or family caregiver who holds the house subscription	Know the resident is safe; act on emergencies fast	Home Overview and live alerts; acknowledge, resolve, mark false positive; configure services, quiet hours, sensitivity; emergency contacts; reports	Own house(s) only, from the `house_ids` JWT claim
IoT staff	`iot_staff`	Tenant's edge-IoT and device team	Keep every virtual device configured, assigned and online	Add, update, delete, configure, enable, disable devices; assign house and room; Wi-Fi settings; test media; device health; test inference	All houses of own tenant
Cloud staff	`cloud_staff`	Platform operators with super-admin rights	Keep the platform, models and rules correct and available	Tenants, users and roles; all houses; AI models and versions; global alert rules; system monitoring; deployments	All tenants
External recipients	none (no login)	Family members, security staff, emergency responders	Learn of an emergency within seconds and confirm receipt	Receive SMS and email; acknowledge through a signed link (30-min expiry)	One alert, through its signed token
Edge Hub	system actor, `X-Device-Key`	Per-house gateway that runs the virtual devices	Deliver every clip and heartbeat, even after an outage	Heartbeat every 30 s; request upload URL; PUT clip to S3; buffer offline in SQLite	Devices of its own house
AI Worker	system actor, IAM role	`ai-worker` SQS consumer in C2	Label each clip inside the latency budget	Consume `media-analysis`; run the active model; write `ai_detections`; send `detection-results`	One clip per message, any tenant (`tenant_id` carried in the message)
Scheduler	system actor, service credentials	Timed jobs inside C1–C4	Run time-based checks with no user action	Offline check (90 s), escalation timer (3 min), inactivity check (4 h), hourly activity-pattern update (C2), daily roll-up to `dashboard_stats_daily`	Platform-wide, one job at a time

2.2 Use-Case Model

The model holds 26 use cases: 14 on the left let staff set up devices, models and rules, and 12 on the right move a clip to a closed alert.

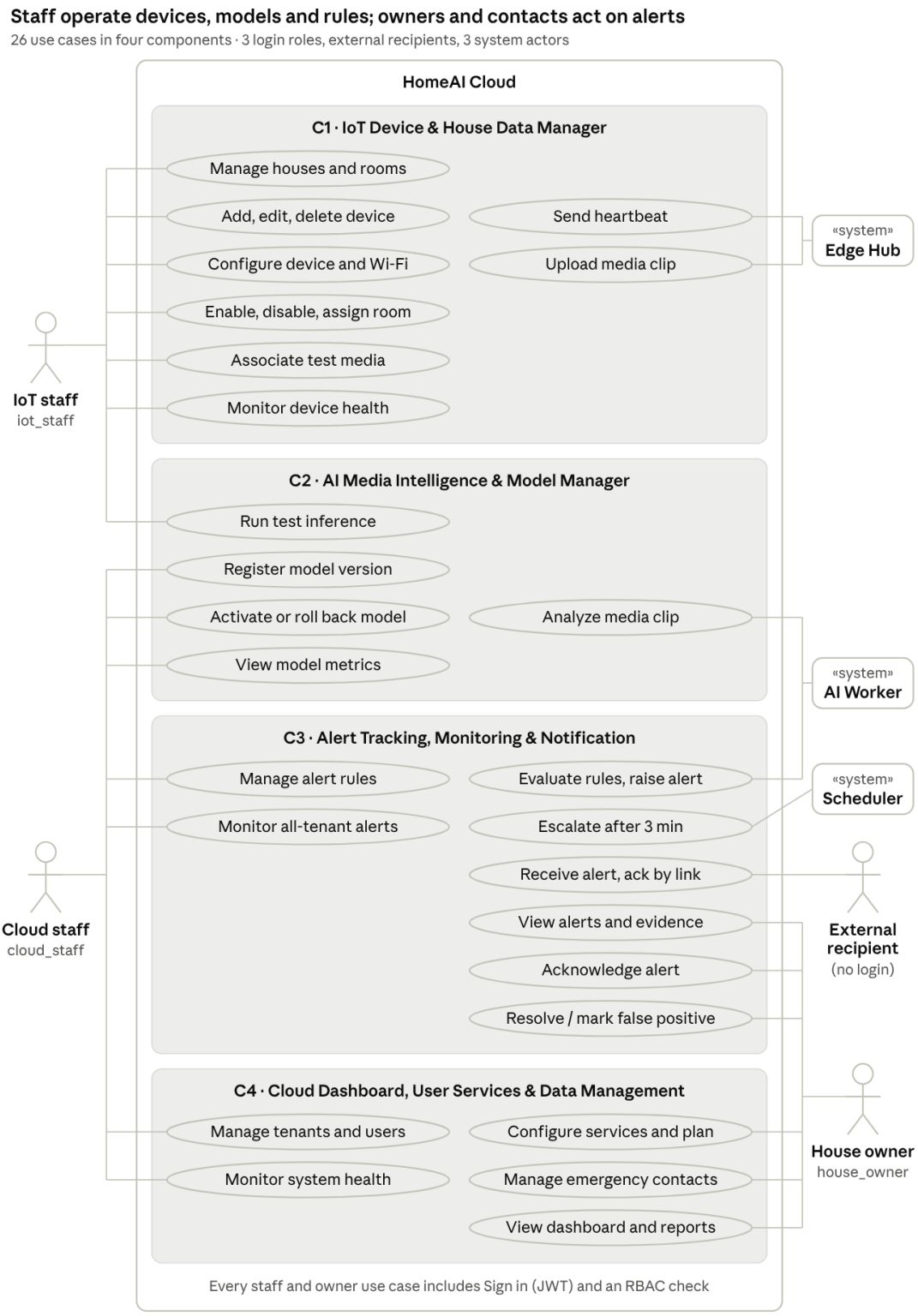


Figure 1: Use-case model: 7 actors and 26 use cases across components C1–C4

Staff on the left configure and monitor; the actors on the right move each clip from upload to a closed alert.

The eight use cases below carry the integrated flow; each member details their own in D2 with UML.

ID	Use case	Primary actor	Trigger	Main flow	Outcome
UC-01	Add and enable a virtual device (C1)	`iot_staff`	A room needs a camera or microphone	Device Manager → Add → set config → assign house and room → link test media → Enable	Device `provisioned` and enabled; device key shown once
UC-02	Upload a media clip (C1)	Edge Hub	Device capture interval elapses	`POST /ingest/media-requests` → PUT clip to S3 → `POST /ingest/media/{media_id}/complete`	`media_events` document `queued`; `media-analysis` message sent
UC-03	Analyze a media clip (C2)	AI Worker	`media-analysis` message	Download → pre-process → run active model version → write `ai_detections`	Labels, confidence and model version stored; `detection-results` sent
UC-04	Evaluate rules and raise an alert (C3)	AI Worker, via queue	`detection-results` message	Load house rules from Redis → match label, `min_confidence`, room, cooldown → insert `alerts` → push and notify	Alert `open` on the dashboard within 10 s p95; contacts notified
UC-05	Handle an alert (C3)	`house_owner`	Alert in the notification bell or Alerts page	Open detail (clip, labels, timeline) → Acknowledge → Resolve or Mark false positive	Status and `alert_status_history` row saved; feedback sent to C2
UC-06	Escalate an unacknowledged alert (C3)	Scheduler	Critical alert still `open` after 3 min	Set `escalated` → notify the next emergency contact by SMS and email	Next contact paged; history row saved
UC-07	Activate or roll back a model version (C2)	`cloud_staff`	New version evaluated, or a regression found	Register version → compare metrics → activate it, or reactivate the previous one	Workers use the active version from the next clip; previous version kept
UC-08	Configure services and contacts (C4)	`house_owner`	Owner changes how the house is watched	Services & Subscription → toggle services, quiet hours, sensitivity → Emergency Contacts	`house_service_settings` and `emergency_contacts` updated; rules apply them from the next detection

2.3 Functional Requirements

The 41 functional requirements are grouped by component: 34 are Must, 6 Should and 1 Could.

C1 — IoT Device & House Data Manager (`device-service`)

ID	Requirement	Priority	Actor
FR-C1-01	The system shall create, update and delete houses and rooms inside the caller's tenant.	Must	`iot_staff`, `cloud_staff`
FR-C1-02	The system shall register a `virtual_camera` or `virtual_microphone` with name, type and a device key stored as a SHA-256 hash.	Must	`iot_staff`
FR-C1-03	The system shall update and delete a device and list devices filtered by house, room, status and type.	Must	`iot_staff`
FR-C1-04	The system shall save each configuration change as a new `device_configs` version: sample rate, resolution, fps, capture interval, sensitivity, Wi-Fi SSID.	Must	`iot_staff`
FR-C1-05	The system shall assign a device to one house and one room and record each change in `audit_logs`.	Must	`iot_staff`
FR-C1-06	The system shall enable and disable a device and reject ingest from a disabled or unassigned device with HTTP 403.	Must	`iot_staff`, Edge Hub
FR-C1-07	The system shall accept a heartbeat every 30 s, mark a device `offline` after 90 s of silence and raise a `device_offline` alert in C3.	Must	Edge Hub, Scheduler
FR-C1-08	The system shall keep a test-media library in `homeai-test-media` and link clips to devices with playback order and a loop flag.	Must	`iot_staff`
FR-C1-09	The system shall issue a pre-signed S3 PUT URL valid 5 min, write a `media_events` document on completion and send a `media-analysis` message.	Must	Edge Hub
FR-C1-10	The system shall show device status and 30 days of telemetry; owners see their own devices read-only.	Should	`iot_staff`, `house_owner`

C2 — AI Media Intelligence & Model Manager (`ai-service`, `ai-worker`)

ID	Requirement	Priority	Actor
FR-C2-01	The system shall resample each audio clip to 16 kHz mono 0.96 s frames, classify it with YAMNet and map AudioSet classes to the 20 home audio labels.	Must	AI Worker
FR-C2-02	The system shall sample video at 1 frame/s and detect person, dog and cat with YOLOv8n on ONNX Runtime.	Must	AI Worker
FR-C2-03	The system shall flag `person_lying` in video frames as a possible fall.	Could	AI Worker
FR-C2-04	The system shall store per clip in `ai_detections` the ranked labels with confidence 0–1, the top label, and the model name and version, after temporal smoothing.	Must	AI Worker

ID	Requirement	Priority	Actor
FR-C2-05	The system shall set `media_events.status = analyzed`, send a `detection-results` message, and move a clip that fails 5 times to the dead-letter queue.	Must	AI Worker
FR-C2-06	The system shall keep a model registry (`ml_models`, `model_versions`) with framework, S3 artifact path, label map and status `staged`, `active` or `deprecated`.	Must	`cloud_staff`
FR-C2-07	The system shall activate one version per model at a time and roll back to the previous version in one action.	Must	`cloud_staff`
FR-C2-08	The system shall run test inference on an uploaded clip or a `media_id` and return labels without raising an alert.	Must	`iot_staff`, `cloud_staff`
FR-C2-09	The system shall report per model version the clips processed, mean and p95 inference time, label counts and false-positive rate.	Should	`cloud_staff`
FR-C2-10	The system shall summarize detections into hourly activity profiles per house in `activity_patterns` to learn the resident's daily routine.	Should	AI Worker
FR-C2-11	The system shall store resolved and false-positive outcomes from C3 on `ai_detections.feedback` for retraining.	Must	`alert-service`

C3 — Alert Tracking, Monitoring & Notification (`alert-service`, `notifier`)

ID	Requirement	Priority	Actor
FR-C3-01	The system shall create, update and delete alert rules per tenant: alert type, severity, `trigger_labels`, `min_confidence`, rooms, time window, consecutive hits, cooldown.	Must	`cloud_staff`
FR-C3-02	The system shall evaluate each `detection-results` message against the house's rules and service settings, and create an `open` alert on a match.	Must	AI Worker
FR-C3-03	The system shall suppress a second alert of the same type and room inside the cooldown and de-duplication window.	Must	AI Worker
FR-C3-04	The system shall push `alert.created` and `alert.updated` events over WebSocket to every open dashboard of that tenant.	Must	`house_owner`, `cloud_staff`
FR-C3-05	The system shall notify contacts by severity: critical and high by in-app, SMS and email; medium by in-app and email; low in-app only.	Must	External recipients
FR-C3-06	The system shall move alerts only along open → acknowledged → resolved, open or acknowledged → escalated → resolved, or to `false_positive`, and reject other changes with HTTP 409.	Must	`house_owner`, External recipients
FR-C3-07	The system shall escalate a critical alert not acknowledged within 3 min to the next emergency contact.	Must	Scheduler
FR-C3-08	The system shall record each status change in `alert_status_history` with actor, time and note, and keep alert history for 2 years.	Must	`house_owner`, `cloud_staff`
FR-C3-09	The system shall report alert counts by type, severity, house and period through `GET /alerts/stats`.	Should	`house_owner`, `cloud_staff`

ID	Requirement	Priority	Actor
FR-C3-10	The system shall send each resolved or false-positive outcome to C2 through `POST /api/v1/detections/{id}/feedback`.	Must	`house_owner`

C4 — Cloud Dashboard, User Services & Data Management (`user-service`, `web-dashboard`)

ID	Requirement	Priority	Actor
FR-C4-01	The system shall authenticate users by email and bcrypt-hashed password and issue JWT access (30 min) and refresh (7 days) tokens.	Must	All login roles
FR-C4-02	The system shall enforce RBAC for `house_owner`, `iot_staff` and `cloud_staff` on every endpoint and page.	Must	All login roles
FR-C4-03	The system shall let cloud staff create and manage tenants, users and role assignments.	Must	`cloud_staff`
FR-C4-04	The system shall let an owner turn detection services on or off and set quiet hours, away or night mode and sensitivity per house.	Must	`house_owner`
FR-C4-05	The system shall subscribe a house to a service plan and keep basic billing records, with no payment processing.	Should	`house_owner`
FR-C4-06	The system shall let an owner add, edit, delete and order emergency contacts with phone and email.	Must	`house_owner`
FR-C4-07	The system shall show role-specific dashboards (Home Overview, System Dashboard, System Health) with statistics cached in Redis for 30 s.	Must	All login roles
FR-C4-08	The system shall export alert reports per house and period as CSV or PDF.	Should	`house_owner`, `cloud_staff`
FR-C4-09	The system shall keep data house-wise: every tenant-owned row and document carries `tenant_id` and `house_id`, and owners query only their `house_ids`.	Must	All login roles
FR-C4-10	The system shall run on AWS EC2 as Docker containers built by GitHub Actions and stored in ECR, applying Alembic migrations at each release.	Must	`cloud_staff`

2.4 Non-Functional Requirements

Fourteen non-functional requirements turn the design targets into testable numbers, led by 10 s p95 alert latency, 300 ms p95 API reads and 99.5 % monthly availability.

ID	Category	Requirement	Target
NFR-01	Performance	The system shall show an alert on the dashboard soon after a clip upload completes.	$\leq$ 10 s p95 for audio clips up to 10 s
NFR-02	Performance	The system shall answer REST read requests quickly at design load.	p95 $<$ 300 ms

ID	Category	Requirement	Target
NFR-03	Scalability	The system shall carry the design load without redesign.	1,000 houses × 6 devices = 6,000 devices; peak 100 clips/s
NFR-04	Scalability	The system shall scale its stateless services horizontally.	API ASG 2–6 × t3.medium at 60 % CPU; AI ASG 1–8 × c6i.xlarge at 20 queued messages per worker
NFR-05	Availability	The system shall keep serving after the loss of one Availability Zone.	99.5 % monthly; 2 AZs; RDS Multi-AZ; MongoDB 3-node replica set; ALB health check every 15 s
NFR-06	Reliability	The system shall lose no clip between the Edge Hub and its detection.	Edge Hub SQLite buffer; SQS visibility 120 s; 5 receives → DLQ; consumers idempotent on `media_id`
NFR-07	Security	The system shall protect data in transit and at rest and hold no static cloud keys.	TLS 1.2+; KMS on S3 and RDS; IAM instance roles; Secrets Manager; bcrypt passwords; device keys as SHA-256 hashes
NFR-08	Security	The system shall authenticate every call and isolate tenants.	JWT access 30 min, refresh 7 days; PostgreSQL row-level security on `tenant_id`; S3 prefix `tenants/{tenant_id}/`; 100 requests/s per tenant, then HTTP 429
NFR-09	Security	The system shall keep an audit trail of staff writes and alert status changes.	`audit_logs` and `alert_status_history`, kept 2 years
NFR-10	Privacy	The system shall keep personal media no longer than needed.	Raw media 30 days in S3, then Glacier 1 year; detections 180 days; telemetry 30 days
NFR-11	Privacy	The system shall turn audio into sound labels only and honour each owner's service choices.	No speech transcription; any detection service can be switched off per house
NFR-12	Usability	The system shall offer an accessible, role-based dashboard.	WCAG 2.1 AA; acknowledge an alert in ≤ 2 clicks from the notification bell; usable at 360 px width
NFR-13	Maintainability	The system shall keep the four services independently buildable, documented and observable.	OpenAPI 3 per service; `/api/v1` versioning; ≥ 70 % unit-test coverage of service logic; CloudWatch logs, metrics and alarms
NFR-14	Deployability	The system shall deploy reproducibly to EC2 from one pipeline.	GitHub Actions → ECR → EC2; demo stack on one t3.large with docker compose; rollback to the previous image tag

2.5 Process Analysis

A clip crosses all 5 lanes in 8 hand-offs before its alert closes, or 10 if it escalates. The 10 s p95 budget runs from upload completion to the alert on the dashboard.

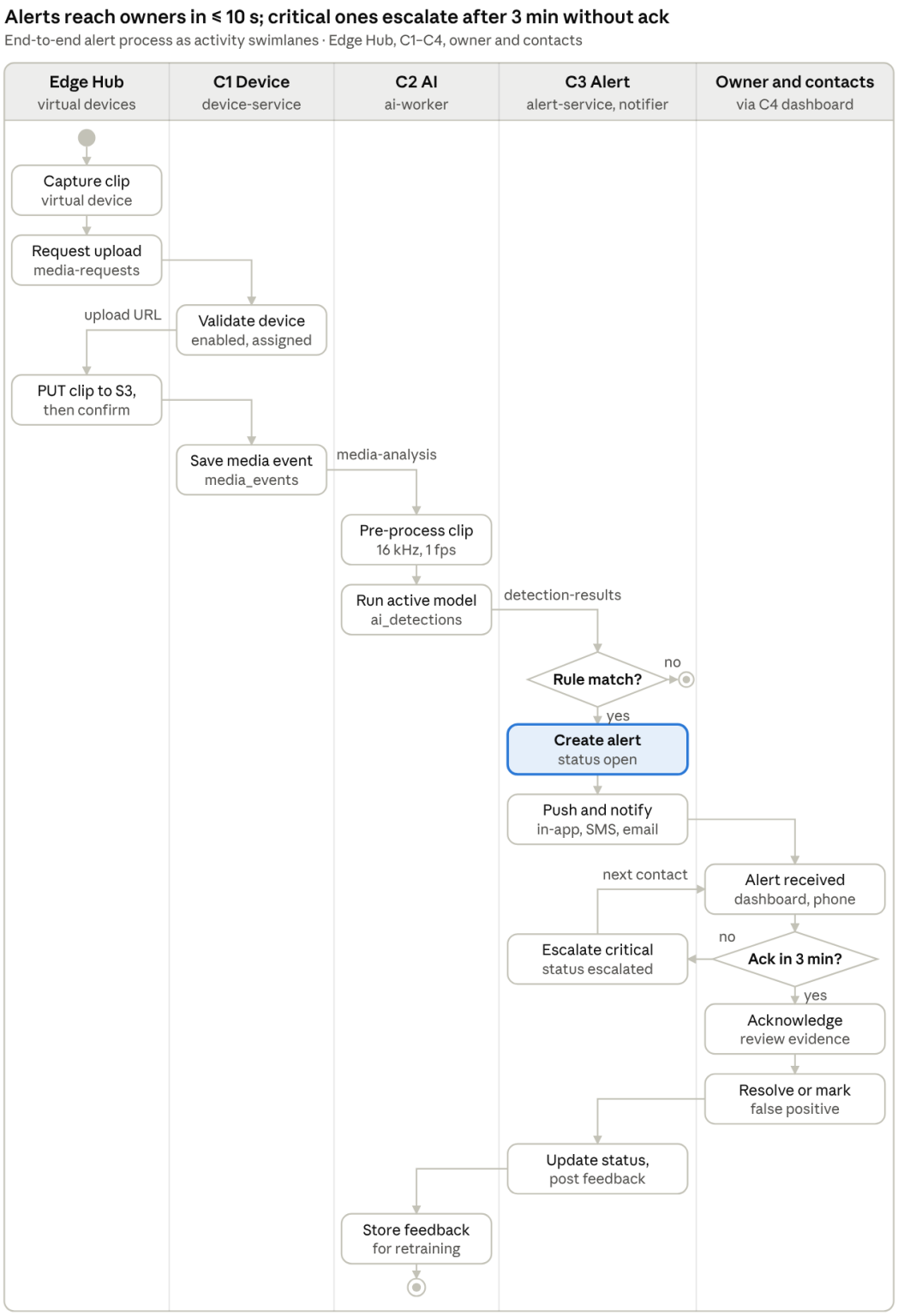


Figure 2: End-to-end alert process in swimlanes: 15 activities and 2 decisions

A detection that matches no rule ends in C3 without an alert; every resolved or false-positive alert returns to C2 as labelled feedback.

Device onboarding. IoT staff take each new virtual device through six steps in C1; ingest rejects the device until step 5.

1. **Register** — Device Manager → Add (`POST /devices`) with type and name; status `provisioned`, device key shown once.
2. **Configure** — `PUT /devices/{device_id}/config` sets sample rate or resolution and fps, capture interval, sensitivity and Wi-Fi SSID as `device_configs` version 1.
3. **Assign** — `PUT /devices/{device_id}/assignment` places the device in one house and room.
4. **Associate test media** — `PUT /devices/{device_id}/test-media` links clips from the Test Media Library with playback order and loop flag.
5. **Enable** — `POST /devices/{device_id}/enable`; the house's Edge Hub loads the device key and configuration.
6. **First heartbeat** — the Edge Hub calls `POST /devices/{device_id}/heartbeat` within 30 s; status turns `online` and the device appears on the owner's room map.

2.6 Constraints and Assumptions

- **EC2 deployment.** The course requires AWS EC2; the demo runs all containers on one t3.large with docker compose, RDS db.t3.micro, MongoDB Atlas M0, S3 and SQS. The two-AZ production layout uses the same images.
- **Virtual devices.** Cameras and microphones are software devices in the Edge Hub that replay assigned test-media clips; no physical hardware is installed.
- **Pre-trained models.** YAMNet (AudioSet, 521 classes) and YOLOv8n (COCO, 80 classes) are used as delivered; the team maps classes to home labels and tunes thresholds, with no training from scratch.
- **Senior privacy and consent.** The resident or a guardian consents when the house is subscribed. Audio becomes sound labels only, raw media expires after 30 days, and owners can switch any detection service off.
- **Course timeline.** D2 is due 2026-10-16, D3 2026-11-06, D4 2026-12-02 and D5 2026-12-16; Must requirements are sized to fit D3 and D4.
- **Demo notifications.** SES and SNS stay in sandbox mode, so email and SMS reach only verified team addresses and phones.
- **Scope limits.** One AWS region (us-west-2), an English-only dashboard, and billing records without payment processing.

3 System Infrastructure and Architectures

HomeAI Cloud runs four FastAPI services and two worker types on EC2 Auto Scaling groups in AWS us-west-2, behind one Application Load Balancer (ALB), with PostgreSQL, MongoDB, S3, SQS and Redis as shared platform services.

3.1 Cloud-Based System Infrastructure

The production layout spans two Availability Zones (AZs) in one VPC (`10.0.0.0/16`), so losing one zone stops no tier. Browsers and Edge Hubs enter only through Route 53 and the ALB; media

bypasses the API tier and goes straight to S3 by pre-signed URL.

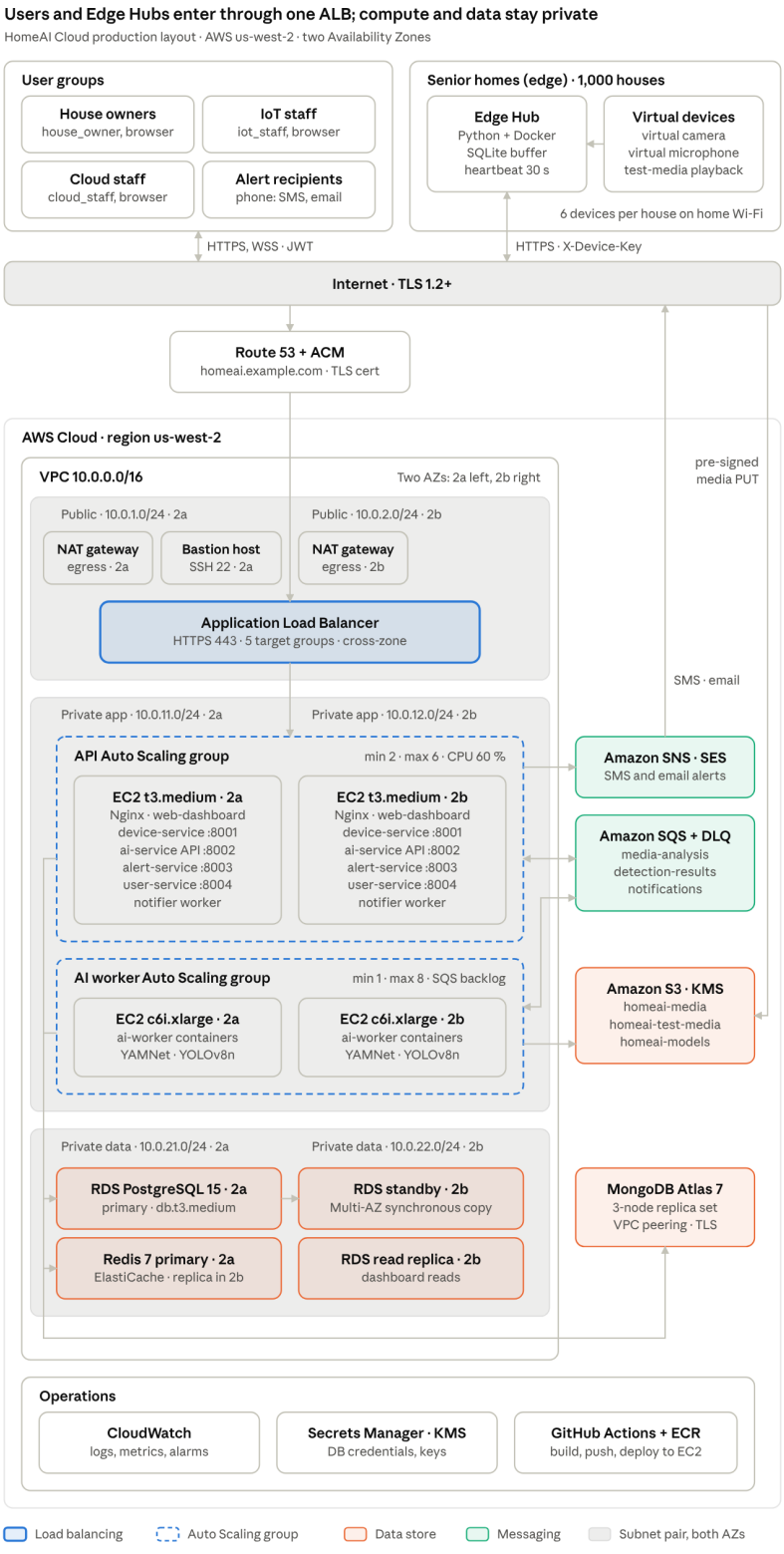


Figure 3: HomeAI Cloud infrastructure on AWS: one VPC, two Availability Zones, three subnet tiers

Requests reach the API group through Route 53 and the ALB; clips go from the Edge Hub to S3 and reach the AI workers through SQS.

The five infrastructure blocks in the instructor's platform model map onto concrete elements of this layout.

Instructor block	HomeAI Cloud element
Edge-based smart agent	Edge Hub per house: runs the virtual cameras and microphones, buffers in SQLite, uploads media
Connectivity network	Home Wi-Fi → Internet → Route 53 → ALB over TLS 1.2+; S3 pre-signed PUT for media
Home audio/sound big data store	S3 `homeai-media` for raw clips; MongoDB `media_events`, `ai_detections`, `device_telemetry`
Cloud assistant agent	`ai-worker` (C2), the `alert-service` rule engine and the `notifier` (C3), driven by SQS queues
Admin monitor and mobile app	`web-dashboard` (C4, responsive React SPA) plus SMS and email to recipients' phones

Infrastructure elements

Every tier except the Edge Hub runs in AWS us-west-2; compute and data stay in private subnets.

Element	AWS service / technology	Sizing	Purpose
Edge	Edge Hub (Python, Docker), one per house	6 devices per house; 1,000 houses (6,000 devices)	Runs virtual devices, sends heartbeats every 30 s, buffers in SQLite when offline
DNS and TLS	Route 53, AWS Certificate Manager (ACM)	`homeai.example.com`, one public certificate	Name resolution; TLS 1.2+ terminated at the ALB
Load balancer	Application Load Balancer	Internet-facing, 2 AZs, cross-zone on	Path routing to 5 target groups; `GET /health` every 15 s; WebSocket upgrade
API tier	EC2 Auto Scaling group + Docker	t3.medium, min 2 / max 6, target CPU 60 %	Nginx, `device-service`, `ai-service` API, `alert-service`, `user-service`, `notifier`, `web-dashboard` build
AI worker tier	EC2 Auto Scaling group + Docker	c6i.xlarge, min 1 / max 8, target 20 visible messages per worker	`ai-worker` containers running YAMNet (TensorFlow 2.15) and YOLOv8n (ONNX Runtime)
Relational database	Amazon RDS for PostgreSQL 15	db.t3.medium Multi-AZ + 1 read replica	Tenants, users, houses, devices, rules, alerts, audit; the replica serves dashboard reads
Document database	MongoDB Atlas 7 on AWS us-west-2	3-node replica set; hashed shard key `house_id` at scale	Media events, AI detections, telemetry, activity patterns, daily statistics
Cache and pub/sub	Amazon ElastiCache for Redis 7	Primary + replica	Rule cache, 30 s dashboard cache, tenant rate limits, `alerts:{tenant_id}` channels, device last-seen

Element	AWS service / technology	Sizing	Purpose
Object storage	Amazon S3 with KMS encryption	`homeai-media`, `homeai-test-media`, `homeai-models`	Raw clips (30 days, then Glacier for 1 year), test clips, model artifacts
Message queues	Amazon SQS standard queues + DLQs	`media-analysis`, `detection-results`, `notifications`; visibility 120 s, 5 receives	Decouple ingestion, inference and alerting
Notification channels	Amazon SNS (SMS), Amazon SES (email)	Pay per message	Alerts to external recipients
Network services	NAT gateway per AZ, bastion host, S3 gateway endpoint	Bastion t3.micro	Outbound AWS API calls; admin SSH; private path to S3
Operations	CloudWatch, Secrets Manager, ECR, GitHub Actions	One ECR repository per service	Logs, metrics, alarms; secrets; container images; CI/CD to EC2

Connectivity and user access

Only port 443 is open to the Internet; every inner hop is limited by a security group or an IAM role.

From → to	Protocol / port	Authentication	Notes
Browser → ALB	HTTPS 443, TLS 1.2+	JWT bearer: access 30 min, refresh 7 days	`house_owner`, `iot_staff`, `cloud_staff`; REST under `/api/v1`
Browser ↔ `alert-service`	WSS 443, `/ws/alerts`	JWT in the `token` query parameter	`alert.created`, `alert.updated`, `device.status` events; Redis pub/sub fan-out, no sticky sessions
Recipient phone → ALB	HTTPS 443	Signed acknowledge token, 30-min expiry	One-click acknowledge from SMS or email; no login
Edge Hub → ALB	HTTPS 443	`X-Device-Key`, stored as a SHA-256 hash	`/api/v1/ingest/*`; heartbeat every 30 s
Edge Hub → S3	HTTPS 443, PUT	Pre-signed URL, 5-min expiry	Clip goes straight to `homeai-media`
ALB → services	HTTP 8001–8004 and 80	Security groups ALB-SG → App-SG	8001 `device-service`, 8002 `ai-service`, 8003 `alert-service`, 8004 `user-service`, 80 Nginx with `web-dashboard`
Services → RDS PostgreSQL	TCP 5432, TLS	Credentials from Secrets Manager; App-SG → DB-SG	Writes go to the primary; dashboard reads go to the replica
Services → ElastiCache Redis	TCP 6379, TLS	Redis AUTH token; App-SG → Redis-SG	Cache, rate limits, pub/sub
Services, workers → MongoDB Atlas	TCP 27017, TLS	Database user per service; VPC peering + Atlas IP access list	Media events, detections, telemetry

From → to	Protocol / port	Authentication	Notes
Services, workers → SQS, SNS, SES	HTTPS 443 through the NAT gateway	IAM instance role	No static keys on instances
Services, workers → S3	HTTPS 443 through the gateway endpoint	IAM instance role	Signs upload URLs; workers read clips and model artifacts
Team → bastion → instances	SSH 22	SSH key pair; team IP allow-list	Administration only

The course demo runs the same container images on one EC2 t3.large with docker compose, using Nginx in place of the ALB. It adds RDS db.t3.micro, MongoDB Atlas M0, S3 and SQS, with SNS and SES in sandbox mode. Moving to the production layout changes instance counts and adds the ALB and Auto Scaling groups; the code does not change.

3.2 System Component-Oriented Function Architecture

HomeAI Cloud splits into 4 components (C1–C4) at the high level and 31 sub-functions at the low level, each owned by exactly one component. Each component also carries one or more of the HomeAI cloud services named in the instructor's platform model.

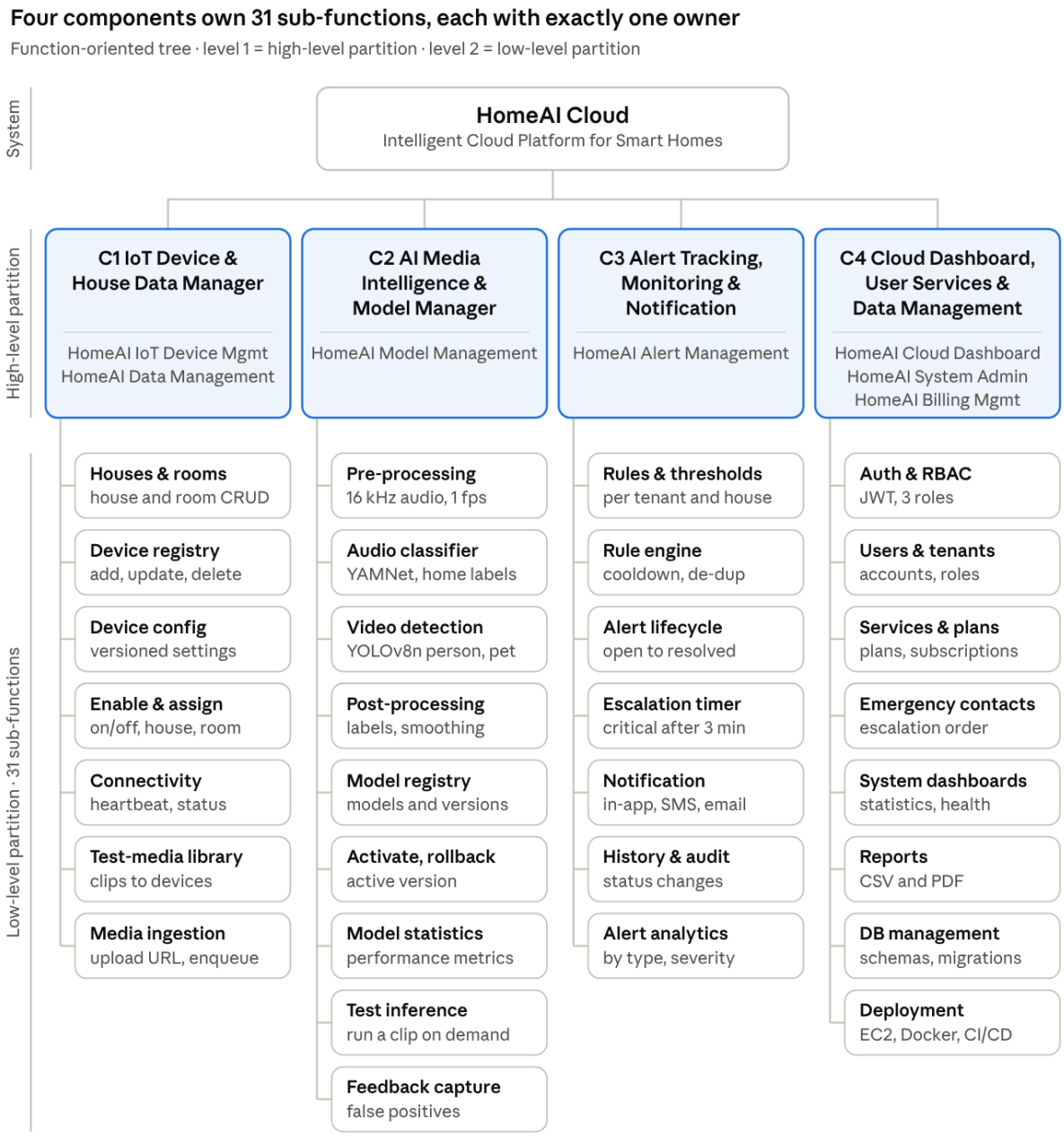


Figure 4: Function-oriented tree: 4 components (high-level partition) and 31 sub-functions (low-level partition)

C2 holds the most sub-functions (9) because it owns both inference and the model lifecycle; C4 holds the cross-cutting ones.

The functional block diagram places the same components in seven layers and shows which way calls and data flow between them.

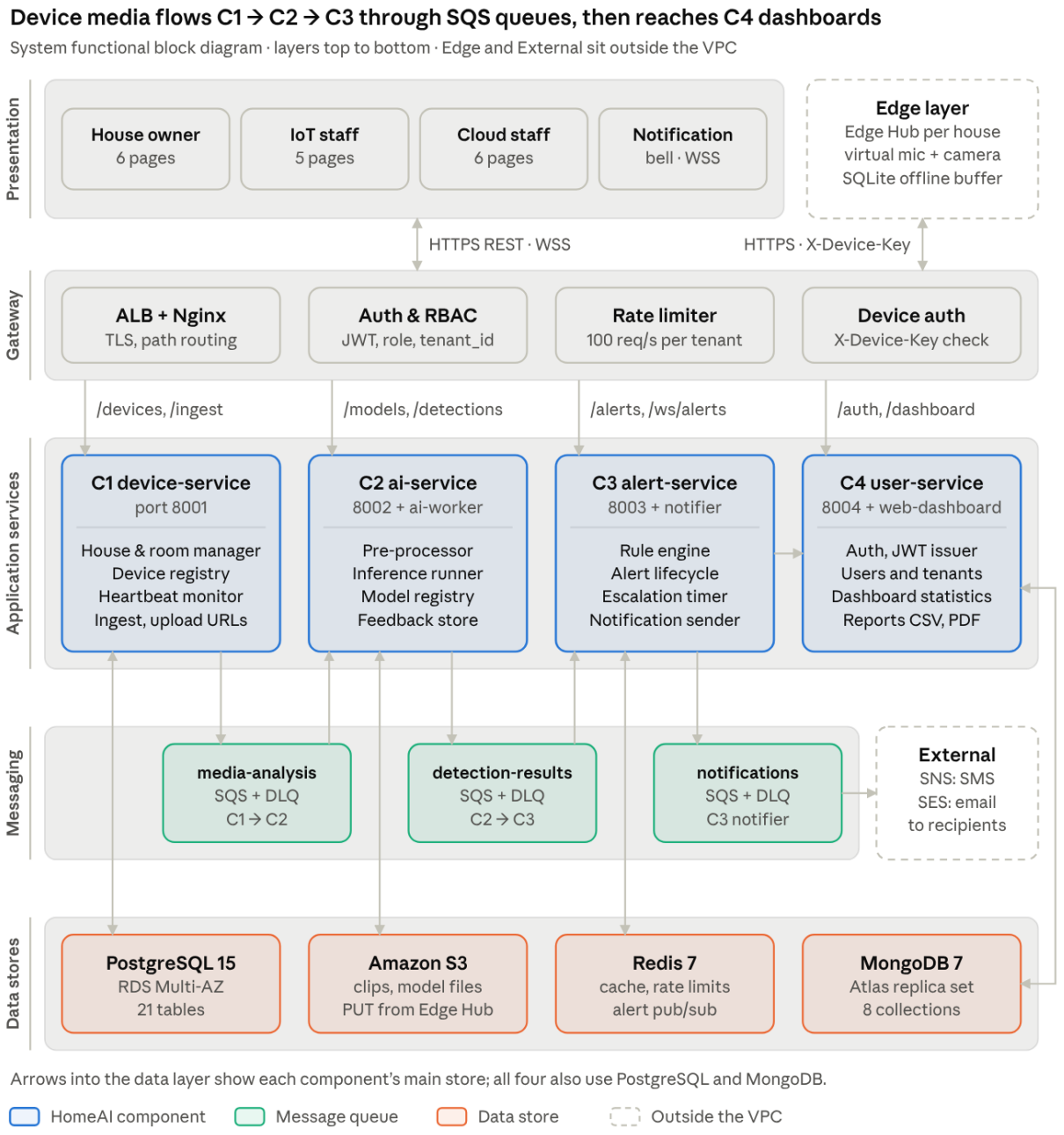


Figure 5: System functional block diagram: five cloud layers plus edge and external services

Every hand-off between C1, C2 and C3 crosses an SQS queue, so each component scales and fails on its own.

C1, C2 and C3 hand work to each other through SQS, so a slow AI tier delays alerts but never blocks ingestion. Only two internal REST calls cross components synchronously: device-offline (C1 → C3) and false-positive feedback (C3 → C2).

Component	Depends on	Provides to	Shared data stores
C1 IoT Device & House Data Manager (`device-service`)	C4 JWT and tenant context; S3 for pre-signed URLs; C3 `POST /internal/alerts/device-offline`	Edge Hub: upload URLs and configuration; C2: `media-analysis` messages; C3, C4: houses, rooms, device status	PostgreSQL `houses`, `rooms`, `devices`, `device_configs`, `test_media`, `device_media_assignments`; MongoDB `media_events`, `device_telemetry`, `device_activity_log`; S3 `homeai-media`, `homeai-test-media`; Redis device last-seen
C2 AI Media Intelligence & Model Manager (`ai-service`, `ai-worker`)	C1 `media-analysis` messages and clips in S3; C4 JWT; C3 false-positive feedback	C3: `detection-results` messages; C4: model metrics and AI throughput; C1: `media_events.status = analyzed`	PostgreSQL `ml_models`, `model_versions`; MongoDB `ai_detections`, `media_events`, `model_eval_runs`, `activity_patterns`; S3 `homeai-models`
C3 Alert Tracking, Monitoring & Notification (`alert-service`, `notifier`)	C2 `detection-results`; C1 device-offline call; C4 `emergency_contacts` and `house_service_settings`; SNS, SES	Users: alerts, history, statistics, WebSocket feed; C2: `POST /api/v1/detections/{id}/feedback`; recipients: SMS and email	PostgreSQL `alert_types`, `alert_rules`, `alerts`, `alert_status_history`, `notifications`; MongoDB `alert_evidence`, `activity_patterns` (read); Redis rule cache and `alerts:{tenant_id}`; SQS `notifications`
C4 Cloud Dashboard, User Services & Data Management (`user-service`, `web-dashboard`)	C1–C3 REST APIs for their pages; RDS read replica and MongoDB aggregations for statistics	All components: JWT `{sub, role, tenant_id, house_ids}`, RBAC, tenant context, schemas and migrations; users: dashboard pages and reports	PostgreSQL `tenants`, `users`, `service_plans`, `subscriptions`, `house_service_settings`, `emergency_contacts`, `audit_logs`; MongoDB `dashboard_stats_daily`; Redis 30 s statistics cache and rate limits

3.3 Cloud-Based System Design and Component Interaction Design

HomeAI Cloud runs four stateless FastAPI services behind one Application Load Balancer and joins them through SQS, Redis and S3, so each tier scales on its own metric up to the design target of 6,000 devices and 100 clips/s.

(a) Cloud Design: Scalability, Load Balancing and Multi-Tenancy

The design scales every tier horizontally, routes all synchronous traffic through one path-based ALB and keeps tenants apart inside one shared (pool) deployment. The AWS network and servers it runs on are drawn in §3.1.

Scalability. No service keeps session state: the JWT carries identity, and PostgreSQL, MongoDB, Redis and S3 hold all data, so any instance can be added or removed at any time.

Tier	Scaling mechanism	Trigger metric and threshold	Min–max	Notes
Web/API: `device-service`, `ai-service` API, `alert-service`, `user-service`, `web-dashboard`	EC2 Auto Scaling group, t3.medium; every instance runs all five containers behind Nginx	Target tracking on average CPU 60 %	2–6 instances, at least 1 per AZ	Stateless; any node serves any request or WebSocket
AI workers: `ai-worker`	EC2 Auto Scaling group, c6i.xlarge, 4 worker processes per instance	Target tracking on `media-analysis` backlog per instance (visible messages ÷ in-service instances) = 20	1–8 instances	Scale-in is safe: an unfinished message reappears after the 120 s visibility timeout
Notifier: `notifier`	One container on every API instance, so it grows with the API group	Follows the API group; CloudWatch alarm when the oldest `notifications` message is older than 60 s	2–6 containers	SNS and SES are managed and absorb send bursts
PostgreSQL 15 (RDS)	Multi-AZ primary for writes; read replica for dashboard and report reads; vertical resize	Primary CPU > 70 % for 15 min → larger instance class; replica CPU > 70 % → one more replica	1 primary + 1 replica, up to 3 replicas	Storage autoscaling on; alerts and history kept 2 years
MongoDB 7 (Atlas)	3-node replica set now; hashed sharding on `house_id` as data grows	Atlas auto-scaling on CPU and storage; shard when `ai_detections` passes 500 GB	1 shard → N shards of 3 nodes	Indexes lead with `tenant_id` / `house_id`; detections kept 180 days, telemetry 30 days (TTL)
Redis 7 (ElastiCache)	Primary + replica; larger node type, cluster mode if needed	Memory > 75 % or CPU > 65 %	1 primary + 1 replica	Dashboard statistics (30 s TTL), rule cache, rate-limit buckets, device last-seen, pub/sub
S3 `homeai-media`	Managed object store, no capacity planning	None; the 3,500 PUT/s per-prefix limit is far above the 100 PUT/s peak	Unbounded	Tenant and house prefixes spread load; lifecycle 30 days → Glacier for 1 year
SQS `media-analysis`, `detection-results`, `notifications`	Managed standard queues	None; queue backlog drives the AI worker group	Unbounded	Long polling 20 s; dead-letter queue after 5 receives

Capacity estimate. Peak load needs 7 AI worker instances and 4 API instances, inside both group maximums, and S3 grows by about 0.73 TB per day. The figures rest on five design assumptions, to be replaced by measurements in D3:

- **A1** Each house has 4 virtual microphones and 2 virtual cameras (1,000 houses × 6 = 6,000

devices).

- **A2** A microphone uploads one 10 s clip every 5 min on average and a camera one every 10 min; capture is event-triggered.
- **A3** A 10 s video clip (640×480 at 15 fps, H.264 at about 1 Mbit/s) is about 1.25 MB.
- **A4** One c6i.xlarge runs 4 `ai-worker` processes; an audio clip takes 0.2 s (download, resample, YAMNet) and a video clip 0.6 s (10 frames through YOLOv8n).
- **A5** One t3.medium API instance serves about 150 requests/s at 60 % CPU; dashboards add about 100 requests/s at peak.

Quantity	Arithmetic	Result
Audio clip size (16 kHz, 16-bit, mono, 10 s)	16,000 samples/s × 2 B × 10 s	320 KB
Average upload rate	4,000 microphones ÷ 300 s + 2,000 cameras ÷ 600 s = 13.3 + 3.3	16.7 clips/s; the 100 clips/s peak is 6× this
Clips per day	13.3 × 86,400 audio + 3.3 × 86,400 video = 1,152,000 + 288,000	1.44 million
S3 growth per day	1,152,000 × 320 KB + 288,000 × 1.25 MB = 369 GB + 360 GB	≈ 0.73 TB/day
S3 footprint	0.73 TB × 30 days in Standard; 0.73 TB × 365 days in Glacier	≈ 22 TB Standard, ≈ 266 TB Glacier
AI work at peak (same 4:1 audio-to-video mix)	80 audio/s × 0.2 s + 20 video/s × 0.6 s = 16 + 12	28 busy worker processes
AI worker instances at peak	28 ÷ 4 processes per instance	7 c6i.xlarge (group max 8)
Queue wait at peak	20 queued messages ÷ (100 ÷ 7 = 14.3 clips/s per instance)	≈ 1.4 s
API requests at peak	100 clips/s × 2 ingest calls + 6,000 ÷ 30 s heartbeats + 100 dashboard requests/s	≈ 500 requests/s
API instances at peak	500 ÷ 150 requests/s per instance	4 t3.medium (group max 6)
MongoDB growth	1.44 million × (1 KB `media_events` + 2 KB `ai_detections`)	≈ 4.3 GB/day, ≈ 780 GB over 180 days

At peak the alert path stays inside the 10 s p95 budget: upload about 2 s on a 2 Mbit/s home uplink, queue wait 1.4 s, inference 0.2 s, rule evaluation and WebSocket push under 1 s, about 4.6 s in total.

Load balancing. One internet-facing ALB spans both Availability Zones, terminates TLS and sends each `/api/v1` path prefix to its own target group; SQS and Redis balance the work that never passes through the ALB.

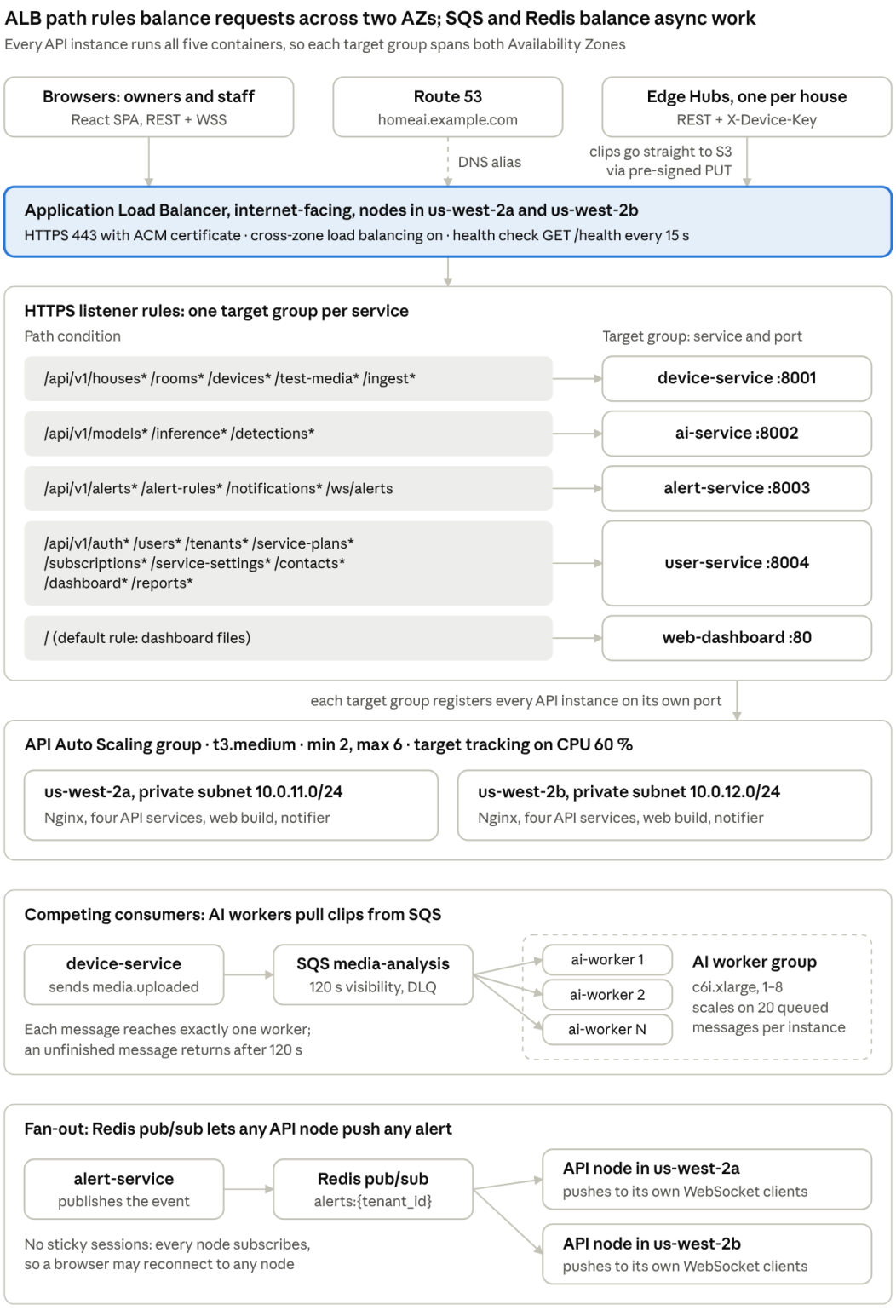


Figure 6: Load balancing: ALB path routing to five target groups in two AZs, SQS competing consumers, Redis fan-out

The ALB spreads every synchronous call across both zones, while SQS spreads clip analysis and Redis spreads alert pushes, so no single node is special.

Target groups use the least-outstanding-requests algorithm, so a slow `POST /inference/test` does not pile requests onto one node. A 30 s deregistration delay drains in-flight calls before scale-in. WebSocket clients ping every 25 s to stay under the ALB idle timeout of 60 s, and reconnect to any node.

Multi-tenancy. HomeAI Cloud uses the pool model: tenants (senior-care communities) share services, schemas, buckets and queues, and `tenant_id` is checked at every layer a request passes.

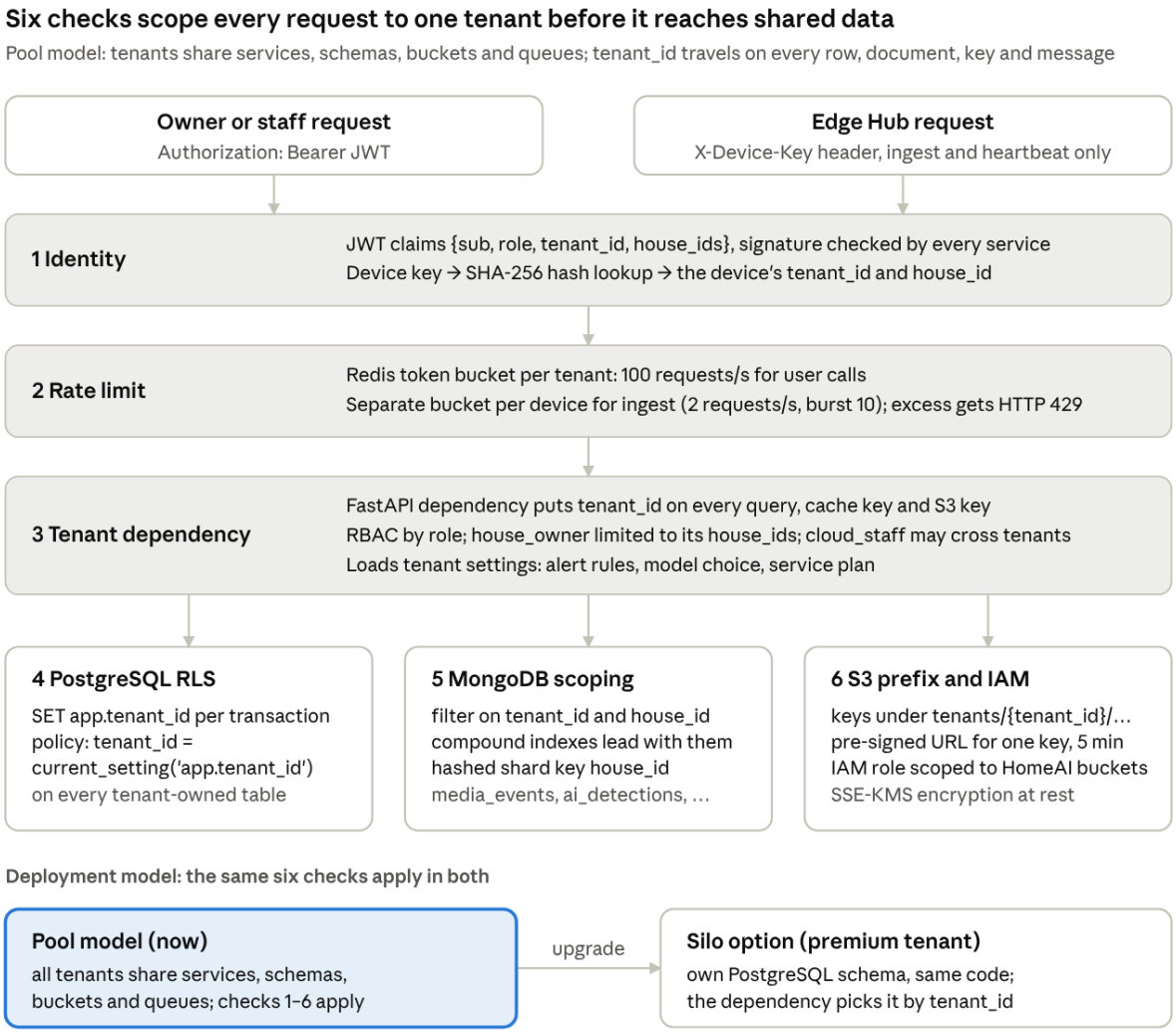


Figure 7: Multi-tenancy: six tenant checks from identity to storage, with pool and silo models

A missing filter in application code is still caught one layer down, because PostgreSQL RLS and the S3 key prefix enforce `tenant_id` on their own.

Resource	Shared across tenants	Separated per tenant
ALB, API group, AI worker group, `notifier`	All instances and containers	Every request carries the JWT `tenant_id`; Redis token bucket of 100 requests/s per tenant
PostgreSQL 15	One database and schema (pool)	Row-Level Security on `tenant_id`; a premium tenant can get its own schema (silo)
MongoDB 7	All collections (`media_events`, `ai_detections`, …)	Every query filters `tenant_id` / `house_id`; shard key `house_id`
S3 `homeai-media`	One bucket and one KMS key	Prefix `tenants/{tenant_id}/`; a pre-signed URL covers one object key only
SQS queues	`media-analysis`, `detection-results`, `notifications`	`tenant_id` in every message; consumers re-check it against the stored record
Redis	One primary-replica pair	Keys and channels carry the tenant, for example `alerts:{tenant_id}`
AI models	Registry in `ml_models` and `model_versions`	Model choice per tenant; default is the active version
Alert rules	Global defaults in `alert_rules`, set by `cloud_staff`	Tenant overrides of thresholds, cooldown and de-duplication window
Service plans	Catalogue in `service_plans`	`subscriptions` and `house_service_settings` per house

Device-key calls use a separate token bucket per device (2 requests/s, burst 10), so ingest traffic never uses up a tenant's 100 requests/s budget for people.

Availability, fault tolerance and security. The design targets 99.5 % monthly availability, at most 3.6 h of downtime a month, by removing single points of failure in every tier.

Concern	Mechanism	Design value
Availability Zone failure	ALB, both Auto Scaling groups, RDS and Redis span us-west-2a and us-west-2b	ALB stops routing to the failed zone; the groups relaunch in the healthy zone
Instance or container failure	ALB health checks; Auto Scaling replaces unhealthy instances; Docker restart policy	`GET /health` every 15 s; 2 failures mark a target unhealthy
PostgreSQL failure or data loss	RDS Multi-AZ standby with automatic failover; automated backups with point-in-time recovery	Failover in 1–2 min; backups kept 7 days
MongoDB node failure	3-node replica set with automatic election; Atlas continuous backup	Writes use `w: majority`
Failed or poison messages	SQS retries, then a dead-letter queue	Visibility timeout 120 s; `maxReceiveCount` 5; CloudWatch alarm when a DLQ is not empty
Duplicate delivery	Idempotent consumers keyed by `media_id`	Unique index on `ai_detections.media_id`; C3 de-duplication window per house and alert type
Home Internet outage	Edge Hub offline buffer in SQLite `pending_uploads` and `local_events`	Replays in capture order on reconnect; the cloud marks a silent device `offline` after 90 s

Concern	Mechanism	Design value
Unacknowledged critical alert	Escalation timer in C3	After 3 min, notifies the next emergency contact
Encryption	TLS 1.2+ in transit with an ACM certificate on the ALB; KMS at rest for S3, RDS and EBS	SSE-KMS on `homeai-media`
Credentials and secrets	bcrypt password hashes; device keys stored as SHA-256 hashes; IAM instance roles; Secrets Manager	JWT access 30 min, refresh 7 days; no static AWS keys
Network exposure	Private app and data subnets; security-group chain ALB-SG → App-SG → DB-SG and Redis-SG	Only the ALB is public; bastion SSH from team IPs only
Traceability	`audit_logs` for every staff change; CloudWatch logs, metrics and alarms	Alerts and their history kept 2 years

(b) Component Black-Box Views and Interactions

The four components never call each other in-process; they exchange only REST/JSON calls, SQS JSON messages, S3 media objects and WebSocket events, so each one is built, deployed and scaled alone. Message fields are specified in §4.4 and the stores in §4.



Figure 8: Component black-box view: inputs and outputs of C1–C4 by data format

A queue name that leaves one component on the right enters the next on the left, so `media-`

analysis couples C1 to C2 and detection-results couples C2 to C3 without either side knowing the other's host.

Component	Inputs (from: content, format)	Outputs (to: content, format)	Data stores owned
C1 IoT Device & House Data Manager (device-service)	Edge Hub: media requests, upload completions, heartbeats (REST JSON, X-Device-Key); iot_staff: house, room, device, config and test-media changes (REST JSON, JWT); test clips (WAV or MP4 into homeai-test-media)	Edge Hub: {media_id, upload_url} (JSON, S3 pre-signed PUT valid 5 min); media-analysis queue: media.uploaded (SQS JSON); C3: device offline (internal REST JSON); dashboards: devices, status, telemetry (REST JSON) and device.status (WebSocket event via Redis)	PostgreSQL houses, rooms, device_types, devices, device_configs, test_media, device_media_assignments; MongoDB media_events, device_telemetry, device_activity_log; S3 homeai-media, homeai-test-media; Redis device last-seen
C2 AI Media Intelligence & Model Manager (ai-service, ai-worker)	media-analysis queue: media.uploaded (SQS JSON); S3 homeai-media: clip (WAV 16 kHz mono or MP4 640×480); cloud_staff: models, versions, activation (REST JSON); iot_staff: test clip (multipart upload or media_id); C3: detection feedback (REST JSON)	detection-results queue: detection.completed (SQS JSON); MongoDB: ai_detections document and media_events.status = analyzed; dashboards: detections, model metrics, test results (REST JSON)	PostgreSQL ml_models, model_versions; MongoDB ai_detections, model_eval_runs, activity_patterns; S3 homeai-models (YAMNet SavedModel, YOLOv8n ONNX)
C3 Alert Tracking, Monitoring & Notification (alert-service, notifier)	detection-results queue: detection.completed (SQS JSON); C1: device offline (internal REST JSON); house_owner, cloud_staff: acknowledge, resolve, false positive, escalate (REST JSON); cloud_staff: alert rules (REST JSON); C4 data: emergency_contacts, house_service_settings (PostgreSQL reads)	Dashboards: alert.created, alert.updated (WebSocket JSON via Redis alerts:{tenant_id}); notifications queue: alert.notify (SQS JSON), then SNS SMS and SES email; C2: feedback (REST JSON); dashboards: alerts, history, statistics (REST JSON)	PostgreSQL alert_types, alert_rules, alerts, alert_status_history, notifications; MongoDB alert_evidence; Redis rule cache and alert channels
C4 Cloud Dashboard, User Services & Data Management (user-service, web-dashboard)	Users: credentials, profile, tenants, plans, settings, contacts (REST JSON over HTTPS); C1–C3 read APIs and WebSocket events (consumed by the SPA); statistics from MongoDB aggregations and PostgreSQL queries	All services: JWT with {sub, role, tenant_id, house_ids} (access 30 min, refresh 7 days); users: dashboard pages (HTML/JS from Nginx), summaries and health (REST JSON), reports (CSV or PDF); C3: contacts and house settings (PostgreSQL rows)	PostgreSQL tenants, users, service_plans, subscriptions, house_service_settings, emergency_contacts, audit_logs; MongoDB dashboard_stats_daily; Redis statistics cache (30 s) and rate-limit buckets

The canonical flow takes one clip from the Edge Hub to a closed alert in six steps; steps 1–4 carry it from ingest to an AI detection.

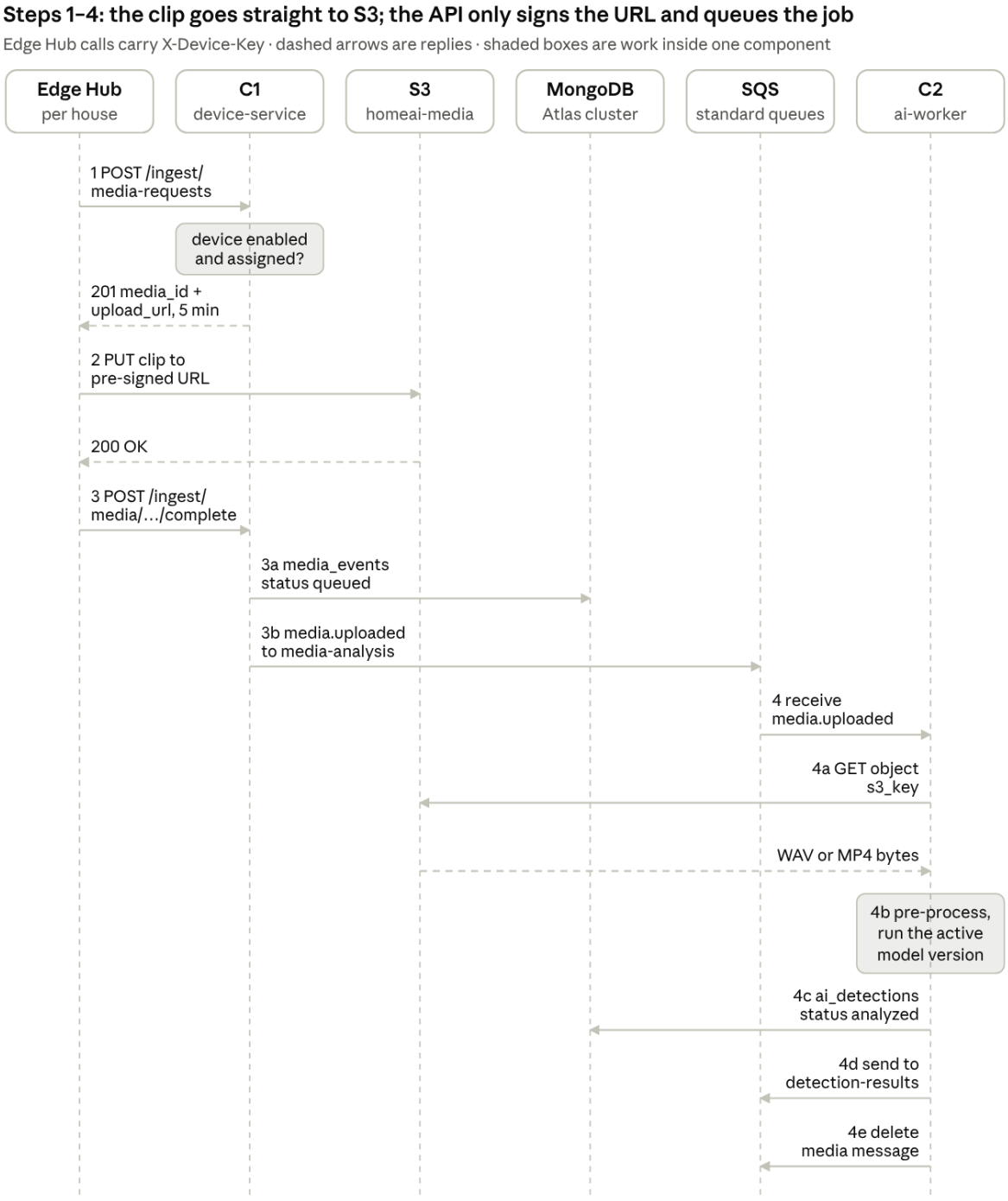


Figure 9: Sequence diagram, steps 1–4: clip ingest to AI detection

C1 never handles clip bytes: it checks the device, signs the upload and queues the job, so ingest

grows with S3 and SQS rather than with the API tier.

Steps 5–6 turn the detection into an alert, notify people and return the outcome to C2.

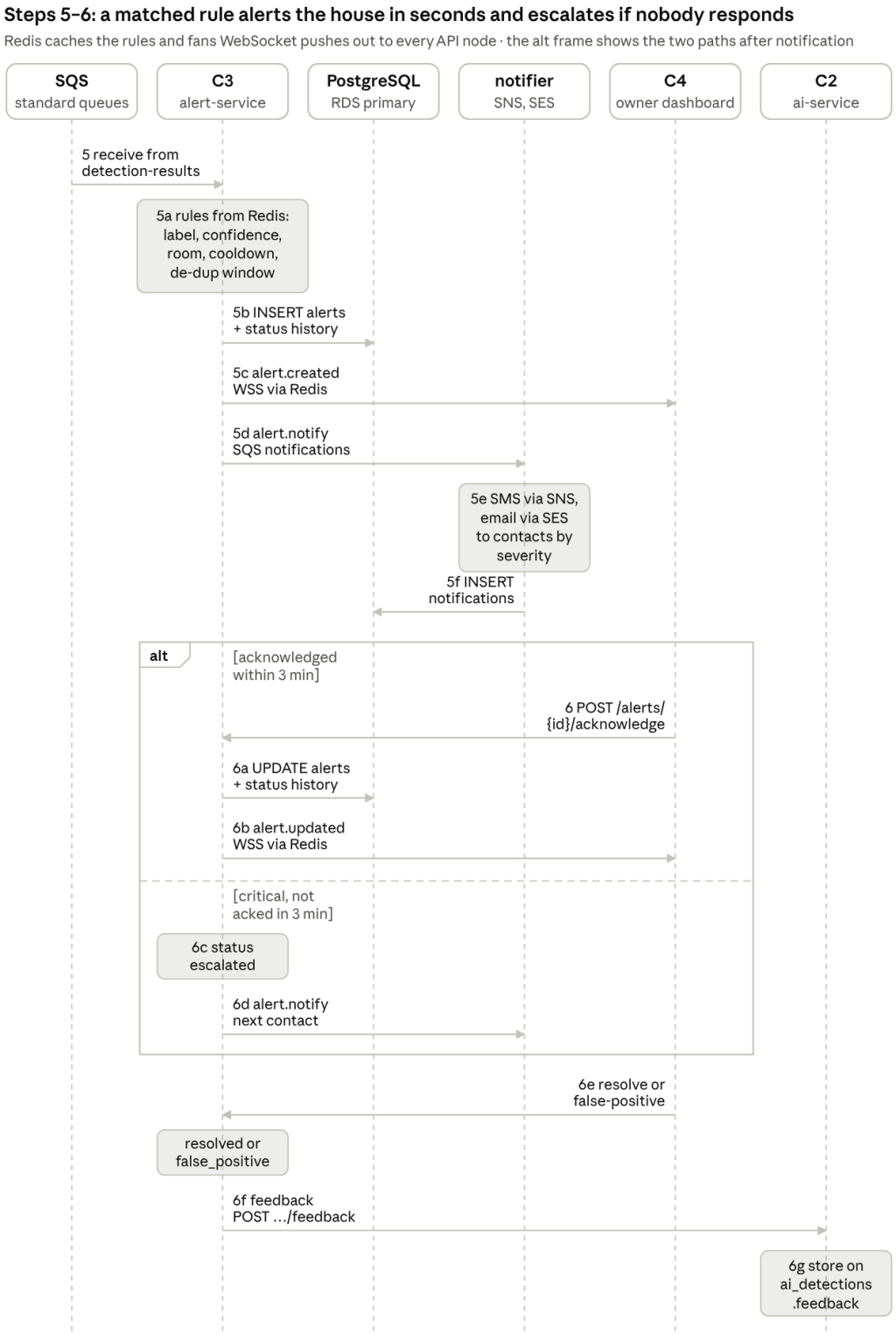


Figure 10: Sequence diagram, steps 5–6: alert, notification, acknowledgment and feedback

The owner sees the alert before the SMS lands because the WebSocket push (5c) does not wait for the notifier, and every closed alert returns its outcome to C2 for retraining.

The heartbeat path reuses steps 5–6: the Edge Hub calls `POST /devices/{device_id}/heartbeat` every 30 s, and after 90 s of silence C1 sets the device `offline` and calls C3 `POST /internal/alerts/device-offline`, which raises a `device_offline` alert.

(c) High-Level Component APIs

The four services expose 56 resource paths. All are under `/api/v1` except `/ws/alerts` and the internal path, exchange JSON and need a JWT bearer token unless marked device-key or internal. Successful responses use `{data, meta:{page, page_size, total}}`; errors use `{error:{code, message, details}}` with HTTP 400, 401, 403, 404, 409, 422, 429 or 500.

Component	Method	Path	Purpose	Caller / role(s)	Sync / async
C1	POST, GET	`/houses`	Create a house; list houses in scope	`iot_staff`, `cloud_staff`; GET also `house_owner` (own)	Sync
C1	GET, PUT, DELETE	`/houses/{house_id}`	Read, update or delete a house; an owner may change only its home, away or night mode	GET any role in scope; PUT of mode only by the house owner; full PUT and DELETE `iot_staff`, `cloud_staff`	Sync
C1	POST, GET	`/houses/{house_id}/rooms`	Add a room; list rooms	`iot_staff`, `cloud_staff`; GET also `house_owner`	Sync
C1	PUT, DELETE	`/rooms/{room_id}`	Update or delete a room	`iot_staff`, `cloud_staff`	Sync
C1	POST, GET	`/devices`	Register a virtual device; list with filters `house_id`, `room_id`, `status`, `type`	`iot_staff`; GET also `house_owner` (read-only)	Sync
C1	GET, PUT, DELETE	`/devices/{device_id}`	Read, update or delete a device	`iot_staff`; GET also `house_owner`	Sync
C1	POST	`/devices/{device_id}/enable`	Enable a device	`iot_staff`	Sync
C1	POST	`/devices/{device_id}/disable`	Disable a device; ingest then rejects it	`iot_staff`	Sync
C1	PUT	`/devices/{device_id}/assignment`	Assign a device to a house and room	`iot_staff`	Sync
C1	GET, PUT	`/devices/{device_id}/config`	Read the current config; save a new config version	`iot_staff`	Sync

Component	Method	Path	Purpose	Caller / role(s)	Sync / async
C1	GET	`/devices/{device_id}/status`	Status, last heartbeat, connectivity	`iot_staff`, `house_owner`	Sync
C1	GET	`/devices/{device_id}/telemetry`	Heartbeat and telemetry time series	`iot_staff`, `cloud_staff`	Sync
C1	POST	`/devices/{device_id}/heartbeat`	Report liveness every 30 s	Edge Hub (device-key)	Sync
C1	POST, GET	`/test-media`	Add a clip to the test-media library; list clips	`iot_staff`	Sync
C1	PUT	`/devices/{device_id}/test-media`	Link clips to a device with playback order and loop flag	`iot_staff`	Sync
C1	POST	`/ingest/media-requests`	Return `media_id` and a 5-min pre-signed S3 PUT URL	Edge Hub (device-key)	Sync
C1	POST	`/ingest/media/{media_id}/complete`	Confirm the upload, write `media_events`, enqueue `media-analysis`	Edge Hub (device-key)	Sync reply, async analysis
C2	GET, POST	`/models`	List models; register a model	GET `cloud_staff`, `iot_staff`; POST `cloud_staff`	Sync
C2	GET, PUT	`/models/{model_id}`	Read or update model metadata	`cloud_staff`	Sync
C2	GET, POST	`/models/{model_id}/versions`	List versions; register a version and its S3 artifact	`cloud_staff`	Sync
C2	POST	`/models/{model_id}/versions/{version_id}/activate`	Make a version active; activating an older one is the rollback	`cloud_staff`	Sync
C2	POST	`/models/{model_id}/versions/{version_id}/deprecate`	Retire a version	`cloud_staff`	Sync
C2	GET	`/models/{model_id}/metrics`	Accuracy, false-positive rate, latency, throughput	`cloud_staff`	Sync
C2	POST	`/inference/test`	Run the model on an uploaded clip or a `media_id`	`iot_staff`, `cloud_staff`	Sync
C2	GET	`/detections`	List detections with filters	`house_owner` (own), `iot_staff`, `cloud_staff`	Sync

Component	Method	Path	Purpose	Caller / role(s)	Sync / async
C2	GET	`/detections/{detection_id}`	Labels, confidence, model version, media key	`house_owner` (own), `iot_staff`, `cloud_staff`	Sync
C2	POST	`/detections/{detection_id}/feedback`	Store the resolve or false-positive outcome on `ai_detections.feedback`	`alert-service` (service JWT)	Sync
C3	GET, POST	`/alert-rules`	List rules; create a rule	GET `cloud_staff`, `iot_staff`; POST `cloud_staff`	Sync
C3	GET, PUT, DELETE	`/alert-rules/{rule_id}`	Read, change or delete a rule	`cloud_staff`	Sync
C3	GET	`/alerts`	List alerts with filters `status`, `severity`, `house_id`, `type`, `from`, `to`	`house_owner` (own), `cloud_staff`	Sync
C3	GET	`/alerts/{alert_id}`	Alert detail with evidence	`house_owner` (own), `cloud_staff`	Sync
C3	POST	`/alerts/{alert_id}/acknowledge`	Mark acknowledged; stop the escalation timer	`house_owner`, `cloud_staff`; external recipient through the signed 30-min link	Sync
C3	POST	`/alerts/{alert_id}/resolve`	Close the alert; send feedback to C2	`house_owner`, `cloud_staff`	Sync; feedback after the reply
C3	POST	`/alerts/{alert_id}/false-positive`	Close as `false_positive`; send feedback to C2	`house_owner`, `cloud_staff`	Sync; feedback after the reply
C3	POST	`/alerts/{alert_id}/escalate`	Escalate to the next emergency contact	`house_owner`, `cloud_staff`, C3 escalation timer	Sync
C3	GET	`/alerts/{alert_id}/history`	Status changes with actor and time	`house_owner` (own), `cloud_staff`	Sync
C3	GET	`/alerts/stats`	Counts by severity, type, status and period	`house_owner` (own), `cloud_staff`	Sync
C3	GET	`/notifications`	Sent SMS, email and push records	`house_owner` (own), `cloud_staff`	Sync
C3	WS	`/ws/alerts`	Push `alert.created`, `alert.updated`, `device.status` events	Every signed-in role (JWT in the query string)	Async push
C3	POST	`/internal/alerts/device-offline`	Raise a `device_offline` alert	`device-service` (internal, not routed by the ALB)	Sync
C4	POST	`/auth/login`	Check email and password; issue access and refresh JWTs	Public	Sync

Component	Method	Path	Purpose	Caller / role(s)	Sync / async
C4	POST	`/auth/refresh`	New access token from a refresh token	Any signed-in user	Sync
C4	POST	`/auth/logout`	Revoke the refresh token	Any signed-in user	Sync
C4	GET	`/auth/me`	Current user, role, tenant and houses	Any signed-in user	Sync
C4	GET, POST	`/users`	List users; create a user with a role	`cloud_staff`	Sync
C4	GET, PUT, DELETE	`/users/{user_id}`	Read, update or delete a user	`cloud_staff`; GET, PUT own profile by any user	Sync
C4	GET, POST	`/tenants`	List tenants; onboard a tenant	`cloud_staff`	Sync
C4	GET, PUT	`/tenants/{tenant_id}`	Read or change tenant settings (plan, model choice, limits)	`cloud_staff`	Sync
C4	GET	`/service-plans`	List service plans	Every role	Sync
C4	GET, PUT	`/subscriptions/{house_id}`	Read or change a house subscription	`house_owner`	Sync
C4	GET, PUT	`/service-settings/{house_id}`	Detection services on or off, quiet hours, sensitivity	`house_owner`	Sync
C4	GET, POST	`/contacts`	List or add emergency contacts	`house_owner`	Sync
C4	PUT, DELETE	`/contacts/{contact_id}`	Change or remove a contact	`house_owner`	Sync
C4	GET	`/dashboard/summary`	Role-scoped KPIs, cached 30 s in Redis	Every role	Sync
C4	GET	`/dashboard/system-health`	Services, EC2, databases and queue depth	`cloud_staff`	Sync
C4	GET	`/reports/alerts`	Alert report as CSV or PDF	`house_owner`, `cloud_staff`	Sync (file download)

4 System Database and Data Design

HomeAI Cloud uses polyglot persistence across four stores, each matched to one data shape. PostgreSQL holds transactional records, MongoDB holds high-volume media, AI and telemetry documents, S3 holds media bytes, and each Edge Hub buffers offline data in SQLite. Each table and collection has one owner component (C1–C4) that writes it; other components read it through the owner's API.

Data	Store	Why this store
Tenants, users, houses, rooms, devices, configurations, plans, contacts, models, rules, alerts, notifications, audit trail	Amazon RDS for PostgreSQL 15, database `homeai`, 21 tables	Foreign keys, ACID alert status changes, Row-Level Security per tenant, SQL reports

Data	Store	Why this store
Media events, AI detections, device telemetry and logs, activity patterns, alert evidence, daily statistics, model evaluations	MongoDB 7 on Atlas, database `homeai`, 8 collections	Nested label arrays, 17.3 M heartbeats a day from 6,000 devices, time-series and TTL collections, hashed sharding on `house_id`
Audio and video clips, test clips, model artifacts	Amazon S3 buckets `homeai-media`, `homeai-test-media`, `homeai-models`	Unbounded binary objects, pre-signed direct upload, lifecycle to Glacier
Clips and device events not yet sent to the cloud	Edge Hub SQLite tables `pending_uploads`, `local_events`	Survives home internet outages; capped at 500 MB

PostgreSQL issues `tenant_id`, `house_id`, `room_id` and `device_id` as UUIDs; MongoDB documents and S3 keys copy them, so one id joins all four stores. Redis caches rule sets, device last-seen times and 30-second dashboard statistics, but it is never the system of record.

4.1 Relational Database Design (PostgreSQL ERD)

The PostgreSQL schema has 21 tables, drawn as three ERDs by subject area. ERD 1 covers tenancy and owner services, ERD 2 houses, rooms and devices, and ERD 3 AI models and alerts. Each box lists the primary key, foreign keys and main attributes; the data dictionary gives every type and constraint.

Schema rules:

- **Keys:** `UUID` primary keys from `gen_random_uuid()`; the append-only `alert_status_history` and `audit_logs` use `BIGINT GENERATED ALWAYS AS IDENTITY`.
- **Time:** every timestamp is `TIMESTAMPTZ` in UTC; every table has `created_at`, and mutable tables also have `updated_at`.
- **Enumerations:** `VARCHAR(20)` plus a `CHECK (… IN …)` constraint instead of PostgreSQL `ENUM` types, so a migration can add a value without `ALTER TYPE`.
- **Tenancy:** 15 tenant-owned tables carry `tenant_id` and a Row-Level Security policy. `device_types`, `service_plans`, `ml_models`, `model_versions` and `alert_types` form a global catalog that only `cloud_staff` writes.
- **Link to MongoDB:** `alerts.detection_id CHAR(24)` stores the ObjectId of the `ai_detections` document. It cannot be a foreign key, so C3 copies it from the `detection-results` message.
- **Migrations:** each service maps its tables with SQLAlchemy 2; one Alembic history lives in C4's `db-migrations` package. CI runs `alembic upgrade head` before new containers start, and every revision has a downgrade.
- **Backup and retention:** RDS Multi-AZ with 7-day point-in-time recovery. A nightly Scheduler job deletes alerts older than 2 years; their history and notifications follow through `ON DELETE CASCADE`.

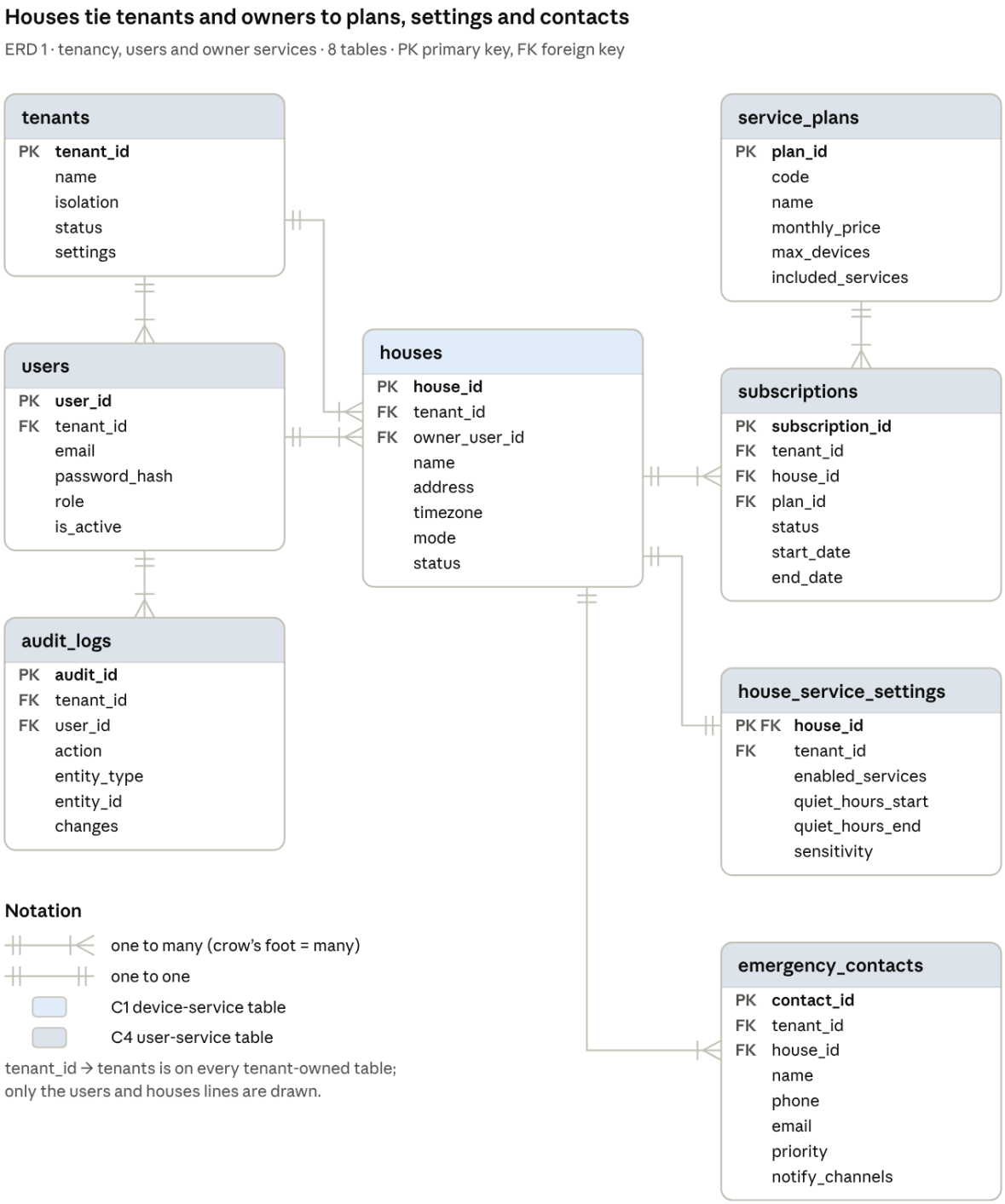


Figure 11: ERD 1: tenancy, users and owner services (8 tables)

Every house has exactly one `house_service_settings` row and any number of subscriptions and emergency contacts; `audit_logs` records each user's actions.

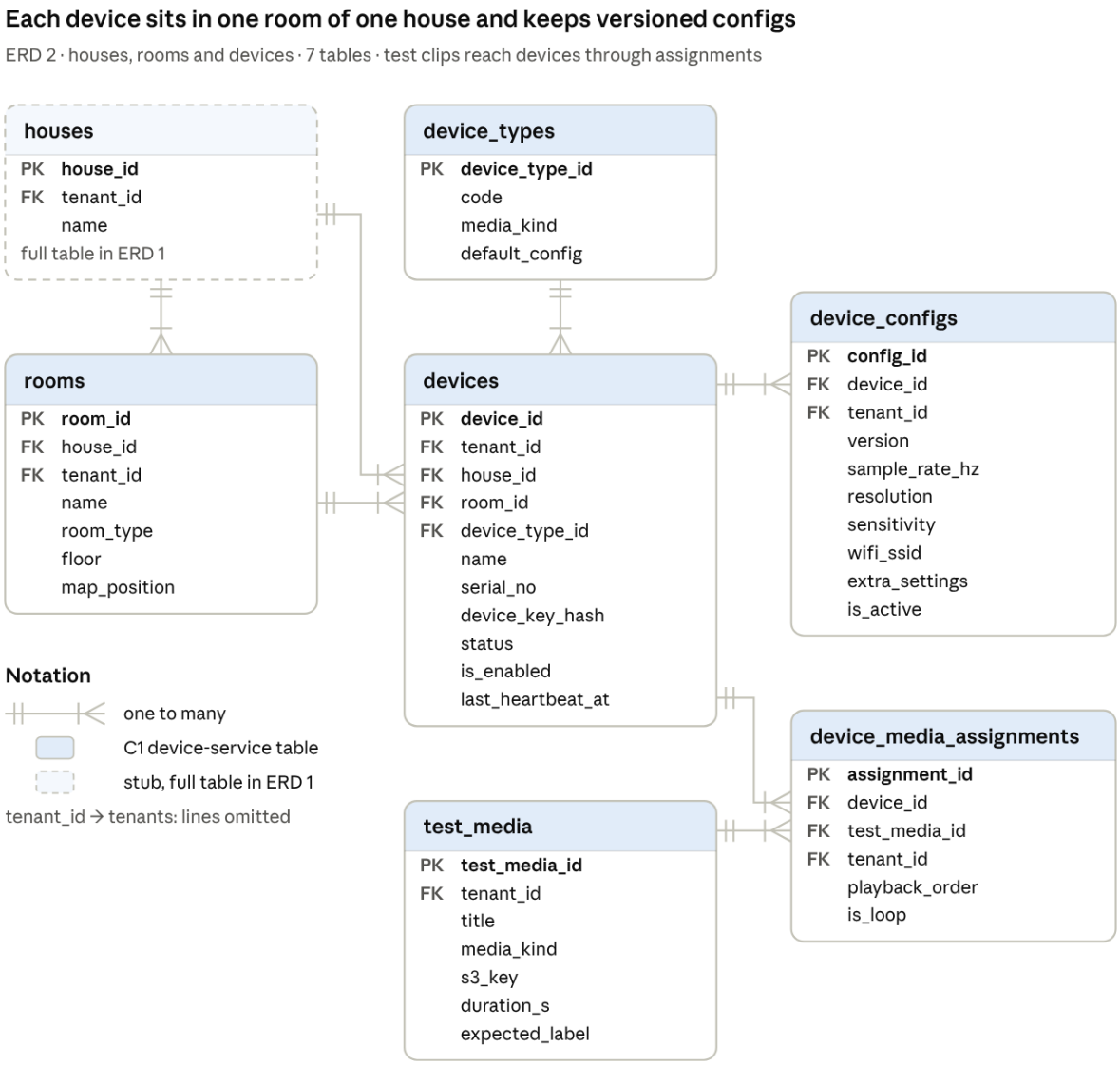


Figure 12: ERD 2: houses, rooms and devices (7 tables)

A device's `house_id` and `room_id` form one composite key into `rooms`, so a device can never sit in a room of another house; `device_media_assignments` resolves the many-to-many link between devices and test clips.

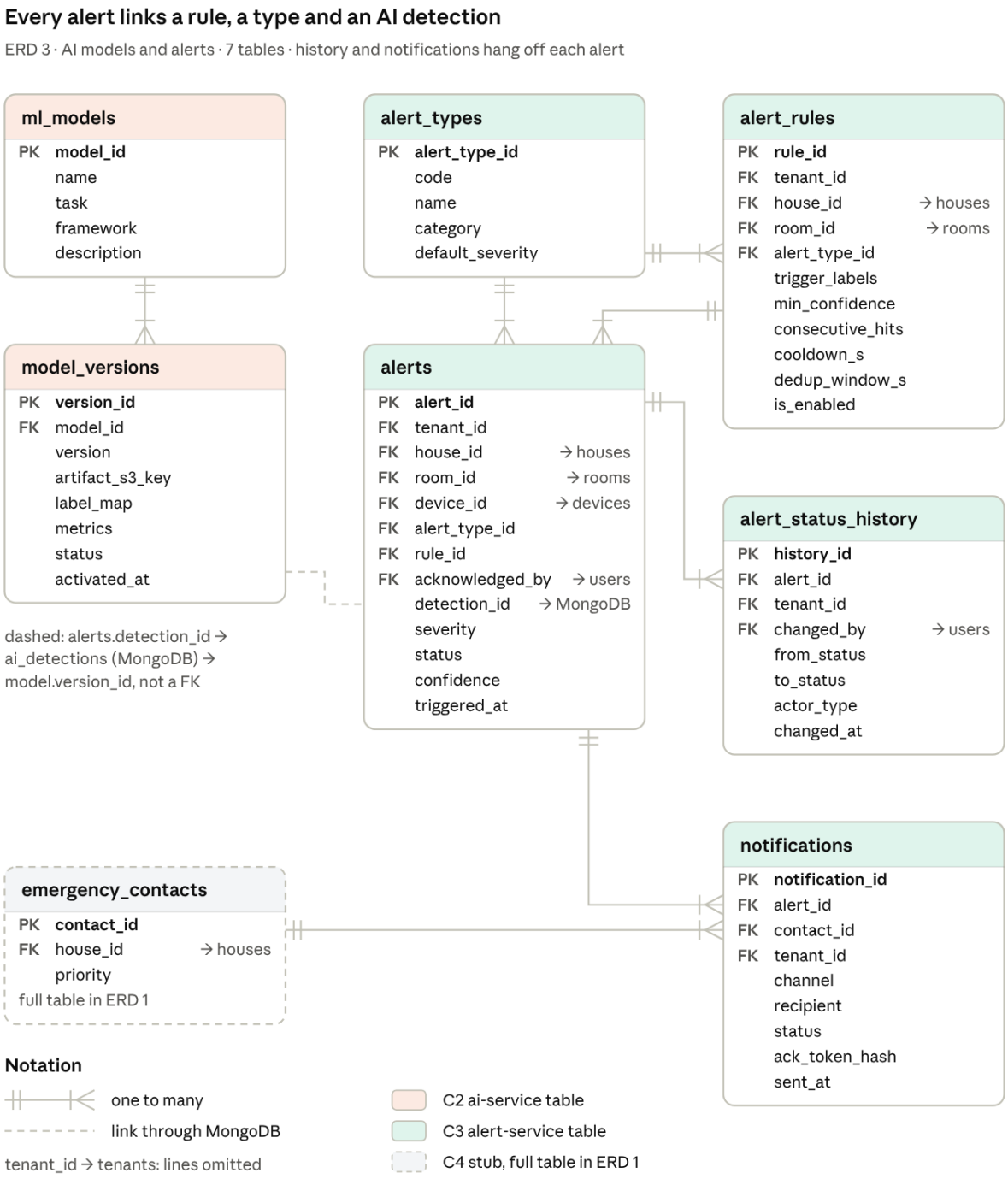


Figure 13: ERD 3: AI models and alerts (7 tables)

`alerts` is the hub: each row points to its type, the rule that fired and the house, room and device, while history and notifications hang off it; the AI model is reached only through `detection_id` in MongoDB.

Data dictionary. Types are PostgreSQL 15. Every table also has `created_at TIMESTAMPTZ NOT`

NULL DEFAULT now(); tenant-owned tables have tenant_id UUID → tenants.

Table	Owner	Primary key	Main columns (type, constraint)	Foreign keys	Indexes and constraints
tenants	C4	tenant_id UUID	name VARCHAR(120), isolation VARCHAR(10) CHECK IN ('pool','silo'), status VARCHAR(20) CHECK IN ('active', 'suspended'), rate_ limit_rps INT DEFAULT 100, settings JSONB (model selection, branding)	none	UNIQUE (name)
users	C4	user_id UUID	email VARCHAR(255), password_hash VARCHAR(100) (bcrypt), full_name VARCHAR(120), phone VARCHAR(20), role VARCHAR(20) CHECK IN ('house_owner','iot_ staff','cloud_ staff'), is_active BOOLEAN, last_login_ at TIMESTAMPTZ	tenant_id → tenants (NULL only for cloud_ staff)	UNIQUE (lower(email)); index (tenant_id, role); CHECK (role = 'cloud_staff' OR tenant_id IS NOT NULL)
houses	C1	house_id UUID	name VARCHAR(120), address TEXT, timezone VARCHAR(40) (IANA name), mode VARCHAR(10) CHECK IN ('home','away', 'night'), status VARCHAR(20) CHECK IN ('active', 'inactive')	tenant_id → tenants; owner_user_ id → users	UNIQUE (tenant_id, name); index (owner_ user_id)
rooms	C1	room_id UUID	name VARCHAR(60), room_type VARCHAR(20) CHECK IN ('bedroom','living_ room','kitchen', 'bathroom', 'hallway', 'entrance','other'), floor SMALLINT, map_ position JSONB (x, y, w, h on the room map)	house_id → houses ON DELETE CASCADE	UNIQUE (house_id, name); UNIQUE (room_ id, house_id) as the target of the devices composite key

Table	Owner	Primary key	Main columns (type, constraint)	Foreign keys	Indexes and constraints
device_types	C1	device_type_id UUID	code VARCHAR(30) CHECK IN ('virtual_camera','virtual_microphone'), media_kind VARCHAR(10) CHECK IN ('audio', 'video'), default_config JSONB (16 kHz mono; 640×480 at 15 fps)	none	UNIQUE (code)
devices	C1	device_id UUID	name VARCHAR(80), serial_no VARCHAR(64), device_key_hash CHAR(64) (SHA-256 of X-Device-Key), status VARCHAR(20) CHECK IN ('provisioned', 'online','offline', 'error', 'maintenance') DEFAULT 'provisioned', is_enabled BOOLEAN DEFAULT false, last_heartbeat_at TIMESTAMPTZ, config_version INT	house_id → houses (NULL = unassigned); (room_id, house_id) → rooms (room_id, house_id); device_type_id → device_types	UNIQUE (serial_no); index (tenant_id, house_id, status); CHECK (room_id IS NULL OR house_id IS NOT NULL)
device_configs	C1	config_id UUID	version INT, sample_rate_hz INT, resolution VARCHAR(12), fps SMALLINT, capture_interval_s INT, sensitivity NUMERIC(3,2) CHECK (sensitivity BETWEEN 0 AND 1), wifi_ssid VARCHAR(64), extra_settings JSONB, is_active BOOLEAN	device_id → devices ON DELETE CASCADE; created_by → users	UNIQUE (device_id, version); partial UNIQUE (device_id) WHERE is_active
test_media	C1	test_media_id UUID	title VARCHAR(120), media_kind VARCHAR(10), s3_key TEXT (bucket homeai-test-media), duration_s NUMERIC(6,2), expected_label VARCHAR(40), size_bytes BIGINT	tenant_id NULL = shared library; uploaded_by → users	UNIQUE (s3_key); index (tenant_id, media_kind)

Table	Owner	Primary key	Main columns (type, constraint)	Foreign keys	Indexes and constraints
device_media_assignments	C1	assignment_id UUID	playback_order SMALLINT, is_loop BOOLEAN DEFAULT true, assigned_at TIMESTAMPTZ	device_id → devices ON DELETE CASCADE; test_media_id → test_media; assigned_by → users	UNIQUE (device_id, test_media_id); UNIQUE (device_id, playback_order)
service_plans	C4	plan_id UUID	code VARCHAR(20), name VARCHAR(60), monthly_price NUMERIC(8,2), max_devices SMALLINT, included_services TEXT[] (subset of emergency, safety, security, wellness), is_active BOOLEAN	none	UNIQUE (code)
subscriptions	C4	subscription_id UUID	status VARCHAR(20) CHECK IN ('trial', 'active','past_due', 'cancelled'), billing_cycle VARCHAR(10) CHECK IN ('monthly', 'annual'), start_date DATE, end_date DATE, amount NUMERIC(8,2)	house_id → houses; plan_id → service_plans	partial UNIQUE (house_id) WHERE status IN ('trial','active'); CHECK (end_date IS NULL OR end_date > start_date)
house_service_settings	C4	house_id UUID (also the FK, 1–1)	enabled_services TEXT[], quiet_hours_start TIME, quiet_hours_end TIME, sensitivity VARCHAR(10) CHECK IN ('low','medium', 'high'), extra JSONB, updated_at TIMESTAMPTZ	house_id → houses ON DELETE CASCADE; updated_by → users	CHECK (enabled_services <@ ARRAY['emergency', 'safety','security', 'wellness']); C4 also checks them against the plan's included_services
emergency_contacts	C4	contact_id UUID	name VARCHAR(120), relationship VARCHAR(40), phone VARCHAR(20) (E.164), email VARCHAR(255), priority SMALLINT (1 = notified first), notify_channels TEXT[] (sms, email), is_active BOOLEAN	house_id → houses ON DELETE CASCADE	UNIQUE (house_id, priority); CHECK (phone IS NOT NULL OR email IS NOT NULL)

Table	Owner	Primary key	Main columns (type, constraint)	Foreign keys	Indexes and constraints
ml_models	C2	model_id UUID	name VARCHAR(60) (yamnet, yolov8n), task VARCHAR(30) CHECK IN ('audio_classification', 'video_detection'), framework VARCHAR(30) (tensorflow, onnxruntime), description TEXT	none	UNIQUE (name)
model_versions	C2	version_id UUID	version VARCHAR(20) (semantic version), artifact_s3_key TEXT (bucket homeai-models), label_map JSONB (source class → home label), metrics JSONB, status VARCHAR(20) CHECK IN ('staged','active', 'deprecated'), activated_at TIMESTAMPTZ	model_id → ml_models; created_by → users	UNIQUE (model_id, version); partial UNIQUE (model_id) WHERE status = 'active', so rollback re-activates an older version
alert_types	C3	alert_type_id UUID	code VARCHAR(40) (10 codes, e.g. fall_suspected), name VARCHAR(80), category VARCHAR(20) CHECK IN ('emergency', 'safety','security', 'wellness', 'system'), default_severity VARCHAR(10)	none	UNIQUE (code)
alert_rules	C3	rule_id UUID	name VARCHAR(120), trigger_labels TEXT[], min_confidence NUMERIC(4,3) CHECK (min_confidence BETWEEN 0 AND 1), consecutive_hits SMALLINT DEFAULT 1, time_window_s INT, cooldown_s INT, dedup_window_s INT, severity VARCHAR(10) (NULL = type default), active_modes TEXT[] (house modes), is_enabled BOOLEAN	tenant_id NULL = global default; house_id → houses (NULL = all houses); room_id → rooms (NULL = any room); alert_type_id → alert_types	GIN index on trigger_labels; index (tenant_id, house_id, is_enabled)

Table	Owner	Primary key	Main columns (type, constraint)	Foreign keys	Indexes and constraints
alerts	C3	alert_id UUID	severity VARCHAR(10), status VARCHAR(20) DEFAULT 'open', title VARCHAR(160), detection_id CHAR(24) (MongoDB ObjectId), top_label VARCHAR(40), confidence NUMERIC(4,3), escalation_level SMALLINT DEFAULT 0, dedup_key VARCHAR(120), triggered_at, acknowledged_at, resolved_at TIMESTAMPTZ, resolution_note TEXT	house_id → houses; room_id → rooms; device_id → devices ON DELETE SET NULL; alert_type_id → alert_types; rule_id → alert_rules ON DELETE SET NULL; acknowledged_by, resolved_by → users	index (tenant_id, house_id, status, triggered_at DESC); partial index (triggered_at) WHERE status = 'open' AND severity = 'critical' for the 3-minute escalation scan; index (detection_id); index (dedup_key, triggered_at)
alert_status_history	C3	history_id BIGINT (identity)	from_status VARCHAR(20) (NULL on creation), to_status VARCHAR(20), actor_type VARCHAR(20) CHECK IN ('user', 'system','ack_link'), note TEXT, changed_at TIMESTAMPTZ	alert_id → alerts ON DELETE CASCADE; changed_by → users (NULL for system)	index (alert_id, changed_at); append-only: UPDATE and DELETE revoked from the app role
notifications	C3	notification_id UUID	channel VARCHAR(10) CHECK IN ('sms', 'email','push'), recipient VARCHAR(255), template VARCHAR(40), status VARCHAR(20) CHECK IN ('queued','sent', 'delivered', 'failed'), provider_message_id VARCHAR(100) (SNS or SES id), attempts SMALLINT, ack_token_hash CHAR(64), ack_token_expires_at (30 min), sent_at, acknowledged_at TIMESTAMPTZ	alert_id → alerts ON DELETE CASCADE; contact_id → emergency_contacts (NULL for in-app push); user_id → users (push target)	index (alert_id); index (status, created_at); UNIQUE (ack_token_hash); CHECK (contact_id IS NOT NULL OR user_id IS NOT NULL)

Table	Owner	Primary key	Main columns (type, constraint)	Foreign keys	Indexes and constraints
audit_logs	C4	audit_id BIGINT (identity)	action VARCHAR(60) (e.g. device.disable, model.activate), entity_type VARCHAR(40), entity_id VARCHAR(64), changes JSONB (before and after), ip_address INET	tenant_id NULL = platform action; user_id → users	index (tenant_id, created_at DESC); index (entity_type, entity_id); append-only

Enumerations. The database enforces each value set with a CHECK constraint; C3 also enforces the allowed alert status transitions.

Enumeration	Columns	Values
Role	users.role	house_owner, iot_staff, cloud_staff
Device status	devices.status	provisioned, online, offline, error, maintenance (is_enabled is separate)
Alert status	alerts.status, alert_status_history.from_status, alert_status_history.to_status	open, acknowledged, escalated, resolved, false_positive (last two terminal)
Severity	alerts.severity, alert_rules.severity, alert_types.default_severity	critical, high, medium, low
Alert category	alert_types.category	emergency, safety, security, wellness, system
House mode	houses.mode, alert_rules.active_modes	home, away, night
Media kind	device_types.media_kind, test_media.media_kind	audio, video
Model version status	model_versions.status	staged, active, deprecated
Subscription status	subscriptions.status	trial, active, past_due, cancelled
Notification channel	notifications.channel	sms, email, push
Notification status	notifications.status	queued, sent, delivered, failed

Row-Level Security. Services connect as homeai_app, which is subject to these policies; the FastAPI tenant dependency sets app.tenant_id from the JWT at the start of each transaction. cloud_staff cross-tenant requests use a separate pool as homeai_admin with BYPASSRLS, and each such request is written to audit_logs.

```
-- Alembic migration, repeated for each tenant-owned table
ALTER TABLE alerts ENABLE ROW LEVEL SECURITY;
ALTER TABLE alerts FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON alerts
```

```
  USING (tenant_id = current_setting('app.tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);

-- per request, inside the transaction
SET LOCAL app.tenant_id = '0b6e4c1a-5f2d-4e8a-9c3b-7a1d2e3f4a5b';
```

For `test_media` and `alert_rules`, where a NULL `tenant_id` means platform-wide, the read policy adds `OR tenant_id IS NULL`.

4.2 NoSQL Database Design (MongoDB)

MongoDB 7 holds 8 collections in database `homeai` on a 3-node Atlas replica set, and every house-level document carries `tenant_id` and `house_id`. Writes use `w: "majority"`; dashboard queries read from secondaries (`secondaryPreferred`), so analytics never slow ingestion. At scale the large collections shard on hashed `house_id`, keeping each house on one shard and spreading 1,000 houses evenly.

Collection	Owner	Purpose	Key fields	Indexes	Shard key at scale	Retention
`media_events`	C1 (C2 sets `status`)	One document per uploaded clip; source of each `media-analysis` message	`_id` = `media_id` (UUID string), `device_id`, `room_id`, `media_kind`, `s3 {bucket, key, size_bytes}`, `captured_at`, `status` (`queued`, `processing`, `analyzed`, `failed`), `detection_id`	`{tenant_id: 1, house_id: 1, captured_at: -1}`; `{device_id: 1, captured_at: -1}`; `{status: 1, updated_at: 1}`	hashed `house_id`	180 days (TTL on `captured_at`)
`ai_detections`	C2	One analysis result per clip; alert evidence and retraining feedback	`_id` = `detection_id` (ObjectId), `media_id`, `device_id`, `model {name, version, version_id}`, `labels[] {label, confidence, source_class, start_s, end_s}`, `top_label`, `top_confidence`, `objects[]` (video boxes), `detected_at`, `feedback`, `expire_at`	`{media_id: 1}`; `{tenant_id: 1, house_id: 1, detected_at: -1}`; `{house_id: 1, top_label: 1, detected_at: -1}`; TTL `{expire_at: 1}`	hashed `house_id`	180 days; kept when feedback exists

Collection	Owner	Purpose	Key fields	Indexes	Shard key at scale	Retention
device_telemetry	C1	Heartbeat metrics, one point per device every 30 s (time-series collection)	ts, meta {tenant_id, house_id, device_id, device_type}, status, rssi_dbm, cpu_pct, mem_pct, pending_uploads, uptime_s	built-in meta + ts clustering; {"meta.device_id": 1, ts: -1}	hashed meta.house_id	30 days (expireAfterSeconds)
device_activity_log	C1	Device lifecycle and status events for the Device Detail page	device_id, event_type (provisioned, enabled, disabled, assigned, config_changed, status_changed, test_media_changed), from, to, actor {user_id, role}, at	{device_id: 1, at: -1}; {tenant_id: 1, house_id: 1, at: -1}	hashed house_id	1 year (TTL on at)
activity_patterns	C2 (hourly Scheduler job)	Learned daily routine per house, room and day; input to the inactivity alert	_id = {house_id}:{room_id}:{date}, date, hours[] {hour, total, counts}, baseline {window_days, hourly_mean[]}, anomaly_score, longest_quiet_daytime_h, last_activity_at	{house_id: 1, room_id: 1, date: -1}; {tenant_id: 1, date: -1, anomaly_score: -1}	hashed house_id	1 year (TTL on updated_at)
alert_evidence	C3	Frozen evidence behind one alert for the Alert Detail page	_id = alert_id (PostgreSQL UUID), detection_ids[], clips[] {media_id, s3_key, media_kind, duration_s}, label_timeline[] {t, label, confidence}, rule_snapshot, created_at	{tenant_id: 1, house_id: 1, created_at: -1}	hashed house_id	2 years, same as alerts

Collection	Owner	Purpose	Key fields	Indexes	Shard key at scale	Retention
`dashboard_stats_daily`	C4 (nightly Scheduler job)	Pre-computed KPIs per house and per tenant per day for dashboards and reports	`_id` = `{tenant_id}:{house_id or all}:{date}`, `date`, `alerts {by_severity, by_type, by_status}`, `detections {total, by_label}`, `devices {total, online_pct}`, `ai {clips, p95_latency_ms}`	`{tenant_id: 1, date: -1}`; `{house_id: 1, date: -1}`	none (about 0.4 M documents a year)	2 years
`model_eval_runs`	C2	Evaluation and test-inference runs per model version for the AI Models page	`model_id`, `version_id` (PostgreSQL UUIDs), `run_type` (`offline_eval`, `test_inference`, `feedback_rollup`), `dataset {name, s3_prefix, clip_count}`, `metrics {accuracy, macro_f1, per_label[]}`, `latency_ms {p50, p95}`, `false_positive_rate`, `started_at`	`{version_id: 1, started_at: -1}`; `{run_type: 1, started_at: -1}`; TTL `{expire_at: 1}`	none (low volume)	Evaluations kept; `test_inference` runs 90 days via `expire_at`

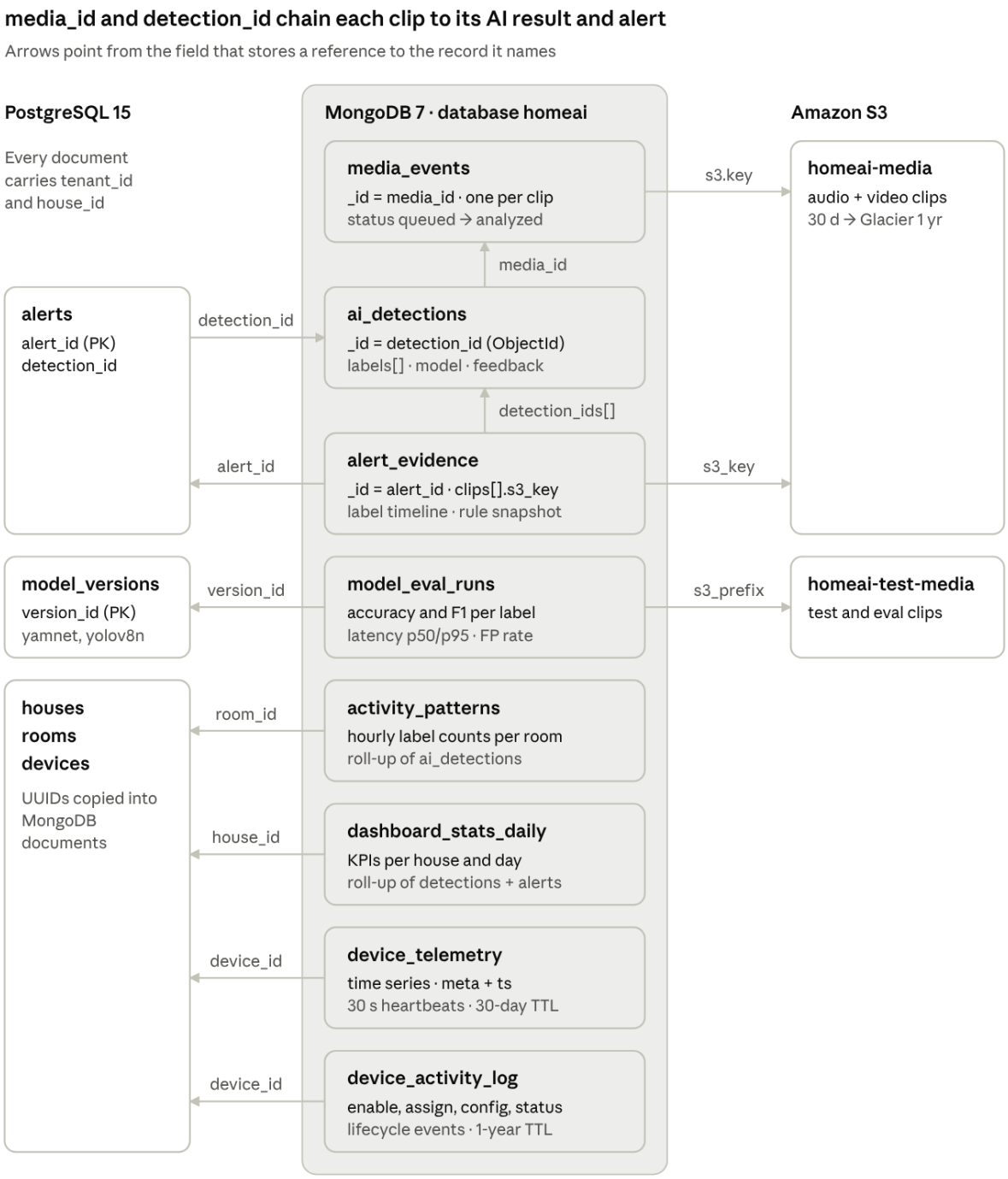


Figure 14: MongoDB reference map: 8 collections linked to PostgreSQL ids and S3 objects

A clip's bytes live in S3, its metadata in `media_events`, its AI result in `ai_detections` and its alert in PostgreSQL; `media_id` and `detection_id` join them, while the five lower collections link back only through PostgreSQL ids.

C2 keeps analysis idempotent through `media_events`. A worker claims a clip with one atomic up-

date that matches `status: "queued"`, or `processing` older than 120 s (the SQS visibility timeout). A duplicate SQS delivery then finds the clip claimed or `analyzed` and is deleted without work.

Sample documents. Dates are BSON `Date` values shown as ISO 8601 strings; all samples use the same house, so their ids join.

`media_events`: one 10-second audio clip after analysis.

```
{
  "_id": "6f1c2a9e-3b7d-4c51-9a0e-2d8b7c4e1f20",
  "tenant_id": "0b6e4c1a-5f2d-4e8a-9c3b-7a1d2e3f4a5b",
  "house_id": "a3d9f7e2-1c4b-4d6a-8e5f-9b0c1d2e3f4a",
  "room_id": "c7e1b2d3-4f5a-4b6c-8d7e-0f1a2b3c4d5e",
  "device_id": "e2f4a6b8-0c1d-4e3f-9a5b-6c7d8e9f0a1b",
  "device_type": "virtual_microphone",
  "media_kind": "audio",
  "source": "test_media",
  "test_media_id": "1a2b3c4d-5e6f-4a7b-8c9d-0e1f2a3b4c5d",
  "s3": {
    "bucket": "homeai-media",
    "key": "tenants/0b6e4c1a-5f2d-4e8a-9c3b-7a1d2e3f4a5b/houses/a3d9f7e2-1c4b-4d6a-8e5f-
    ↪  9b0c1d2e3f4a/devices/e2f4a6b8-0c1d-4e3f-9a5b-6c7d8e9f0a1b/2026/10/03/6f1c2a9e-
    ↪  3b7d-4c51-9a0e-2d8b7c4e1f20.wav",
    "size_bytes": 320044,
    "content_type": "audio/wav"
  },
  "duration_s": 10.0,
  "sample_rate_hz": 16000,
  "captured_at": "2026-10-03T21:14:05Z",
  "completed_at": "2026-10-03T21:14:07Z",
  "status": "analyzed",
  "detection_id": "6710a3c5e4b0f2a1c9d8e7f6",
  "updated_at": "2026-10-03T21:14:10Z"
}
```

`ai_detections`: the result for that clip, later marked a false positive by the house owner.

```
{
  "_id": { "$oid": "6710a3c5e4b0f2a1c9d8e7f6" },
  "media_id": "6f1c2a9e-3b7d-4c51-9a0e-2d8b7c4e1f20",
  "tenant_id": "0b6e4c1a-5f2d-4e8a-9c3b-7a1d2e3f4a5b",
  "house_id": "a3d9f7e2-1c4b-4d6a-8e5f-9b0c1d2e3f4a",
  "room_id": "c7e1b2d3-4f5a-4b6c-8d7e-0f1a2b3c4d5e",
  "device_id": "e2f4a6b8-0c1d-4e3f-9a5b-6c7d8e9f0a1b",
  "media_kind": "audio",
  "model": {
    "name": "yamnet",
    "version": "1.0.0",
    "version_id": "9d2e4b6a-8c0e-4f1a-b3c5-d7e9f1a3b5c7"
  },
  "labels": [
    { "label": "scream", "confidence": 0.874, "source_class": "Screaming", "start_s":
    ↪  2.88, "end_s": 4.80 },
    { "label": "tv", "confidence": 0.552, "source_class": "Television", "start_s": 0.0,
    ↪  "end_s": 10.0 },
```

```
    { "label": "speech", "confidence": 0.431, "source_class": "Speech", "start_s": 0.48,
    ↪  "end_s": 9.12 }
  ],
  "top_label": "scream",
  "top_confidence": 0.874,
  "inference_ms": 184,
  "is_test": false,
  "detected_at": "2026-10-03T21:14:09Z",
  "feedback": {
    "verdict": "false_positive",
    "alert_id": "4b8d0f2a-6c1e-4a3b-9d5f-7e0a2c4e6f81",
    "by_user_id": "7c9e1a3b-5d7f-4b2a-8c4e-6f8a0b2c4d6e",
    "note": "Scream came from a TV drama.",
    "at": "2026-10-03T21:19:42Z"
  }
}
```


Detections without feedback carry `expire_at` = `detected_at` + 180 days, and a TTL index removes them. C2 unsets `expire_at` when feedback arrives, so labelled examples stay for retraining.

`device_telemetry`: created as a time-series collection with `timeseries: {timeField: "ts", metaField: "meta", granularity: "seconds"}` and `expireAfterSeconds: 2592000` (30 days).


```
{
  "ts": "2026-10-03T21:14:30Z",
  "meta": {
    "tenant_id": "0b6e4c1a-5f2d-4e8a-9c3b-7a1d2e3f4a5b",
    "house_id": "a3d9f7e2-1c4b-4d6a-8e5f-9b0c1d2e3f4a",
    "device_id": "e2f4a6b8-0c1d-4e3f-9a5b-6c7d8e9f0a1b",
    "device_type": "virtual_microphone"
  },
  "status": "online",
  "rssi_dbm": -58,
  "cpu_pct": 12.5,
  "mem_pct": 31.0,
  "pending_uploads": 0,
  "uptime_s": 86412
}
```


`activity_patterns`: one room-day; `hours` holds 24 entries, of which three are shown.


```
{
  "_id": "a3d9f7e2-1c4b-4d6a-8e5f-9b0c1d2e3f4a:c7e1b2d3-4f5a-4b6c-8d7e-0f1a2b3c4d5e:2026-
  ↪  10-02",
  "tenant_id": "0b6e4c1a-5f2d-4e8a-9c3b-7a1d2e3f4a5b",
  "house_id": "a3d9f7e2-1c4b-4d6a-8e5f-9b0c1d2e3f4a",
  "room_id": "c7e1b2d3-4f5a-4b6c-8d7e-0f1a2b3c4d5e",
  "room_type": "living_room",
  "date": "2026-10-02",
  "timezone": "America/Los_Angeles",
  "hours": [
    { "hour": 7, "total": 46, "counts": { "footsteps": 21, "speech": 12, "tv": 9,
    ↪  "cough": 4 } },
```

```
    { "hour": 8, "total": 63, "counts": { "tv": 30, "speech": 22, "footsteps": 11 } },
    { "hour": 9, "total": 0, "counts": {} }
  ],
  "baseline": {
    "window_days": 14,
    "hourly_mean": [0, 0, 0, 0, 0, 1.2, 8.5, 41.0, 55.3, 38.7, 30.1, 27.4, 33.9, 25.6,
    ↪  22.8, 24.0, 29.5, 36.2, 44.8, 52.1, 47.3, 30.6, 9.4, 1.1]
  },
  "anomaly_score": 0.31,
  "longest_quiet_daytime_h": 2.5,
  "first_activity_at": "2026-10-02T14:05:12Z",
  "last_activity_at": "2026-10-03T05:41:37Z",
  "updated_at": "2026-10-03T07:00:00Z"
}
```

`anomaly_score` is 1 minus the cosine similarity between the day's 24 hourly totals and `baseline.hourly_mean`, so 0 means a typical day. The `inactivity` rule reads `longest_quiet_daytime_h` and fires at 4 h.

Aggregation example. Detections per label per house over the last 24 h, used by the System Dashboard and Home Overview:

```
db.ai_detections.aggregate([
  { $match: {
      tenant_id: "0b6e4c1a-5f2d-4e8a-9c3b-7a1d2e3f4a5b",
      detected_at: { $gte: new Date(Date.now() - 24 * 60 * 60 * 1000) },
      is_test: false
  } },
  { $unwind: "$labels" },
  { $match: { "labels.confidence": { $gte: 0.5 } } },
  { $group: {
      _id: { house_id: "$house_id", label: "$labels.label" },
      count: { $sum: 1 },
      avg_confidence: { $avg: "$labels.confidence" },
      last_seen: { $max: "$detected_at" }
  } },
  { $sort: { "_id.house_id": 1, count: -1 } },
  { $group: {
      _id: "$_id.house_id",
      total: { $sum: "$count" },
      labels: { $push: {
          label: "$_id.label",
          count: "$count",
          avg_confidence: { $round: ["$avg_confidence", 3] },
          last_seen: "$last_seen"
      } }
  } }
])
```

The first `$match` uses the `{tenant_id, house_id, detected_at}` index, and C4 caches the result in Redis for 30 s.

4.3 Edge-Based Data Buffer (SQLite)

Each Edge Hub keeps a SQLite database in WAL mode with two tables, so clips and events captured during an internet outage reach the cloud later, in capture order. The clip spool is capped at 500 MB.

`pending_uploads`: one row per clip that C1 has not yet acknowledged.

Column	Type	Notes
`local_id`	`TEXT PRIMARY KEY`	UUID made by the Edge Hub at capture
`device_id`	`TEXT NOT NULL`	Virtual device that captured the clip
`media_kind`	`TEXT CHECK IN ('audio', 'video')`	Selects `.wav` or `.mp4`
`file_path`	`TEXT NOT NULL`	Clip in the local spool folder
`size_bytes`	`INTEGER`	Counted against the 500 MB cap
`duration_s`	`REAL`	Sent in the media request
`captured_at`	`TEXT` (ISO 8601, UTC)	Original capture time, sent unchanged
`media_id`	`TEXT UNIQUE`	Issued by `POST /api/v1/ingest/media-requests`; NULL until then
`upload_url`	`TEXT`	Pre-signed S3 PUT URL
`url_expires_at`	`TEXT`	Issue time + 5 minutes
`state`	`TEXT CHECK IN ('captured', 'requested','uploaded')`	Upload progress; index `(state, next_attempt_at)`
`attempts`	`INTEGER DEFAULT 0`	Failed tries so far
`next_attempt_at`	`TEXT`	Earliest next retry
`last_error`	`TEXT`	Last HTTP status or network error

`local_events`: device events waiting to be sent.

Column	Type	Notes
`event_id`	`INTEGER PRIMARY KEY AUTOINCREMENT`	Send order
`device_id`	`TEXT NOT NULL`	Device that raised the event
`event_type`	`TEXT CHECK IN ('heartbeat','status_change', 'error','config_applied','buffer_overflow')`	Kind of event
`payload`	`TEXT` (JSON)	Metrics or error details
`occurred_at`	`TEXT` (ISO 8601, UTC)	Original event time
`synced`	`INTEGER DEFAULT 0`	1 after C1 accepts the event
`synced_at`	`TEXT`	Row deleted 24 h after sync

Sync procedure:

1. **Store locally.** A virtual device writes the clip to the spool and inserts a `pending_uploads` row (`state = 'captured'`) in one transaction, online or not.

2. **Request and upload.** The upload worker takes due rows, oldest `captured_at` first. It calls `POST /api/v1/ingest/media-requests` (→ `requested`), then PUTs the file to S3 (→ `uploaded`); an expired URL sends the row back to `captured`.
3. **Retry with exponential backoff.** A network error, HTTP 5xx or 429 increments `attempts` and sets `next_attempt_at` to now + min(2 s × 2^attempts, 5 min), ±20 % jitter. Any other 4xx, such as 403 for a disabled device, stops retries and logs an `error` event.
4. **Complete idempotently.** The worker calls `POST /api/v1/ingest/media/{media_id}/complete`. C1 inserts the `media_events` document with `_id = media_id` and sends the SQS message only on the first call; a repeat returns the stored status.
5. **Purge after acknowledgement.** A 2xx answer deletes the clip file and its row. If the spool passes 500 MB, the oldest `captured` clips go first and a `buffer_overflow` event is logged.
6. **Flush events.** Unsynced `local_events` go out oldest first once the hub is online, at most 100 per heartbeat call, and are marked `synced`.

Backlogged clips keep their original `captured_at`, so C3 rule windows and `activity_patterns` use capture time, not upload time.

4.4 Cloud-Based System Communication Design

HomeAI Cloud uses three communication styles: synchronous REST over HTTPS, real-time WebSocket (WSS) push, and asynchronous SQS messaging. Media clips take a fourth path, straight from the Edge Hub to S3 through pre-signed URLs.

REST carries commands and queries, and WSS replaces dashboard polling for alerts and device status. SQS absorbs bursts of up to 100 clips/s between `device-service`, `ai-worker` and `alert-service`. Direct S3 upload keeps binary media off the API instances.

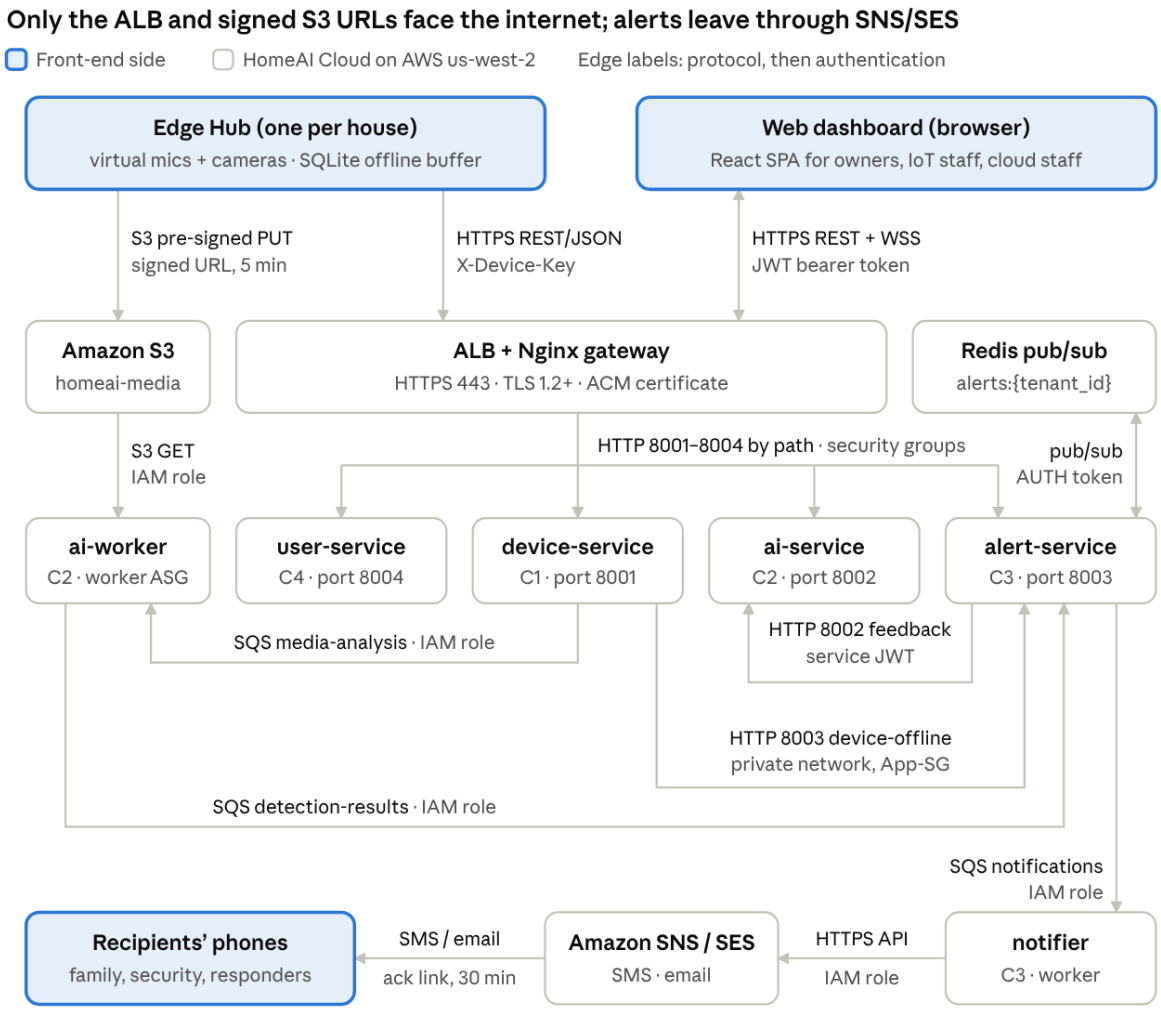


Figure 15: Communication channels between the front-end parts and the back-end cloud

Requests enter through the ALB, and clips go straight to S3. Three SQS queues chain the back end; alerts leave by WSS push and SNS/SES.

4.4.1 Channels and reliability

Eleven channels connect the parts, and each has one protocol, one credential and one failure policy.

Channel	From → to	Protocol / port	Payload	Authentication	Mode	Timeout and retry
Dashboard API	`web-dashboard` → ALB/Nginx → C1–C4 services	HTTPS 443; HTTP 8001–8004 inside the VPC	JSON envelope	JWT bearer (access 30 min)	Sync	Axios timeout 10 s; GET retried once on 502/503/504; 401 → one refresh, then retry

Channel	From → to	Protocol / port	Payload	Authentication	Mode	Timeout and retry
Live alert feed	`alert-service` → browser	WSS 443, `/ws/alerts`	JSON events	Access JWT in `token` query, checked at connect	Async push	Ping every 25 s; socket closed after 2 missed pongs; client reconnects after 1, 2, 4, 8, 16 s (cap 30 s), then re-reads open alerts
Device API	Edge Hub → `device-service`	HTTPS 443, `/ingest/*` and `/heartbeat`	JSON	`X-Device-Key`	Sync	Timeout 10 s; 5 tries with backoff 1, 2, 4, 8, 16 s; media calls then wait in SQLite `pending_uploads`
Media upload	Edge Hub → S3 `homeai-media`	HTTPS 443, S3 `PUT`	WAV / MP4 binary	Pre-signed URL (5-min expiry, one object key)	Sync	Timeout 60 s; same backoff; an expired URL triggers a new `media-requests` call
Media queue	`device-service` → `ai-worker`	SQS `media-analysis`	JSON `media.uploaded`	IAM instance role	Async	Long poll 20 s; visibility 120 s; 5 receives → `media-analysis-dlq`
Detection queue	`ai-worker` → `alert-service` rule engine	SQS `detection-results`	JSON `detection.completed`	IAM instance role	Async	Same policy → `detection-results-dlq`
Notification queue	`alert-service` → `notifier`	SQS `notifications`	JSON `alert.notify`	IAM instance role	Async	Same policy → `notifications-dlq`
Alert fan-out	`alert-service` → all `alert-service` instances	Redis pub/sub `alerts:{tenant_id}`, TCP 6379 with TLS	JSON event	Redis AUTH token; Redis-SG	Async	Fire-and-forget; clients recover missed events by re-reading on reconnect
Evidence read	`ai-worker` → S3 `homeai-media`	HTTPS through the VPC gateway endpoint	WAV / MP4 binary	IAM instance role	Sync	boto3 standard retry, 3 attempts
Outbound alert	`notifier` → SNS (SMS) / SES (email) → recipients' phones	AWS HTTPS API; SMS and email	Template text + acknowledge link	IAM role; link token 30 min	Async	3 retries (2, 4, 8 s); then `notifications.status = failed`, shown on the alert timeline
Internal calls	`device-service` → `alert-service`; `alert-service` → `ai-service`	HTTP 8003 / 8002 on private addresses	JSON	Private network (App-SG); service JWT on the C3 → C2 feedback call	Sync	Timeout 5 s; 3 tries (1, 2, 4 s); failures logged for replay

SQS long polling returns as soon as a message arrives, so the asynchronous chain adds no fixed wait. That keeps clip-to-dashboard latency inside the 10 s p95 target. Every retry is safe because writes are idempotent (§4.4.4).

4.4.2 Authentication flows

Users prove identity with JWTs, Edge Hubs with a per-device key, and services with IAM roles plus private-network placement.

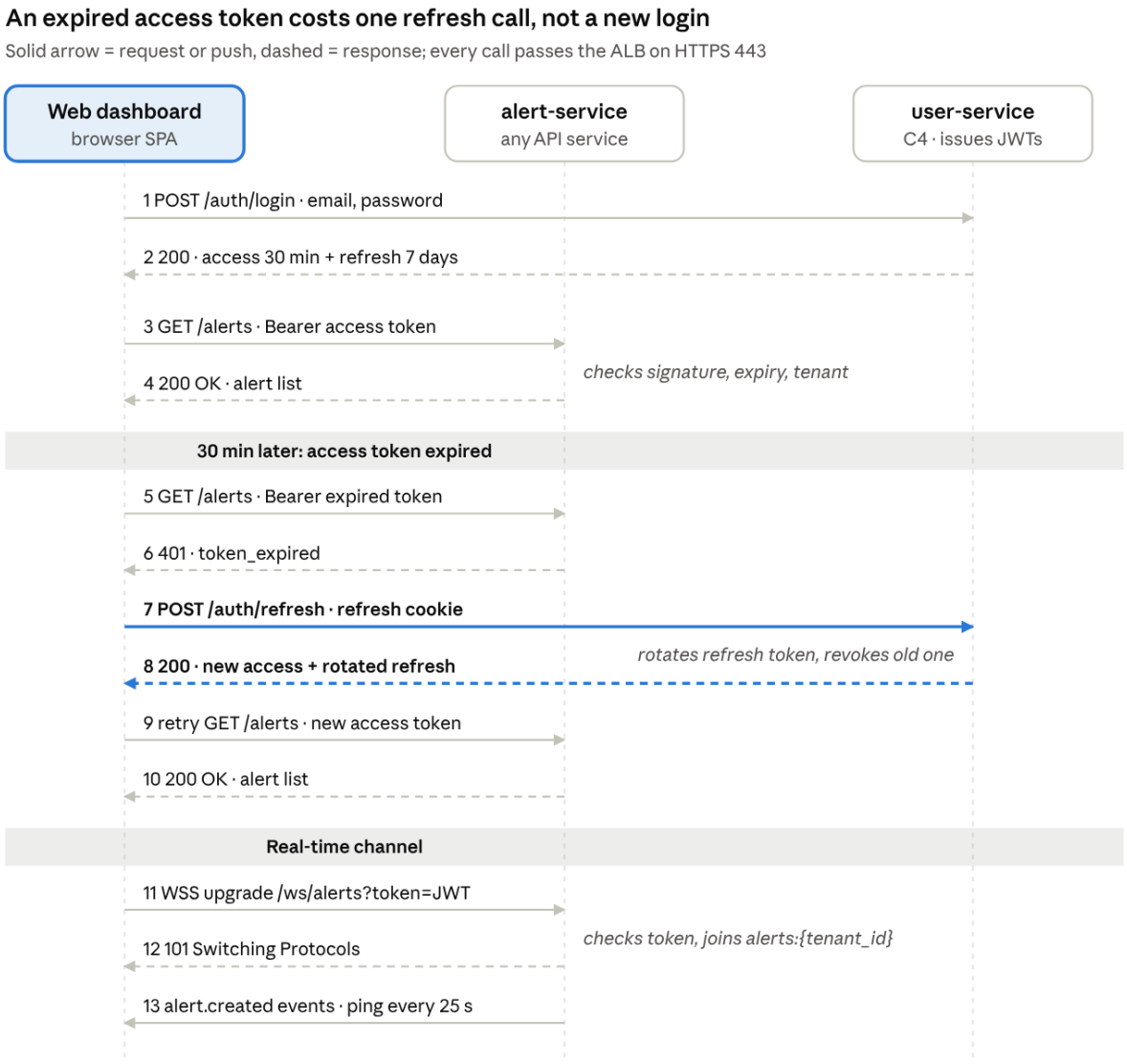


Figure 16: Authentication sequence: login, token refresh after a 401, WebSocket connect

A 401 triggers one silent refresh and a retry; only a rejected refresh token sends the user back to the Login page.

Caller	Credential	Check
Dashboard user	Access JWT (30 min) + refresh token (7 days); claims `{sub, role, tenant_id, house_ids}`	`user-service` checks the bcrypt hash and signs RS256; each service verifies signature, expiry, role and tenant with the public key, with no session store
WebSocket client	Access JWT in `?token=`	Checked once at the upgrade; closed with code 4401 at token expiry, so the client refreshes and reconnects
Edge Hub	`X-Device-Key`, 32 random bytes	`device-service` compares its SHA-256 hash with the stored hash; the device must be enabled, assigned and match the `device_id` in the call; `iot_staff` rotate a key with Rotate key on Device Detail (PUT /devices/{device_id} with rotate_key: true), and the new key is shown once
Service → AWS	IAM instance role	Short-lived credentials from the instance profile; no static keys
Service → service	Private address in App-SG; service JWT for C3 → C2 feedback	`/internal/*` has no ALB rule, and Nginx returns 404 for it on the public listener
External recipient	Signed link token, 30 min	Allows acknowledge on one alert only

The SPA keeps the access token in memory and the refresh token in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie scoped to `/api/v1/auth`. Each refresh issues a new refresh token and revokes the old one in Redis. Nginx masks the `token` query parameter in access logs.

4.4.3 Message formats

All bodies are UTF-8 JSON. The examples follow one glass-break clip from upload to acknowledgement and reuse the same ids.

1. SQS `media-analysis` — `device-service` → `ai-worker`, after the clip lands in S3:

```
{
  "message_type": "media.uploaded",
  "media_id": "4f7a2c9e-8b1d-4c3e-a5f6-9e0d1c2b3a4f",
  "tenant_id": "7c1e5a90-3b2d-4f6e-8a19-2d4c6e8f0a1b",
  "house_id": "5d0c8a7e-2f4b-4a19-b3e6-8c7d6e5f4a3b",
  "room_id": "a3f19c2d-6e4b-4d8a-9f1e-3b5c7d9e1f20",
  "device_id": "e8b4d2c6-1a3f-4e5d-b7c9-0d2e4f6a8b1c",
  "media_kind": "audio",
  "s3_bucket": "homeai-media",
  "s3_key": "tenants/7c1e5a90-3b2d-4f6e-8a19-2d4c6e8f0a1b/houses/5d0c8a7e-2f4b-4a19-b3e6-
  ↪  8c7d6e5f4a3b/devices/e8b4d2c6-1a3f-4e5d-b7c9-0d2e4f6a8b1c/2026/10/03/4f7a2c9e-8b1d-
  ↪  4c3e-a5f6-9e0d1c2b3a4f.wav",
  "duration_s": 5.0,
  "captured_at": "2026-10-03T18:40:41Z"
}
```

2. SQS `detection-results` — `ai-worker` → `alert-service` rule engine:

```
{
  "message_type": "detection.completed",
  "detection_id": "6711f3a2c4e8b91d2f0a7c55",
  "media_id": "4f7a2c9e-8b1d-4c3e-a5f6-9e0d1c2b3a4f",
  "tenant_id": "7c1e5a90-3b2d-4f6e-8a19-2d4c6e8f0a1b",
  "house_id": "5d0c8a7e-2f4b-4a19-b3e6-8c7d6e5f4a3b",
  "room_id": "a3f19c2d-6e4b-4d8a-9f1e-3b5c7d9e1f20",
  "device_id": "e8b4d2c6-1a3f-4e5d-b7c9-0d2e4f6a8b1c",
  "model": {"name": "yamnet", "version": "1.0.0"},
  "top_label": "glass_break",
  "top_confidence": 0.91,
  "labels": [{"label": "glass_break", "confidence": 0.91}, {"label": "thud",
  ↪  "confidence": 0.18}],
  "detected_at": "2026-10-03T18:40:50Z"
}
```

3. WebSocket `alert.created` — `alert-service` → every open dashboard of the tenant:

```
{
  "event": "alert.created",
  "tenant_id": "7c1e5a90-3b2d-4f6e-8a19-2d4c6e8f0a1b",
  "data": {
    "alert_id": "9b2f4c1e-7d3a-4e8b-a6c5-0f1e2d3c4b5a",
    "alert_type": "glass_break",
    "severity": "high",
    "status": "open",
    "house_id": "5d0c8a7e-2f4b-4a19-b3e6-8c7d6e5f4a3b",
    "room_id": "a3f19c2d-6e4b-4d8a-9f1e-3b5c7d9e1f20",
    "detection_id": "6711f3a2c4e8b91d2f0a7c55",
    "created_at": "2026-10-03T18:40:51Z"
  }
}
```

4. SQS `notifications` — `alert-service` → `notifier`, one message per contact and channel:

```
{
  "message_type": "alert.notify",
  "alert_id": "9b2f4c1e-7d3a-4e8b-a6c5-0f1e2d3c4b5a",
  "severity": "high",
  "channel": "sms",
  "recipient": "+14085550142",
  "template": "alert_created_sms"
}
```

5. REST request — the house owner acknowledges the alert:

```
POST /api/v1/alerts/9b2f4c1e-7d3a-4e8b-a6c5-0f1e2d3c4b5a/acknowledge HTTP/1.1
Host: homeai.example.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
X-Request-ID: 1f6e2b9c-4a7d-4e3f-9b8a-7c6d5e4f3a2b

{"note": "Called Mom. She dropped a glass in the kitchen; no injury."}
```

6. REST response `200 OK` — a single object returns `data` only; the change is also pushed as `alert.updated`:

```
{
  "data": {
    "alert_id": "9b2f4c1e-7d3a-4e8b-a6c5-0f1e2d3c4b5a",
    "alert_type": "glass_break",
    "severity": "high",
    "status": "acknowledged",
    "house_id": "5d0c8a7e-2f4b-4a19-b3e6-8c7d6e5f4a3b",
    "acknowledged_by": "c2e9a4b7-1d6f-4e8a-9c3b-5a4f3e2d1c0b",
    "acknowledged_at": "2026-10-03T18:42:07Z",
    "note": "Called Mom. She dropped a glass in the kitchen; no injury."
  }
}
```

7. List response — `GET /api/v1/alerts?status=open&page=1&page_size=20` adds `meta`:

```
{
  "data": [
    {"alert_id": "9b2f4c1e-7d3a-4e8b-a6c5-0f1e2d3c4b5a", "alert_type": "glass_break",
    ↪  "severity": "high", "status": "open", "created_at": "2026-10-03T18:40:51Z"},
    {"alert_id": "1d4b7e2a-9c3f-4a6e-b8d1-5f2e3a4b6c7d", "alert_type": "device_offline",
    ↪  "severity": "medium", "status": "open", "created_at": "2026-10-03T17:12:30Z"}
  ],
  "meta": {"page": 1, "page_size": 20, "total": 2}
}
```

8. Error envelope `409 Conflict` — acknowledging an alert that is already resolved:

```
{
  "error": {
    "code": "invalid_status_transition",
    "message": "Alert is resolved and can no longer be acknowledged.",
    "details": {"alert_id": "9b2f4c1e-7d3a-4e8b-a6c5-0f1e2d3c4b5a", "current_status":
    ↪  "resolved"}
  }
}
```

4.4.4 Communication conventions

All four services follow the same twelve rules, so the SPA and the Edge Hub each need only one API client.

Convention	Rule	Enforced by
Versioning	Every path starts with `/api/v1`; additive fields stay in v1, and breaking changes ship as `/api/v2` beside it	FastAPI router prefix; ALB path rules
Encoding	JSON in UTF-8 with `Content-Type: application/json`; binary media only on the S3 path	Pydantic v2 models
Field names	snake_case for JSON fields and query parameters; lowercase enum values such as `false_positive`	Pydantic v2 models

Convention	Rule	Enforced by
Timestamps	ISO-8601 in UTC with `Z`, e.g. `2026-10-03T18:40:51Z`; the dashboard shows the house's local time	Pydantic serializers
Pagination	`page` from 1 and `page_size` (default 20, max 100); `meta.total` gives the count; time series such as `/telemetry` use `from` and `to` instead	Shared FastAPI dependency
Idempotency	Creating POSTs take an `Idempotency-Key` header; the first response is kept 24 h and replayed. `ai-worker` skips a `media_id` already `analyzed`; `alert-service` ignores a repeated `detection_id`	Redis key per tenant and key; consumer checks in MongoDB and PostgreSQL
Errors	`{error:{code, message, details}}` with 400, 401, 403, 404, 409, 422, 429 or 500; `code` is a stable snake_case string	Global FastAPI exception handlers
CORS	Only `https://homeai.example.com` (plus `http://localhost:5173` in development); credentials allowed for the refresh cookie	FastAPI CORS middleware
Rate limits	100 requests/s per tenant; beyond that HTTP 429 with `Retry-After` in seconds, which the SPA and Edge Hub honour	Redis token bucket middleware
Compression	gzip on JSON responses over 1 KB when the client sends `Accept-Encoding: gzip`	Nginx
Tracing	`X-Request-ID` is set when absent, returned on every response, logged on every line and copied into the SQS attribute `request_id`; one CloudWatch Logs Insights query follows a clip end to end	Nginx `$request_id`; JSON logger
Transport security	HTTPS and WSS only; port 80 redirects to 443; TLS 1.2+	ALB listener with ACM certificate

5 System Dashboard UI Design

HomeAI Cloud has one web dashboard: a React 18 single-page app (`web-dashboard`, owned by C4) with three role-based workspaces for `house_owner`, `iot_staff` and `cloud_staff`. All three share one layout, one design system and one live alert channel, the `/ws/alerts` WebSocket, so a new alert reaches every open screen without a reload. REST paths in this section sit under `/api/v1`.

5.1 System User Dashboard Architecture

The SPA has five layers: a router with role guards, a shared app shell, four feature modules that map one-to-one onto C1–C4, shared UI and state, and two transport clients. Feature modules never call the network directly; they use per-module hooks over one API client, so token refresh and error envelopes are handled in one place.

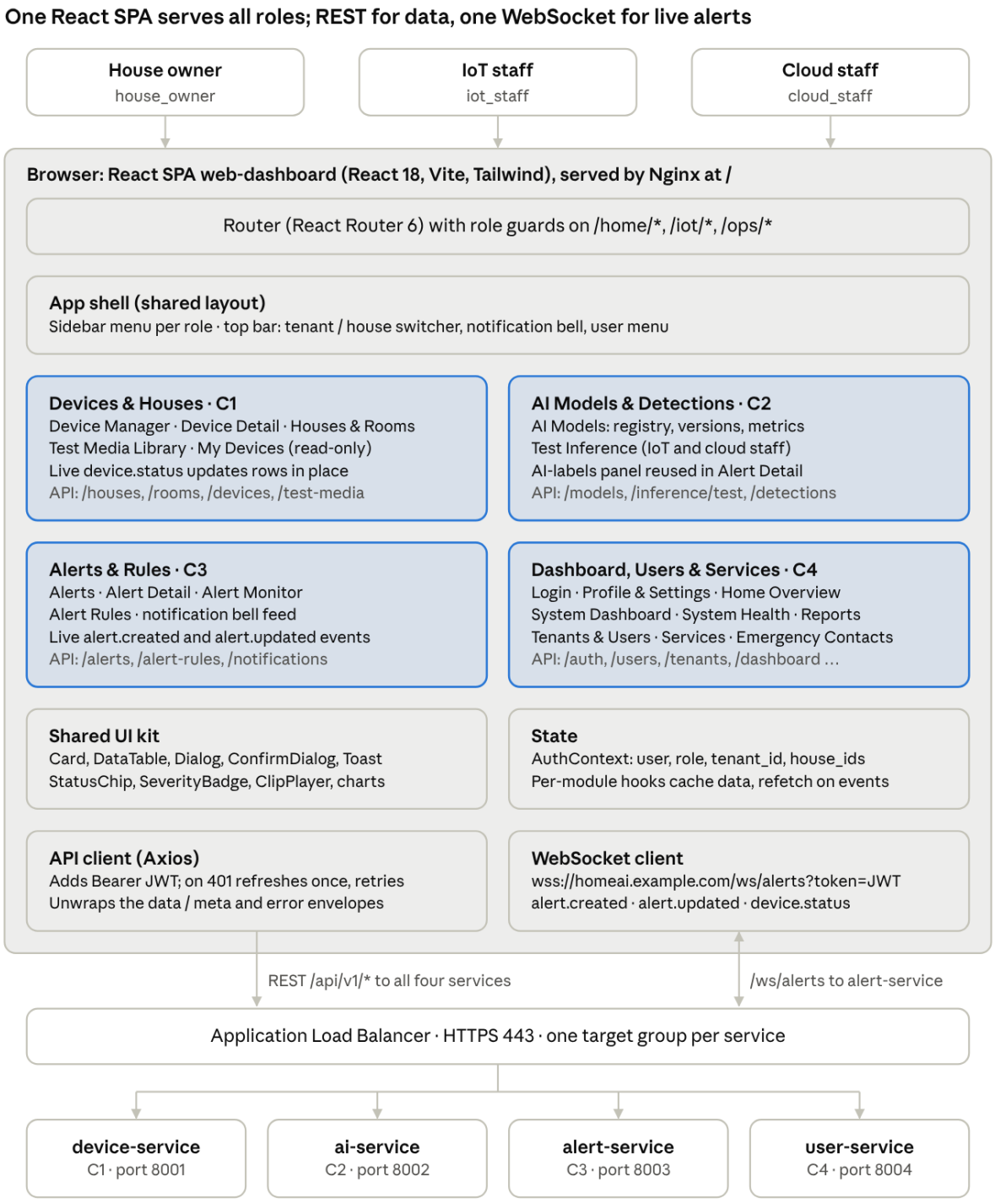


Figure 17: Web dashboard architecture: 5 layers, 4 feature modules, 2 transport clients

Each feature module owns its component's pages and API paths, so each member builds and tests their pages against their own service.

Each workspace has its own route prefix: `/home/*` for `house_owner`, `/iot/*` for `iot_staff` and `/`

`ops/*` for `cloud_staff`. A guard reads the JWT `role` claim before rendering and sends a user who opens another role's URL to their own landing page; the server still enforces RBAC on every call. On a 401 the API client calls `POST /auth/refresh` once and retries, and a second 401 returns the user to Login.

The top-bar switcher scopes every module to one context. Owners pick one of their `house_ids`, IoT staff pick a house in their tenant, and cloud staff pick a tenant or All tenants.

The UI design system below is shared by all modules, so the same status looks the same on every page.

Element	Design rule
Severity colours	`critical` red, `high` orange, `medium` amber, `low` blue; always shown with an icon and the word, never colour alone
Device status colours	`online` green, `offline` grey, `error` red, `maintenance` amber, `provisioned` blue outline; a disabled device adds a grey Disabled tag
Typography	Inter with system-ui fallback; page title 20 px / 600, body 14 px, table text 13 px; tabular numerals in KPI tiles
Layout	Left sidebar 240 px (collapses to a 64 px icon rail), top bar 56 px, 12-column content grid with 24 px gutters
Components	Card, DataTable (sort, filter, 25 rows per page), Dialog (create and edit), ConfirmDialog (destructive or irreversible actions), Toast (5 s; critical stays until dismissed), StatusChip, ClipPlayer, Recharts charts, skeleton loaders, empty states
Responsive breakpoints	Tailwind `md` 768 px and `lg` 1024 px: below 768 px, down to 360 px wide, the sidebar becomes a drawer and tables become stacked cards; from 1024 px dashboards use two columns
Accessibility	WCAG 2.1 AA: 4.5:1 text contrast, visible focus ring, every alert action reachable by keyboard, new alerts announced through an `aria-live` region

5.2 Page and Frame Designs

The dashboard has 21 pages and panels across the three roles, listed below with the buttons and APIs each one uses. Six key frames follow as low-fidelity wireframes with sample data from one house, Maple House.

Page	Role	Purpose	Key widgets	Function buttons	APIs used
Login	all	Sign in and route by role	Email and password fields, error banner	Sign in	`POST /auth/login`, `GET /auth/me`
Profile & Settings	all	Own account and preferences	Profile form, password change, notification channel preferences	Save, Change password, Log out	`GET /auth/me`, `PUT /users/{user_id}`, `POST /auth/logout`
Notification bell (panel)	all	Live alert feed on every page; IoT staff see device status events only	Unread badge, latest events list	Open alert, View all	`WS /ws/alerts`, `GET /notifications`

Page	Role	Purpose	Key widgets	Function buttons	APIs used
Home Overview	house_owner	One-screen status of a house	Status cards, room map with device dots, live alert feed	Acknowledge (on a feed card), View all alerts	GET /dashboard/summary, GET /houses/{house_id}/rooms, GET /devices, GET /alerts
Alerts	house_owner	Find and act on past and open alerts	Filter bar (status, severity, type, date), alert table	Open, Acknowledge	GET /alerts
Alert Detail	house_owner, cloud_staff	Decide on one alert	Evidence clip player, AI labels with confidence, status timeline, notified contacts	Acknowledge, Resolve, Mark false positive, Escalate, Call contact	GET /alerts/{alert_id}, GET /alerts/{alert_id}/history, GET /detections/{detection_id}, POST /alerts/{alert_id}/acknowledge, /resolve, /false-positive, /escalate
My Devices	house_owner	Check device health (read-only)	Device cards grouped by room, status chip, last heartbeat	Filter by room	GET /devices, GET /devices/{device_id}/status
Services & Subscription	house_owner	Choose plan and detection services	Plan card, service toggles, home/away/night switch, quiet-hours picker, sensitivity slider	Save, Change plan	GET /service-plans, GET/PUT /subscriptions/{house_id}, GET/PUT /service-settings/{house_id}
Emergency Contacts	house_owner	Who is notified, in what order	Ordered contact list with channels (SMS, email)	Add, Edit, Delete, Move up, Move down	GET/POST /contacts, PUT/DELETE /contacts/{contact_id}
Reports	house_owner	Export alert history	Date range, type filter, summary chart	Export CSV, Export PDF	GET /reports/alerts
Device Manager	iot_staff	Manage the tenant's virtual devices	Filter bar, device table with status chips, bulk action bar	Add Device, Edit, Enable, Disable, Assign, Delete	GET/POST /devices, PUT/DELETE /devices/{device_id}, POST /devices/{device_id}/enable, /disable, PUT /devices/{device_id}/assignment
Device Detail	iot_staff	Configure and diagnose one device	Config form with version list, heartbeat and telemetry chart, test-media playlist, Wi-Fi settings	Save config, Restore version, Edit playlist, Enable or Disable, Rotate key	GET/PUT /devices/{device_id}/config, GET /devices/{device_id}/status, GET /devices/{device_id}/telemetry, PUT /devices/{device_id}/test-media

Page	Role	Purpose	Key widgets	Function buttons	APIs used
Houses & Rooms	iot_staff	Model houses and their rooms	House list, room table with device counts	Add House, Add Room, Edit, Delete	POST/GET /houses, PUT/DELETE /houses/{house_id}, POST/GET /houses/{house_id}/rooms, PUT/DELETE /rooms/{room_id}
Test Media Library	iot_staff	Keep reusable test clips	Clip table (name, kind, duration, expected label), audio preview	Upload clip, Preview, Assign to device	POST/GET /test-media, PUT /devices/{device_id}/test-media
Test Inference	iot_staff, cloud_staff	See what the model detects in one clip	Clip picker or upload, result table (label, confidence), model version, latency	Run	POST /inference/test
System Dashboard	cloud_staff	Platform health at a glance	KPI tiles, alerts by severity and type, AI throughput and latency, queue depth, service health list	Tenant filter, Time range, Open System Health	GET /dashboard/summary, GET /alerts/stats, GET /dashboard/system-health
Tenants & Users	cloud_staff	Manage communities and accounts	Tenant table, user table with role chips	Add Tenant, Add User, Edit, Delete	GET/POST /tenants, GET/PUT /tenants/{tenant_id}, GET/POST /users, GET/PUT/DELETE /users/{user_id}
AI Models	cloud_staff	Control which model versions run	Model registry table, version list, metrics chart	Register version, Activate, Rollback, Deprecate, Test inference	GET/POST /models, GET/POST /models/{model_id}/versions, POST /models/{model_id}/versions/{version_id}/activate, /deprecate, GET /models/{model_id}/metrics, POST /inference/test
Alert Rules	cloud_staff	Tune when alerts fire	Rule table (alert type, trigger labels, min confidence, room, cooldown, de-dup window)	Add Rule, Edit, Delete	GET/POST /alert-rules, GET/PUT/DELETE /alert-rules/{rule_id}
Alert Monitor	cloud_staff	Watch alerts across all tenants	Live alert table with tenant column, severity filter, time since opened	Open, Escalate	GET /alerts, GET /alerts/stats, WS /ws/alerts

Page	Role	Purpose	Key widgets	Function buttons	APIs used
System Health	`cloud_staff`	Check services and infrastructure	Service list with health, EC2 instances, databases, SQS queues and DLQ	Refresh	`GET /dashboard/system-health`

Login. One form serves every role; the JWT `role` claim picks the landing page, so there is no role selector.

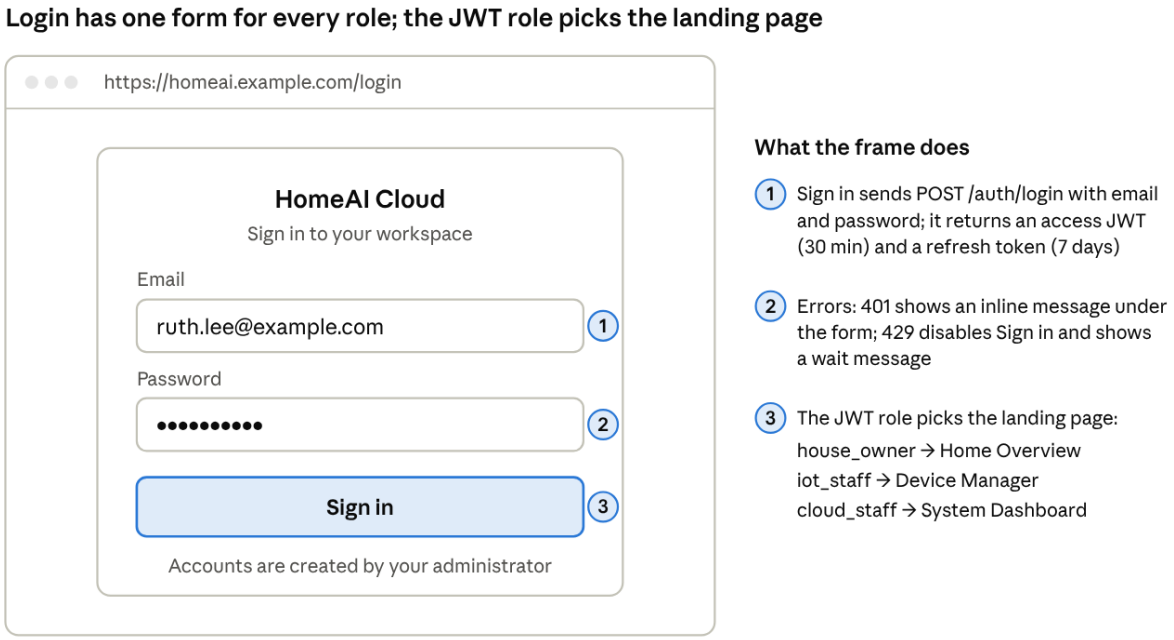


Figure 18: Wireframe 1: Login (all roles)

Sign-in failures stay on this frame; only a valid JWT leaves it, and its role decides which workspace opens.

Home Overview (`house_owner`). The owner's landing page answers one question, whether the house is safe, on one screen: status cards on top, the room map on the left and the live alert feed on the right. Acknowledging takes 1 click on a feed card and 2 from any other page through the bell.

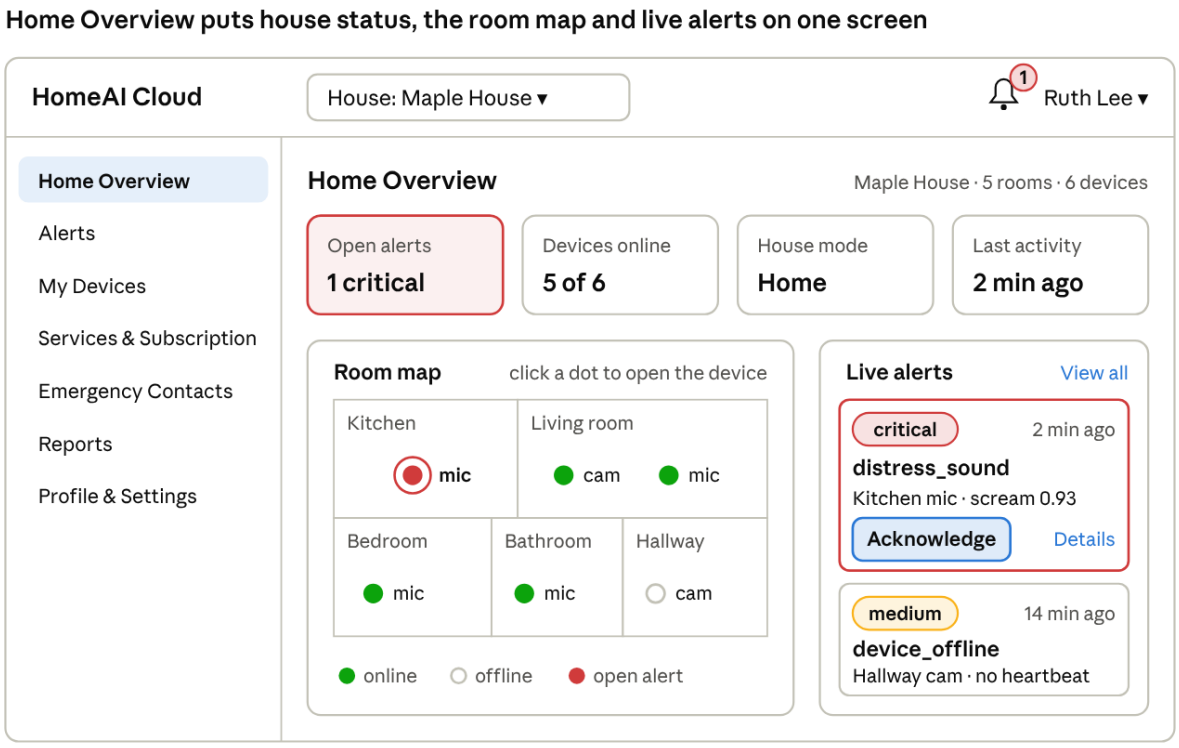


Figure 19: Wireframe 2: Home Overview (house owner landing page)

A critical alert shows in three places at once: the Open alerts card, a red ring on the Kitchen mic dot and the top of the live feed.

Alert Detail. Evidence, AI labels and actions sit on one screen so an owner can decide inside the 3-minute escalation window for critical alerts.

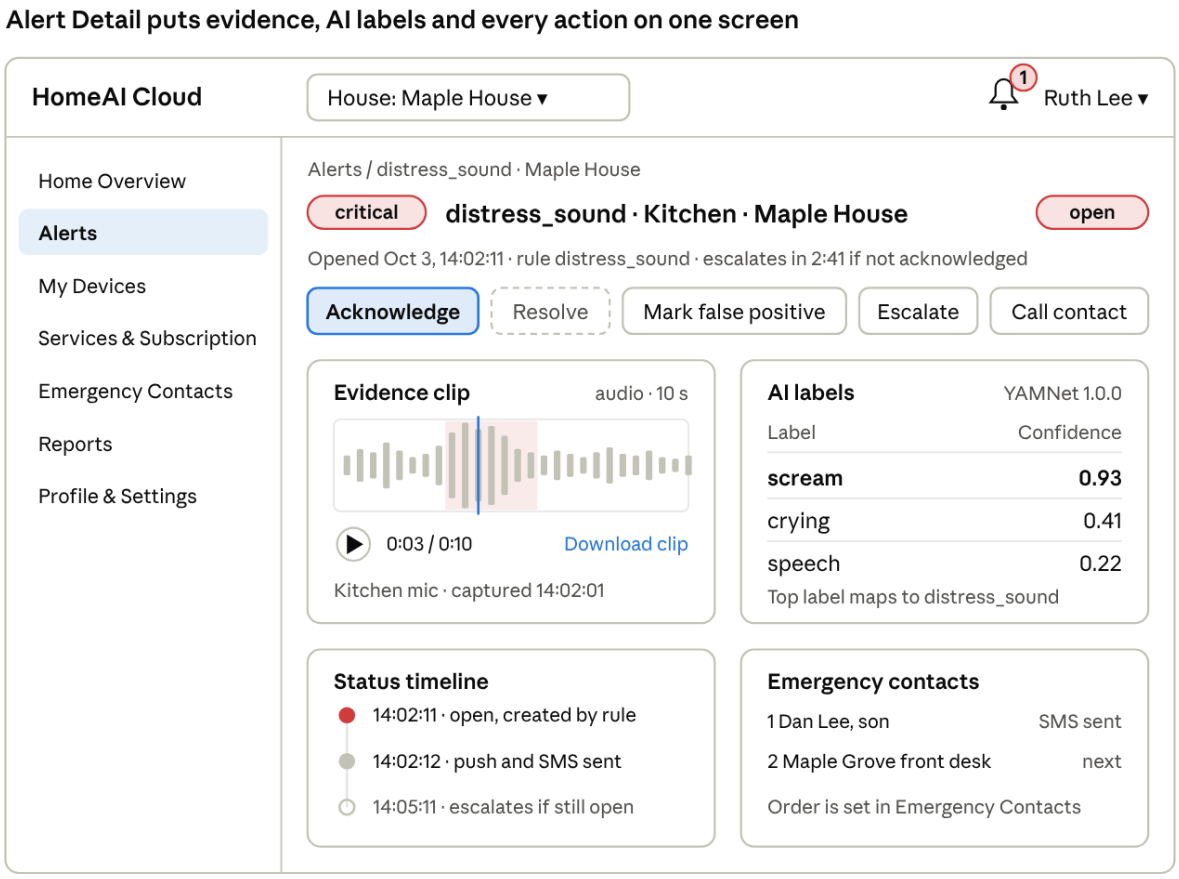


Figure 20: Wireframe 3: Alert Detail (house owner; cloud staff from Alert Monitor)

Resolve stays disabled until the alert is acknowledged, and the header countdown shows when a critical alert escalates to the next contact.

Device Manager and Add Device dialog (`iot_staff`). Bulk actions apply to the selected rows, and the Add Device dialog shows only the config fields that fit the chosen device type.

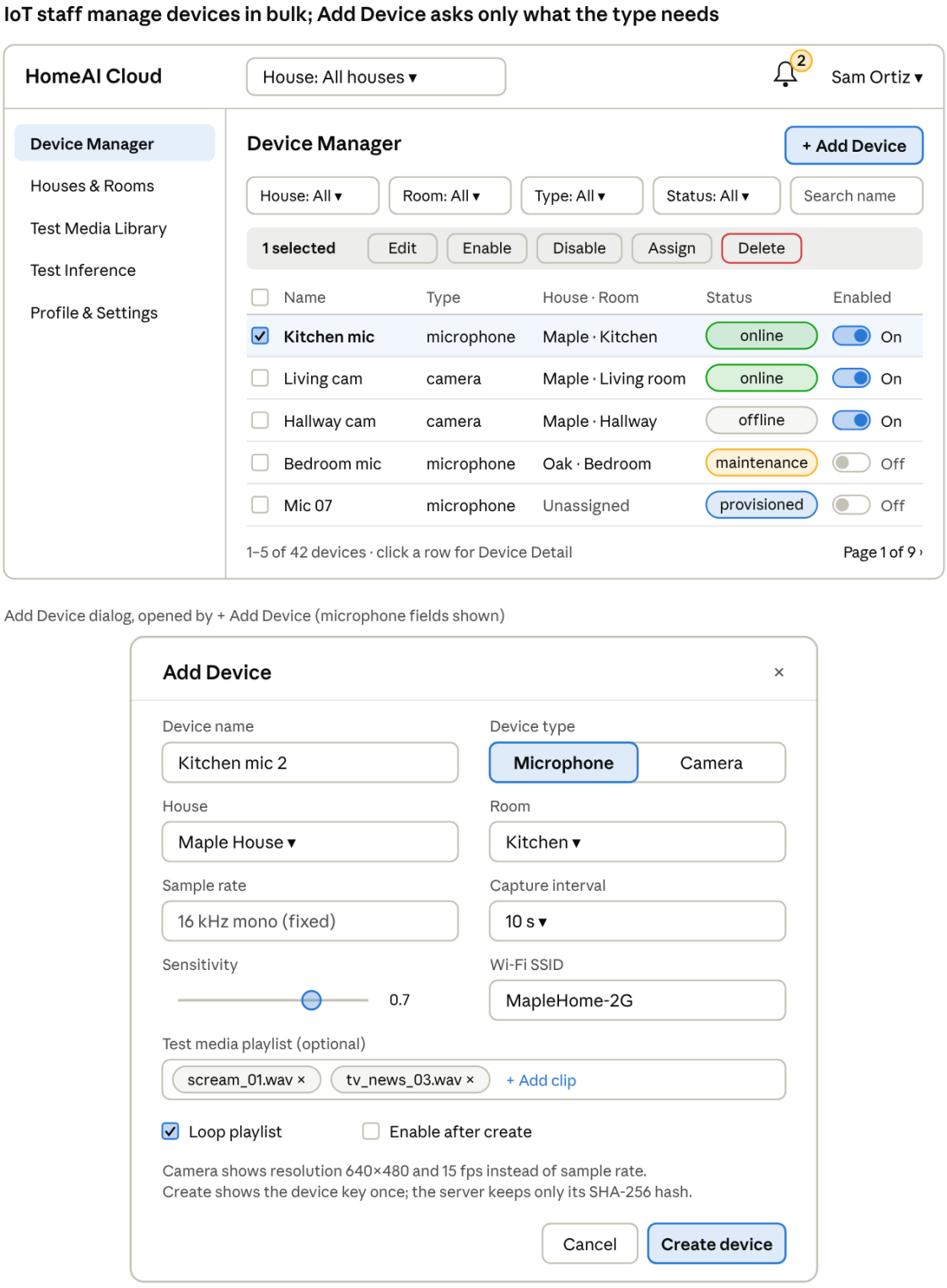


Figure 21: Wireframe 4: Device Manager and Add Device dialog (IoT staff landing page)

A new device stays `provisioned` and disabled until it has a room and is enabled; its first heartbeat turns the chip `online`.

System Dashboard (`cloud_staff`). KPI tiles track the design targets, such as alert latency ≤ 10 s p95 and queue depth, so a breach shows before users report it.

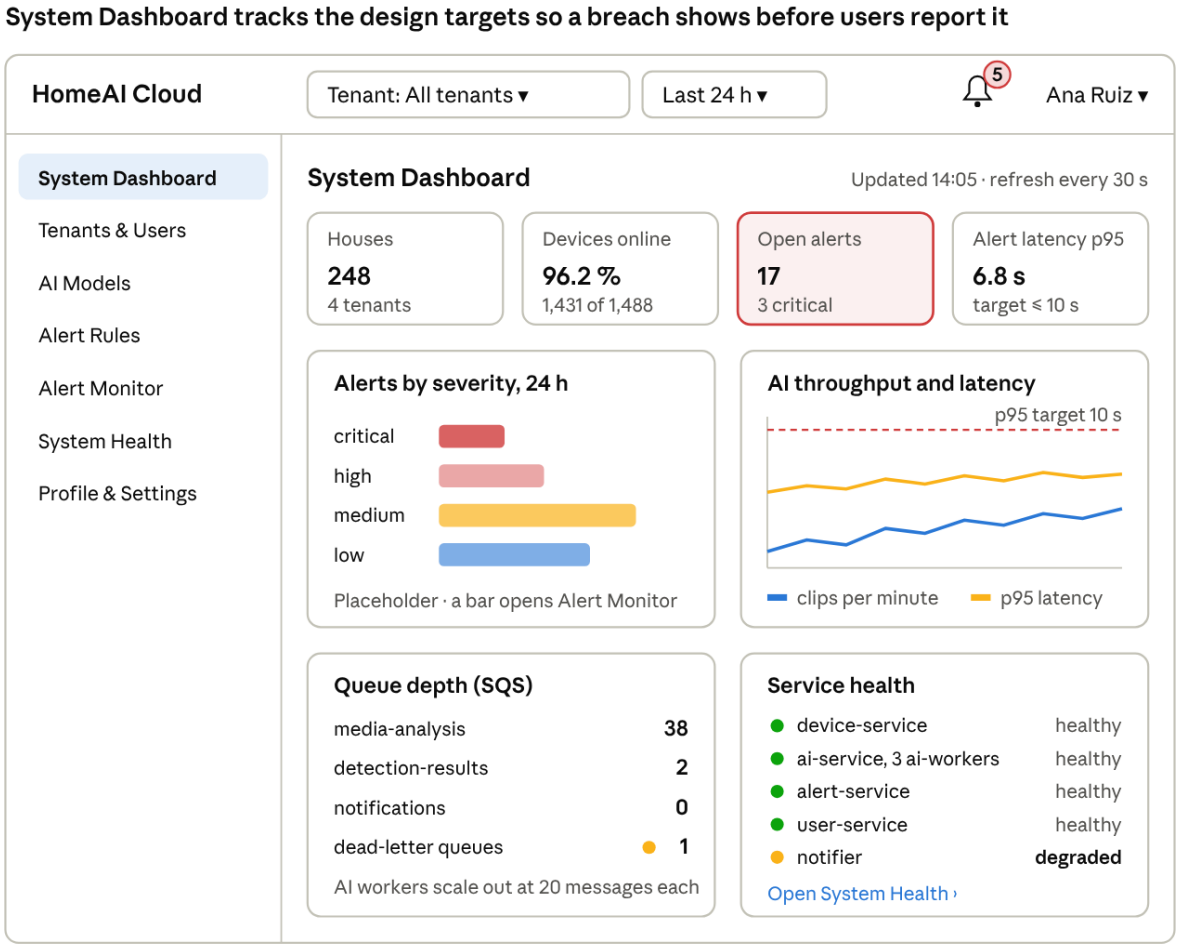


Figure 22: Wireframe 5: System Dashboard (cloud staff landing page)

Each tile and panel opens its detail page: Open alerts and the severity chart go to Alert Monitor, queue depth and service health go to System Health.

AI Models (`cloud_staff`). Activation is two steps, test inference and then Activate with a confirm dialog, and the previous active version keeps a Rollback button.

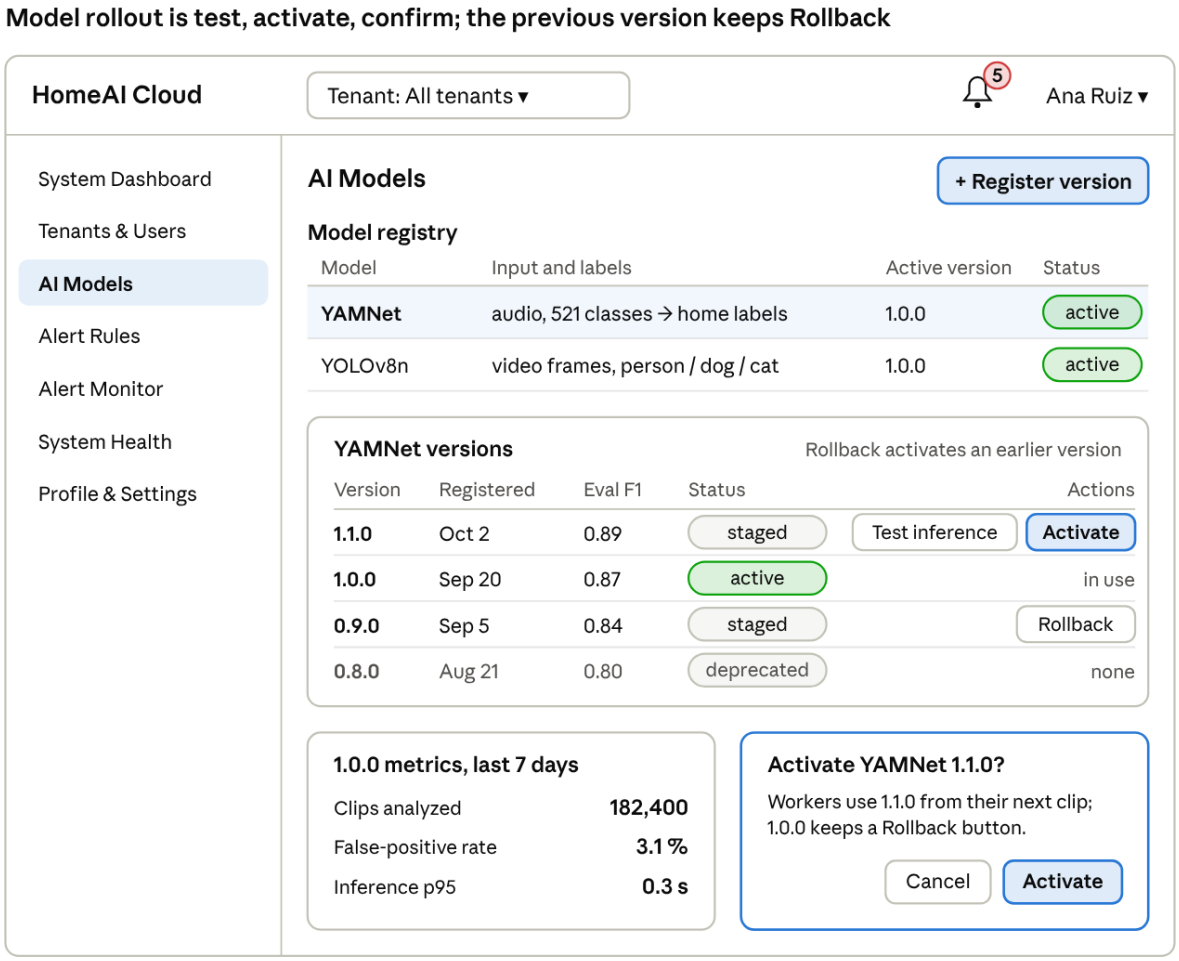


Figure 23: Wireframe 6: AI Models with the Activate confirmation dialog (cloud staff)

One version per model is `active` at a time. Activating a `staged` version returns the old one to `staged` with a Rollback button; Deprecate retires a version, and every activation or rollback is written to `audit_logs`.

5.3 Navigation Flow

Every session starts at Login and branches once, on the JWT role, into one of three workspaces. Each workspace is flat: every page is one click from the sidebar, and detail frames and dialogs sit one level below their page.

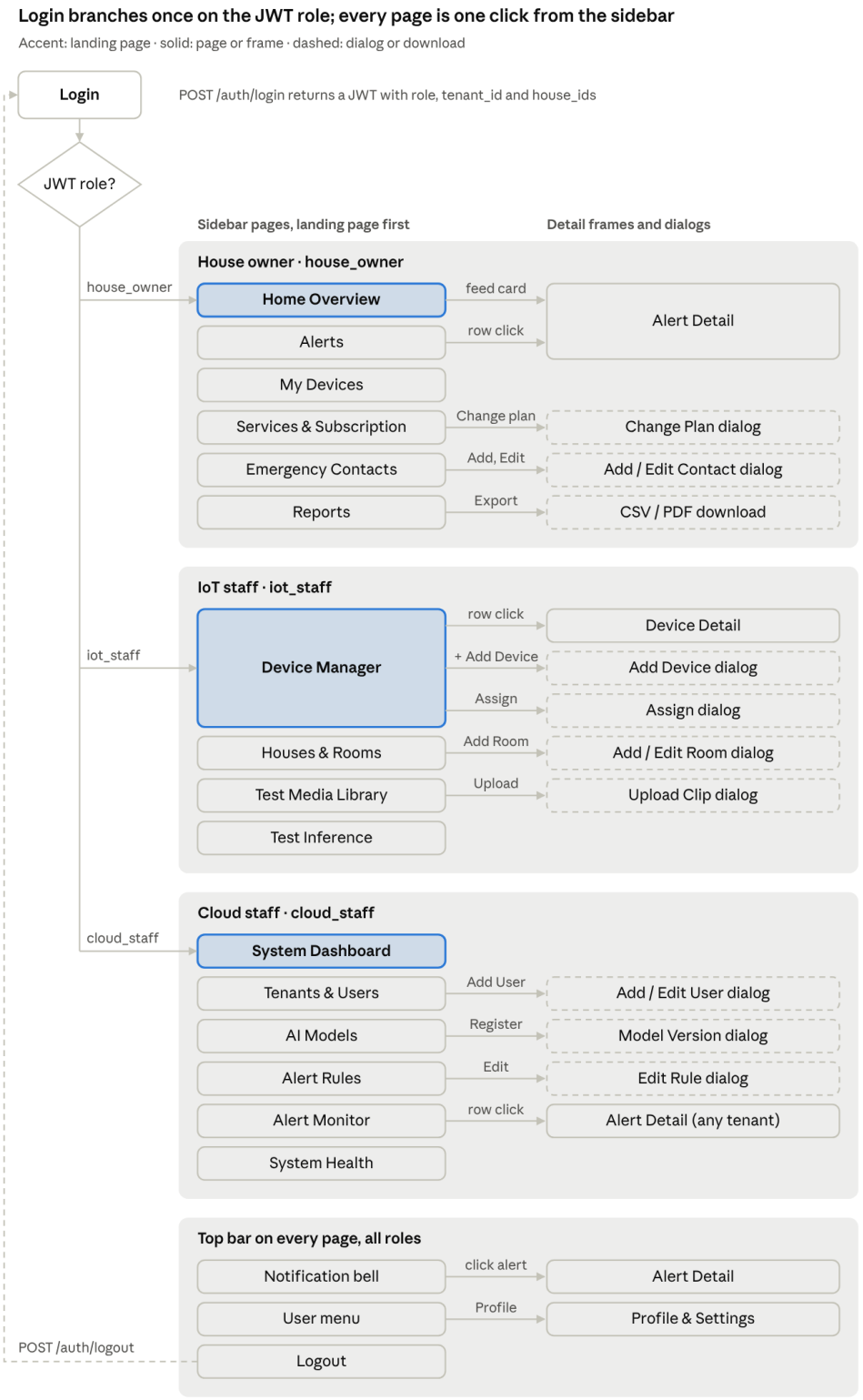


Figure 24: Navigation flow: three role lanes, 16 sidebar pages, detail frames and dialogs

No frame is more than two clicks from a landing page, and the bell opens Alert Detail from any page; a user who follows a link outside their role is sent back to their own landing page.

5.4 System UI Operation Behavior

Nine operations cover the path from sign-in to model rollout; each row gives the UI steps, the API calls behind them, what the user sees and how errors surface.

Operation	Role	UI steps	API calls	UI feedback / result	Error handling
Log in and role redirect	all	1) Enter email and password. 2) Sign in.	`POST /auth/login`, `GET /auth/me`	Spinner on the button, then redirect by `role` to Home Overview, Device Manager or System Dashboard	401: inline "Wrong email or password"; 429: wait message, button disabled
Acknowledge an alert from a push notification	`house_owner`	1) Tap the browser push, SMS link or bell item. 2) Alert Detail opens. 3) Acknowledge.	`GET /alerts/{alert_id}`, `POST /alerts/{alert_id}/acknowledge`	Chip turns `acknowledged`; timeline adds "Acknowledged by Ruth Lee"; escalation countdown stops; bell badge drops by 1	409 (someone else acted first): card reloads, banner names who acted
Mark a false positive	`house_owner`	1) Alert Detail, Mark false positive. 2) Pick a reason (TV or radio, pet, visitor, other). 3) Confirm.	`POST /alerts/{alert_id}/false-positive`; C3 then calls `POST /detections/{detection_id}/feedback`	Card turns grey and leaves the live feed; toast "Marked false positive, sent as AI feedback"	409 if the alert is already terminal: card reloads
Enable a detection service and set quiet hours	`house_owner`	1) Services & Subscription. 2) Toggle a service, for example Intrusion detection. 3) Set quiet hours 22:00–07:00 and sensitivity. 4) Save.	`GET /service-plans`, `GET/PUT /service-settings/{house_id}`; `PUT /subscriptions/{house_id}` when the plan changes	Save is enabled only after a change; toast "Settings saved"; help text states that critical alerts ignore quiet hours	422: inline field errors; service not in plan: Change plan dialog

Operation	Role	UI steps	API calls	UI feedback / result	Error handling
Add, configure, assign and enable a device	iot_staff	1) Device Manager, Add Device. 2) Fill name, type, house, room and config. 3) Create; the device key is shown once. 4) Assign changes later from the row. 5) Enable.	POST /devices, PUT /devices/{device_id}/config, PUT /devices/{device_id}/assignment, POST /devices/{device_id}/enable	Row appears as provisioned, then turns online on the first heartbeat through a device.status event	409: name already used in that room; 422: config out of range; Enable is disabled until a room is assigned
Associate test media and run a test inference	iot_staff	1) Device Detail, Test media tab: pick clips, order, loop flag, Save. 2) Test Inference: choose a clip, Run.	GET /test-media, PUT /devices/{device_id}/test-media, POST /inference/test	Toast "Playlist saved"; result table of labels with confidence, model version and latency	422: unsupported clip format; 500 or timeout: Retry button, last result kept
Register and activate a model version, with rollback	cloud_staff	1) AI Models, pick a model, Register version. 2) Test inference on reference clips. 3) Activate, confirm. 4) If metrics drop: Rollback on the previous version, confirm.	POST /models/{model_id}/versions, POST /inference/test, POST /models/{model_id}/versions/{version_id}/activate (rollback activates the previous version), GET /models/{model_id}/metrics	New version chip turns active; the previous version shows Rollback; audit line added under the version list	409: another activation in progress; 422: model artifact not found
Edit an alert-rule threshold	cloud_staff	1) Alert Rules, filter by tenant and alert type. 2) Edit. 3) Change min_confidence (for example 0.80 to 0.85) or cooldown. 4) Save.	GET /alert-rules, PUT /alert-rules/{rule_id}	Dialog shows the rule's scope (tenant, rooms) before Save; toast; the next detection uses the new value	422: min_confidence outside 0–1; 409: edited by someone else, dialog reloads

Operation	Role	UI steps	API calls	UI feedback / result	Error handling
Live update when a new alert arrives	all signed-in users	None; the server pushes	`WS /ws/alerts` events `alert.created`, `alert.updated`, `device.status`; on reconnect `GET /alerts`	Toast (critical stays until dismissed and plays a tone); bell badge +1; alert lists and the Home Overview feed add the card; the room dot turns red; device rows update in place	Socket drop: banner "Live updates paused, reconnecting"; retry with backoff from 1 s to 30 s, then a catch-up fetch

All error responses use the `{error:{code, message, details}}` envelope. The UI maps 422 `details` to inline field errors, reloads the record on 409, and shows a toast with Retry on 429 and 500.

The alert card in Home Overview, Alerts and Alert Monitor shows only the buttons its status allows. Its state machine mirrors the C3 alert lifecycle.

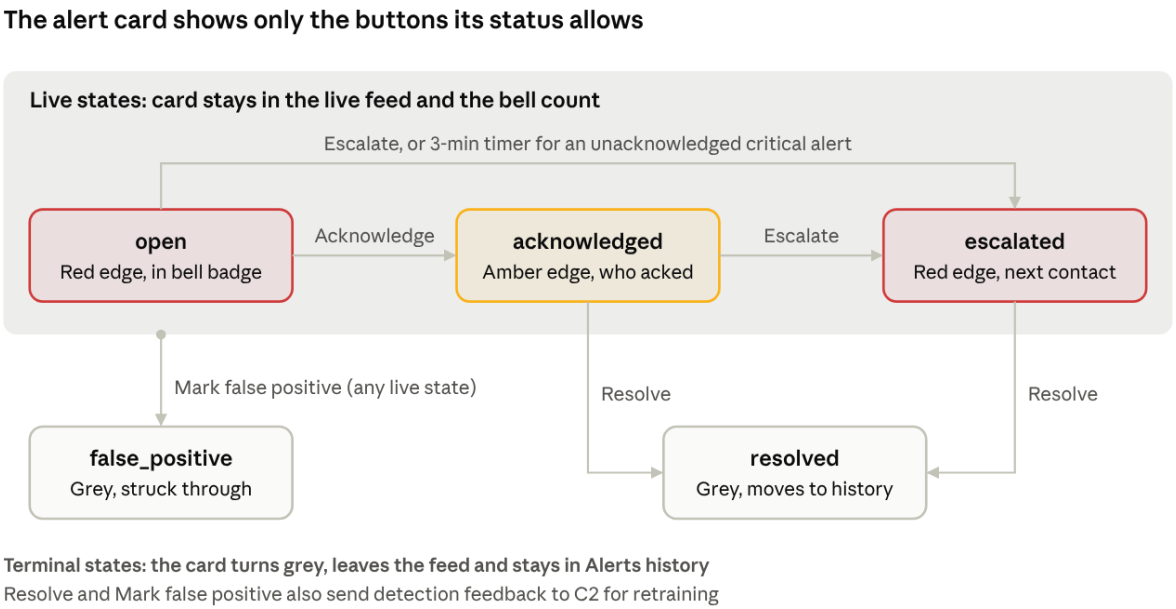


Figure 25: Alert card UI states: 5 states, 7 transitions and 1 escalation timer

Every transition is a C3 call (`/acknowledge`, `/escalate`, `/resolve`, `/false-positive`); the card changes only when the `alert.updated` event arrives, so all open screens agree.

6 Project Development Plan and Schedule

Five milestones remain: D1 design document Oct 4, 2026, D2 component design Oct 16, 2026, D3 component build Nov 6, 2026, D4 integrated demo Dec 2, 2026 and D5 final package Dec 16, 2026. D0, project selection and team building, closed on 2026-09-13. D2 and D3 are individual deliverables; D1, D4 and D5 are team deliverables.

6.1 Project Team and Roles

Each of the four members owns one component, C1–C4, end to end, plus one cross-cutting role. The owner delivers that component's UI pages, backend APIs, data structures, UML design (D2), implementation, unit tests and demo video (D3).

Member	Component and services	UI pages	APIs (`/api/v1`)	Data owned (PostgreSQL; MongoDB)	Cross-cutting role
Member 1	C1 IoT Device & House Data Manager: `device-service`	Device Manager, Device Detail, Houses & Rooms, Test Media Library, My Devices	`/houses`, `/rooms`, `/devices`, `/test-media`, `/ingest`	`houses`, `rooms`, `device_types`, `devices`, `device_configs`, `test_media`, `device_media_assignments`; `media_events`, `device_telemetry`, `device_activity_log`	Edge simulator (Edge Hub, virtual devices) and test-media library
Member 2	C2 AI Media Intelligence & Model Manager: `ai-service`, `ai-worker`	AI Models, Test Inference	`/models`, `/inference`, `/detections`	`ml_models`, `model_versions`; `ai_detections`, `activity_patterns`, `model_eval_runs`	AI model evaluation and GPU/CPU sizing
Member 3	C3 Alert Tracking, Monitoring & Notification: `alert-service`, `notifier`	Alerts, Alert Detail, Alert Rules, Alert Monitor, notification bell	`/alert-rules`, `/alerts`, `/notifications`, `/ws/alerts`	`alert_types`, `alert_rules`, `alerts`, `alert_status_history`, `notifications`; `alert_evidence`	Integration testing and QA lead
Member 4	C4 Cloud Dashboard, User Services & Data Management: `user-service`, `web-dashboard`	Login, Profile & Settings, Home Overview, Services & Subscription, Emergency Contacts, Reports, System Dashboard, Tenants & Users, System Health	`/auth`, `/users`, `/tenants`, `/service-plans`, `/subscriptions`, `/service-settings`, `/contacts`, `/dashboard`, `/reports`	`tenants`, `users`, `service_plans`, `subscriptions`, `house_service_settings`, `emergency_contacts`, `audit_logs`; `dashboard_stats_daily`	AWS/EC2 DevOps, database administration and CI/CD

The team works from one GitHub repository and freezes the shared API contract on 2026-10-16.

Practice	Rule	When
Team sync	One meeting a week (day and time to be agreed) that walks the Projects board and the critical-path tasks	Weekly, all term
Source control	One GitHub repository; protected `main`; one feature branch per issue; every pull request approved by one other member before merge	All term
Build checks	GitHub Actions runs lint and unit tests on every pull request; only a green build merges	From 2026-10-17
Issue tracking	GitHub Projects board; one issue per task (T01–T22) and per bug, each with owner and milestone	All term
API contract	Each service commits the OpenAPI spec that FastAPI generates; a change after the freeze needs approval from the consuming component's owner	Frozen 2026-10-16
Integration environment	Shared EC2 environment (course demo footprint, docker compose) deployed from `main` and run by Member 4	From 2026-11-09

6.2 Project Schedule (PERT Chart)

The plan has 22 PERT tasks. The 11-task critical path totals 104.0 expected days; with the Nov 7–8 weekend before integration it spans exactly Sep 2 to Dec 16. On 2026-10-03, T01–T06 are complete and T07 is in progress.

Durations are calendar days: optimistic (o), most likely (m) and pessimistic (p). Each stage starts the day after the previous milestone, so slack is measured to the next milestone. Planned dates round te up to whole days.

$$t_e = \frac{o + 4m + p}{6} \qquad \sigma^2 = \left(\frac{p - o}{6}\right)^2$$

ID	Task	Owner	Predecessors	o / m / p (days)	te (days)	Planned dates (2026)	Slack (days)	Critical
T01	Project selection, team building and scope (D0)	All	—	10 / 12 / 14	12.0	Sep 2 – Sep 13	0.0	Yes
T02	Requirements analysis and SRS	All	T01	5 / 7 / 9	7.0	Sep 14 – Sep 20	0.0	Yes
T03	System architecture and C1–C4 partition	All	T02	4 / 5 / 6	5.0	Sep 21 – Sep 25	0.0	Yes
T04	Database design (PostgreSQL ER, MongoDB)	Member 4	T03	4 / 6 / 8	6.0	Sep 26 – Oct 1	0.0	Yes
T05	Dashboard UI design (pages, navigation)	All	T03	3 / 5 / 7	5.0	Sep 26 – Sep 30	1.0	No
T06	Cloud infrastructure and communication design	Member 4	T03	3 / 4 / 8	4.5	Sep 26 – Sep 30	1.5	No

ID	Task	Owner	Predecessors	o / m / p (days)	te (days)	Planned dates (2026)	Slack (days)	Critical
T07	Design document assembly and review (D1)	All	T04, T05, T06	2 / 3 / 4	3.0	Oct 2 – Oct 4	0.0	Yes
T08	C1 detailed design with UML	Member 1	T07	8 / 10 / 12	10.0	Oct 5 – Oct 14	2.0	No
T09	C2 detailed design with UML	Member 2	T07	9 / 12 / 15	12.0	Oct 5 – Oct 16	0.0	Yes
T10	C3 detailed design with UML	Member 3	T07	8 / 11 / 13	10.8	Oct 5 – Oct 15	1.2	No
T11	C4 detailed design with UML; OpenAPI contract freeze	Member 4	T07	9 / 11 / 14	11.2	Oct 5 – Oct 16	0.8	No
T12	AWS environment setup (VPC, EC2, RDS, S3, SQS, Atlas)	Member 4	T07	6 / 10 / 14	10.0	Oct 5 – Oct 14	2.0	No
T13	Edge simulator (Edge Hub, virtual devices, test-media playback)	Member 1	T08	4 / 6 / 8	6.0	Oct 17 – Oct 22	15.0	No
T14	C1 implementation, unit tests, demo video	Member 1	T08, T12	11 / 14 / 18	14.2	Oct 17 – Oct 31	6.8	No
T15	C2 implementation, unit tests, demo video	Member 2	T09, T12	16 / 21 / 26	21.0	Oct 17 – Nov 6	0.0	Yes
T16	C3 implementation, unit tests, demo video	Member 3	T10, T12	15 / 19 / 25	19.3	Oct 17 – Nov 5	1.7	No
T17	C4 implementation, unit tests, demo video	Member 4	T11, T12	15 / 20 / 24	19.8	Oct 17 – Nov 5	1.2	No
T18	Integration C1 → C2 → C3 → C4 on EC2	All (lead Member 3)	T13–T17	7 / 10 / 13	10.0	Nov 9 – Nov 18	0.0	Yes
T19	End-to-end and load testing (alert ≤ 10 s p95)	Member 3	T18	5 / 7 / 9	7.0	Nov 19 – Nov 25	0.0	Yes
T20	EC2 production deployment and CI/CD pipeline	Member 4	T18	3 / 5 / 9	5.3	Nov 19 – Nov 24	1.7	No
T21	Presentation and integrated demo video (D4)	All	T19, T20	5 / 7 / 9	7.0	Nov 26 – Dec 2	0.0	Yes
T22	IEEE-format report and final project package (D5)	All	T21	10 / 14 / 18	14.0	Dec 3 – Dec 16	0.0	Yes

The network shows each stage's parallel tasks in one column; arrows meet at the milestone that gates the next stage.

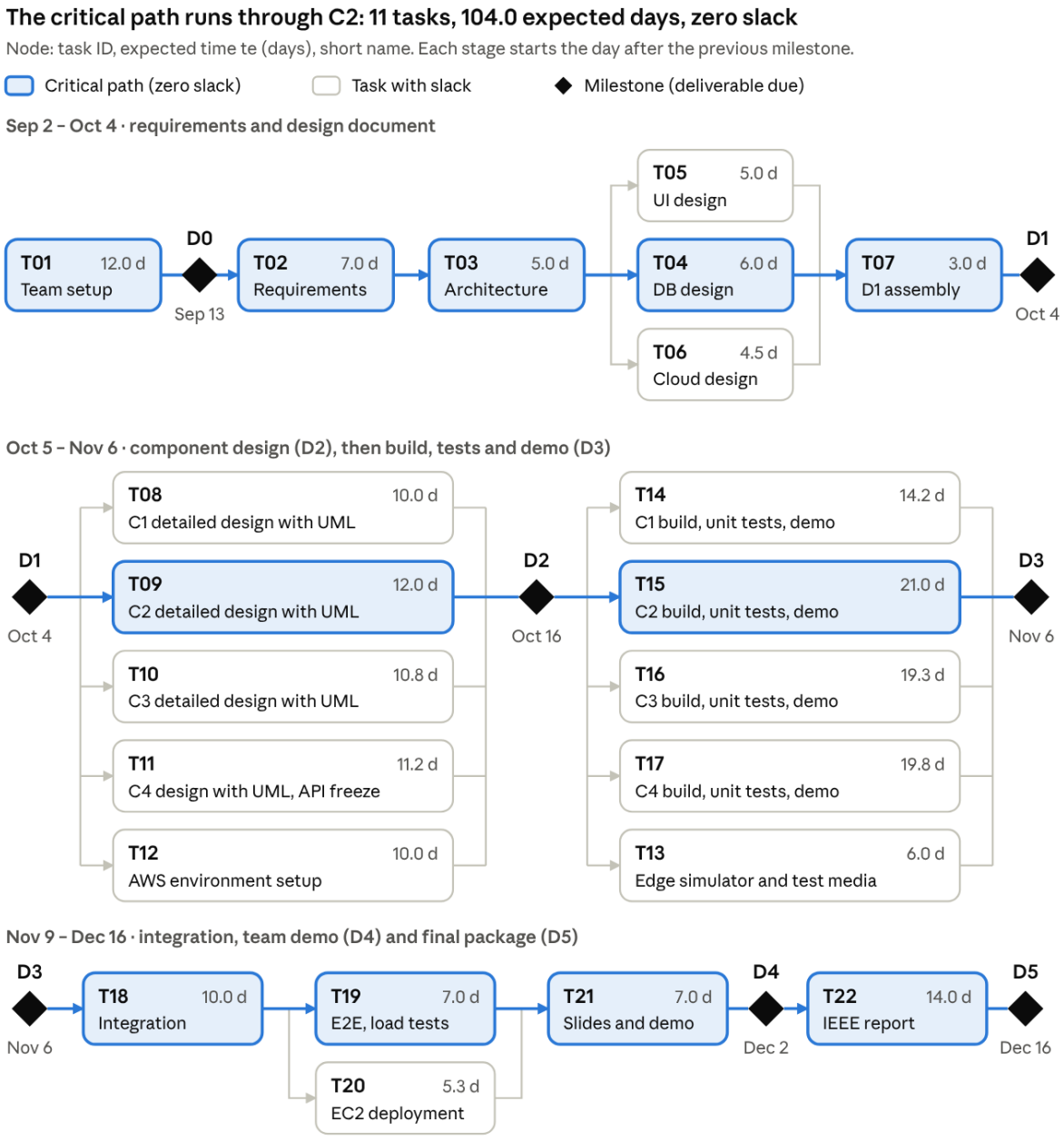


Figure 26: PERT network: 22 tasks and 6 milestones, critical path highlighted

Highlighted nodes and arrows form the critical path T01 → T02 → T03 → T04 → T07 → T09 → T15 → T18 → T19 → T21 → T22.

C2 is the only component on the critical path, through T09 and T15. The path's variance is 9.0 days^2 ($\sigma = 3.0$ days), and T15 alone adds 2.8 days^2. T11 and T17 (C4) are near-critical with 0.8 and 1.2 days of slack, so Member 4's DevOps tasks must not take time from them.

The timeline puts every task on the calendar; a thin line after a bar shows its slack to the latest finish.

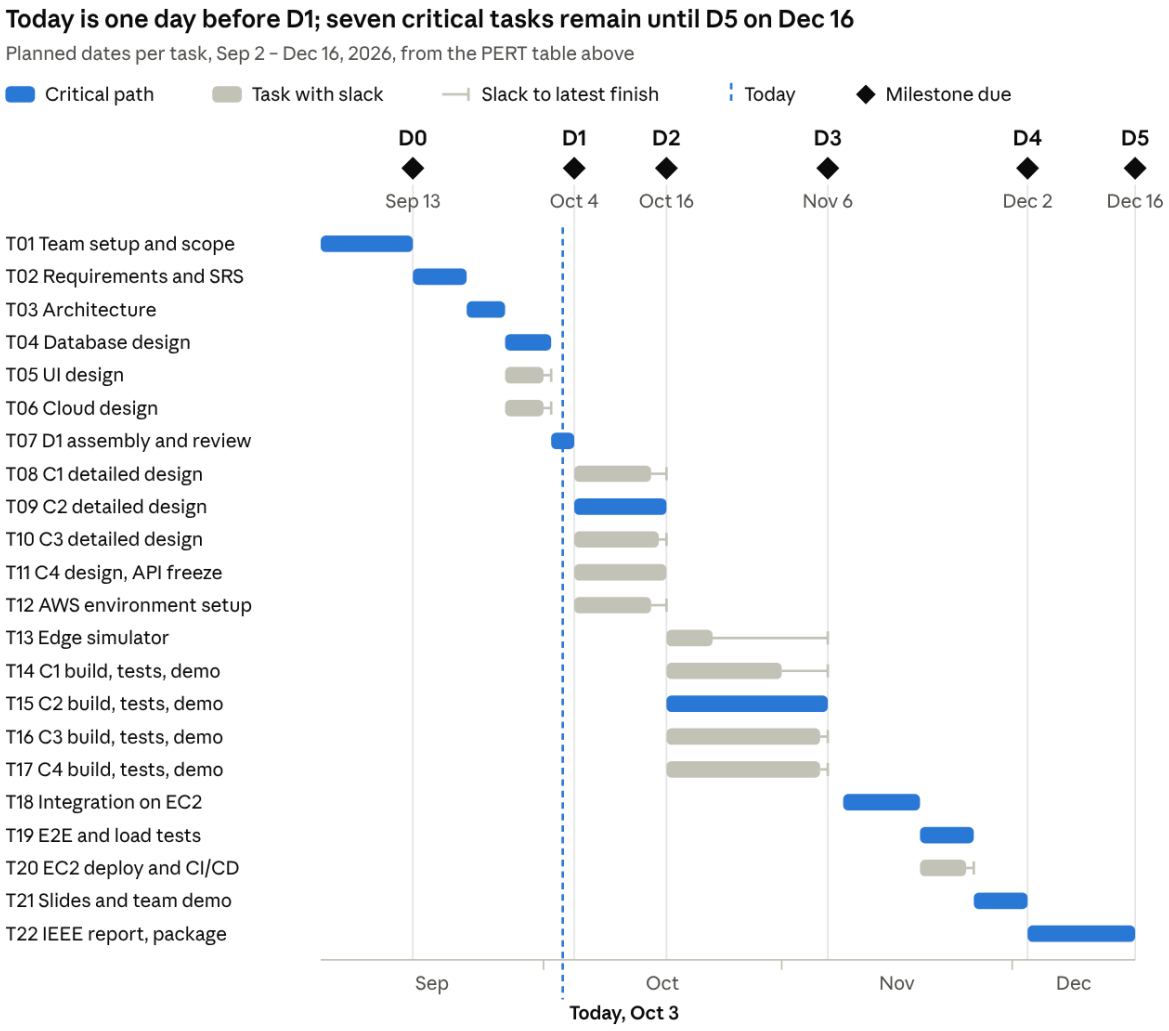


Figure 27: Project timeline drawn from the PERT table, Sep 2 – Dec 16, 2026

Every bar ends at or before its milestone line; the gap after D3 is the Nov 7–8 weekend before the integration environment opens on Nov 9.

Five risks could move a milestone; each has an owner and a planned response.

Risk	Schedule impact	Mitigation	Owner
AI inference on CPU misses the 10 s alert target	T15 (critical) slips, moving D3 and D4	Benchmark YAMNet and YOLOv8n on c6i.xlarge during T09; if slow, batch frames per clip, quantize the video model to INT8 and alert from audio first	Member 2
AWS spend exceeds the free tier and credits	Environment stopped before D4	Course demo footprint (one t3.large, RDS db.t3.micro, Atlas M0); stop EC2 outside test sessions; AWS Budgets alarm	Member 4
Components disagree at integration	T18 overruns, moving D4	OpenAPI specs and SQS message formats frozen at D2; contract tests run in CI from 2026-10-17	Member 3

Risk	Schedule impact	Mitigation	Owner
A member is unavailable (illness; Thanksgiving, Nov 26–29)	That member's tasks stop; T21 overlaps the holiday	Pair ownership: Member 1 with Member 2, Member 3 with Member 4 review each other's code and can take over; demo segments recorded during T19	All
Test clips missing for some alert labels	C2 and C3 cannot test or demo those alerts	Member 1 adds labelled clips for every alert type to the test-media library by 2026-10-22 (end of T13)	Member 1

7 References

The course materials and the instructor's papers set the document structure. The tool documentation supports the technology choices.

1. J. Gao, "Intelligent Cloud Platform for Smart Homes," CMPE 281 Option #1 project slides, San Jose State University, Fall 2026.
2. CMPE 281 Option #1 project description and Deliverable #1 assignment page, Canvas, posted 2026-09-02.
3. IEEE Software Requirements Specification template.
4. OGCIO, Effective Systems Analysis and Design Guide, Appendix C.
5. An Edge-Based Smart Mobile Service System for Illegal Dumping Detection and Monitoring in San Jose, IEEE Xplore; also on ResearchGate.
6. Building an AI-Based Surveillance Drone Cloud Platform, ResearchGate.
7. UAV-Based Transport Management for Smart Cities Using Machine Learning, ResearchGate.
8. Green Energy AI Cloud: Intelligent Community-Based Electricity Management System, ResearchGate.
9. Engineering on the Internet for Global Software Production, ResearchGate.
10. YAMNet sound classification tutorial, TensorFlow: pre-trained on AudioSet, 521 sound classes.
11. Ultralytics YOLOv8 documentation: object detection models pre-trained on COCO.
12. Smart Home Senior Care Platform, an earlier CMPE 281 project on GitHub, reviewed for comparison only.